

Иванчева Н.А., Иваньчева Т.А.

Программирование на языке С# в среде Visual Studio Net с примерами задач

Новосибирск 2013

Издание подготовлено в рамках реализации *Программы развития государственного образовательного учреждения высшего профессионального образования «Новосибирский государственный университет»* на 2009-2018 годы.

Учебное пособие «Программирование на языке С# в среде Visual Studio Net с примерами задач» соответствует Приоритетному Направлению Развития – 1 (ПНР-1) Программы Развития НИУ-НГУ. Задача 1, ПНР-1. Математика, фундаментальные основы информатики и информационные технологии. Разработка курса отвечает задаче развития образовательного процесса НГУ по следующим позициям (1) направленность на развитие профессиональных компетенций обучающихся, (2) направленность на реализацию увеличения самостоятельной работы студентов, (3) возможности формирования индивидуальных образовательных траекторий, (4) представление материала в соответствии с макетом государственного образовательного стандарта профессионального образования 3-го поколения.

Язык С# (произносится Си-Шарп) был разработан корпорацией Microsoft как современный язык программирования, предназначенный для решения совершенно новых задач. С# взял от своих предшественников, таких наиболее распространенных языков, как С++ и Java, все самое лучшее.

Язык С# использует средства системы .Net Framework, поэтому его код остается в высшей степени переносимым и может быть использован при разработке программных комплексов в многоязычной среде.

Задача данного учебного пособия научить основам программирования на С#. Для этого пособие наряду с теоретическим материалом снабжено многочисленными практическими примерами, заданиями для самостоятельной работы, контрольными вопросами. Пособие не претендует на полноту, но, ознакомившись с изложенным здесь материалом, учащийся получит полное представление об основных принципах программирования на С#. Освоив данное методическое пособие, учащийся получит необходимые знания и навыки для освоения других более сложных аспектов языка С#.

Об авторах:

Иванчева Наталья Александровна, доцент кафедры Систем информатики факультета информационных технологий НГУ.

Иваньчева Татьяна Александровна, доцент кафедры Информационных технологий Высшего колледжа информатики НГУ

Оглавление

ОСОБЕННОСТИ ЯЗЫКА C#.....	6
ОСНОВЫ C#. ЧТО ТАКОЕ NET RUNTIME?.....	7
БЫСТРОЕ НАЧАЛО. КОНСОЛЬНОЕ ПРИЛОЖЕНИЕ.....	7
АНАЛИЗ ПЕРВОЙ ПРОГРАММЫ	9
КОММЕНТАРИИ В КОДЕ ПРОГРАММЫ.....	9
ВВОД ИНФОРМАЦИИ. МЕТОД READLINE КЛАССА CONSOLE	10
ПРАВИЛА НАПИСАНИЯ ОПЕРАТОРОВ	12
ИСПОЛЬЗОВАНИЕ ОТСТУПОВ В КОДЕ ПРОГРАММЫ.....	13
КОНТРОЛЬНЫЕ ВОПРОСЫ	13
ЗАДАНИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ	13
ТИПЫ ДАННЫХ И ОПЕРАТОРЫ.....	14
ПЕРЕМЕННЫЕ И КОНСТАНТЫ	14
ИМЕНОВАНИЕ. ИДЕНТИФИКАТОРЫ	14
ОПЕРАТОР ПРИСВАИВАНИЯ.....	15
ТИПЫ ДАННЫХ C#.....	15
КОНТРОЛЬНЫЕ ВОПРОСЫ	18
ЗАДАНИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ	18
ФОРМАТИРОВАНИЕ ВЫВОДА.....	19
СОСТАВНЫЕ ОПЕРАТОРЫ ПРИСВАИВАНИЯ	19
ПРЕОБРАЗОВАНИЕ ТИПА В ОПЕРАЦИЯХ ПРИСВАИВАНИЯ	20
КОНТРОЛЬНЫЕ ВОПРОСЫ	21
ЗАДАНИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ	21
ОПЕРАТОРЫ ВЕТВЛЕНИЙ: IF И SWITCH	22
КОНТРОЛЬНЫЕ ВОПРОСЫ	24
ЗАДАНИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ	24
ОБЪЕКТНО-ОРИЕНТИРОВАННЫЙ ПОДХОД В ПРОГРАММИРОВАНИИ.....	24
ОСНОВНЫЕ ПОНЯТИЯ ООП.....	25
ОПРЕДЕЛЕНИЕ КЛАССА В C#	25
ОСНОВНЫЕ ПРИНЦИПЫ ООП.....	27
Инкапсуляция.....	27
Наследование.....	28
Полиморфизм	30
СТАТИЧЕСКИЕ ДАННЫЕ И МЕТОДЫ.....	31
ЗАДАНИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ	32
КОНТРОЛЬНЫЕ ВОПРОСЫ	32
СРЕДА РАЗРАБОТКИ VISUAL STUDIO NET	32
СОЗДАНИЕ ПРИЛОЖЕНИЯ WINDOWS FORMS	33
РАБОТА С ФОРМАМИ	34
СОЗДАНИЕ ФОРМЫ	34
ОСНОВНЫЕ СВОЙСТВА И МЕТОДЫ ФОРМЫ	36
СПОСОБНОСТЬ ФОРМ ССЫЛАТЬСЯ ДРУГ НА ДРУГА	38
КОНТРОЛЬНЫЕ ВОПРОСЫ	39
ЭЛЕМЕНТЫ УПРАВЛЕНИЯ ФОРМ.....	40
ЭЛЕМЕНТ LABEL.....	40
ЭЛЕМЕНТ TEXTBOX	41
ЭЛЕМЕНТ BUTTON	41
КЛАСС MESSAGEBOX.....	43
СПИСКИ LISTBOX И COMBOBOX.....	45
КОНТРОЛЬНЫЕ ВОПРОСЫ	47
ЗАДАНИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ	48
ЦИКЛЫ	48
ЦИКЛ СО СЧЕТЧИКОМ FOR.....	48
ЦИКЛ С ПРЕДУСЛОВИЕМ – WHILE.....	49

ЦИКЛ С ПОСТУСЛОВИЕМ DO	50
ИСПОЛЬЗОВАНИЕ ОПЕРАТОРОВ CONTINUE И BREAK В ЦИКЛАХ	50
СЛУЧАЙНОЕ ЧИСЛО. КЛАСС RANDOM	51
КОНТРОЛЬНЫЕ ВОПРОСЫ	51
ЗАДАНИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ	52
МАССИВЫ	53
Одномерные массивы	53
Инициализация массива.....	53
Многомерные массивы.....	55
Невыровненные (ступенчатые) массивы.....	56
Использование цикла FOREACH	57
КОНТРОЛЬНЫЕ ВОПРОСЫ	57
ЗАДАНИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ	57
ОБРАБОТКА ИСКЛЮЧЕНИЙ	58
КЛАСС SYSTEM.EXCEPTION.....	58
РАБОТА СО СТРОКАМИ. КЛАССЫ STRING И STRINGBUILDER	60
КЛАСС STRING.....	60
КЛАСС STRINGBUILDER	62
КОНТРОЛЬНЫЕ ВОПРОСЫ	64
ЗАДАНИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ	64
ПОБИТОВЫЕ ОПЕРАТОРЫ AND, OR, XOR, NOT	65
ОПЕРАТОР AND	65
ОПЕРАТОР OR	67
ОПЕРАТОР XOR	67
ПОДРОБНЕЕ ОБ ЭЛЕМЕНТАХ УПРАВЛЕНИЯ ФОРМ.....	68
ФЛАЖОК - ЭЛЕМЕНТ CHECKBOX.....	68
РАДИОКНОПКА - RADIOBUTTON	68
ЭЛЕМЕНТ GROUPBOX.....	69
ЭЛЕМЕНТ PANEL.....	70
ЭЛЕМЕНТ NUMERICUPDOWN(ВЕРТУШКА)	70
ЭЛЕМЕНТ MASKTEXTBOX.....	70
ЭЛЕМЕНТ CHECKEDLISTBOX.....	71
ДОБАВЛЕНИЕ ЭЛЕМЕНТА УПРАВЛЕНИЯ ПРОГРАММНО	72
ВЫРАВНИВАНИЕ ЭЛЕМЕНТОВ УПРАВЛЕНИЯ	72
ГЛАВНОЕ МЕНЮ. ЭЛЕМЕНТ MENUSTRIP.....	72
ЭЛЕМЕНТ TOOLSTRIP	73
ЭЛЕМЕНТ DATETIMEPICKER	73
ЭЛЕМЕНТ LISTVIEW	75
РАБОТА СО СТАНДАРТНЫМИ ДИАЛОГОВЫМИ ОКНАМИ WINDOWS.....	81
<i>Диалог открытия файла, класс OpenFileDialog.....</i>	<i>81</i>
<i>Диалог выбора шрифта - класс FontDialog.....</i>	<i>83</i>
ПРИМЕРЫ РАЗРАБОТКИ НЕСЛОЖНЫХ ПРИЛОЖЕНИЙ	83
КОНВЕРТЕР ВАЛЮТ.....	84
КНОПОЧНЫЙ КАЛЬКУЛЯТОР.....	85
ЗАДАНИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ	88
РАБОТА С ФАЙЛАМИ.....	92
ЧТЕНИЕ И ЗАПИСЬ БАЙТОВ ДАННЫХ С ПОМОЩЬЮ КЛАССА FILESTREAM	93
ВВОД/ВЫВОД В СИМВОЛЬНЫЕ ФАЙЛЫ. ИСПОЛЬЗОВАНИЕ КЛАССА STREAMWRITER.....	95
ИСПОЛЬЗОВАНИЕ КЛАССА STREAMREADER	95
ПРОИЗВОЛЬНЫЙ ДОСТУП К СОДЕРЖИМОМУ ФАЙЛА	96
ПРИМЕРЫ РАБОТЫ С ФАЙЛАМИ.....	96
ЗАДАНИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ	101
ДЕЛЕГАТЫ	102
СОЗДАНИЕ ГРАФИЧЕСКИХ ОБЪЕКТОВ	104

СОЗДАНИЕ ГРАФИЧЕСКОЙ ПОВЕРХНОСТИ.....	104
1 способ. Параметр <i>PaintEventArgs</i> в событии <i>Paint</i>	104
2 способ. Метод <i>CreateGraphics</i>	104
3 способ. Создание из объекта <i>Image</i>	104
РИСОВАНИЕ И МАНИПУЛИРОВАНИЕ ПОВЕРХНОСТЯМИ И ИЗОБРАЖЕНИЯМИ	105
Карандаш – класс <i>Pen</i>	105
Нарисовать линию. Метод <i>DrawLine</i>	105
Нарисовать прямоугольник. Метод <i>DrawRectangle</i>	106
ЗАДАНИЕ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ	106
Нарисовать эллипс. Метод <i>DrawEllipse</i>	107
Кисть – класс <i>Brush</i>	107
Метод <i>FillRectangle</i> . Прямоугольник, заполненный цветом.....	108
Замкнутая фигура. Методы <i>DrawPolygon</i> , <i>FillPolygon</i> и <i>FillEllipse</i>	108
ЗАДАНИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ	111
Рисование дуг и секторов на экране	112
Метод <i>Clear</i>	113
Кривая Безье	114
Использование мыши	114
Вывод текста на экран. Метод <i>DrawString</i>	116
Работа с цветом.....	117
Прозрачность.....	117
Работа с изображениями	117
Перемещение изображения с помощью стрелок.....	118
СПИСОК ЛИТЕРАТУРЫ.....	119

Особенности языка C#

Язык C# в настоящее время один из самых продвинутых и современных языков программирования. C# (произносится *си шарп*) является объектно-ориентированным языком программирования. Он разрабатывался в 1998—2001 годах группой инженеров под руководством Андерса Хейлсберга в компании Microsoft как язык разработки приложений для платформы Microsoft .NET Framework.

Одна из причин разработки нового языка корпорацией Microsoft - это создание **компонентно-ориентированного языка** для новой платформы .NET. Другие языки были созданы до появления платформы .NET, язык же C# **создавался специально под эту платформу** и не несет с собой груза совместимости с предыдущими версиями языков. Хотя это не означает, что для новой платформы это единственный язык.

Еще одна из причин разработки компанией Microsoft нового языка программирования - **это создание альтернативы языку Java**. Как известно, реализация Java у Microsoft не была лицензионно чистой - Microsoft в присущей ей манере внесла в свою реализацию много нового от себя. Компания Sun, владелица Java, подала на Microsoft в суд, и Microsoft этот суд проиграла. Тогда Microsoft решила вообще отказаться от Java, и создать свой Java-подобный язык, который и получил название C#. Противоборство именно с Java – это еще одна из целей разработки C# (недаром C# называют еще Java-killer'ом).

C# относится к семье языков с C-подобным синтаксисом. Его синтаксис наиболее близок к C++ и Java.

Язык имеет статическую типизацию, поддерживает полиморфизм, перегрузку операторов (в том числе операторов явного и неявного приведения типа), делегаты, атрибуты, события, свойства, обобщённые типы и методы, итераторы, исключения, комментарии в формате XML.

Переняв многое от своих предшественников — языков C++, Delphi, Модула, Smalltalk и в особенности Java — C#, опираясь на практику их использования, исключает некоторые модели, зарекомендовавшие себя как проблематичные при разработке программных систем, например, C# в отличие от C++ не поддерживает множественное наследование классов (между тем допускается множественное наследование интерфейсов).

Укажем некоторые особенности языка C#, отличающие его от C++:

- По умолчанию, C# **запрещает прямое манипулирование памятью**, предоставляя взамен богатую систему типов и сборку мусора. Непосредственная работа с памятью по-прежнему доступна в специальном режиме "опасного" кода, но требует явного декларирования. Как следствие, в C# активно используется всего один оператор доступа ".".
- **Преобразования типов в C# значительно строже, чем в C++**, в частности, большинство преобразований может быть совершено только явным образом. Кроме того, все приведения должны быть безопасными (т.е. запрещены неявные преобразования с переполнением, использование целых переменных как указателей и т.п.). Естественно, это заметно упрощает анализ типов при компиляции.
- Одной из типичных ошибок в C++ было отсутствие оператора *break* при обработке одной из веток оператора *switch*. Проблема "провала" (*fall-through*) в C# решена кардинальным образом: **компилятор требует наличия явного оператора перехода** (*break* или *goto case <name>*) в любой ветке.
- В C#, как и в Java, **нет множественного наследования**, вместо него предлагается использовать реализацию нескольких интерфейсов.

Основы C#. Что такое NET Runtime?

Если перевести слова NET Runtime на русский язык, то мы получим что-то вроде "Среда выполнения". Именно в этой среде и выполняется код, получаемый в результате компиляции программы написанной на C#. NET Runtime основан не на ассемблере (т. е. не на коде, родном для процессора), а на некотором промежуточном коде. Отдаленно он напоминает виртуальную Java-машину. Только если в случае Java у нас был только один язык для виртуальной машины, то для NET Runtime таких языков может быть несколько. Теоретически программа для среды NET Runtime может выполняться под любой операционной системой, в которой NET Runtime установлена. Но на практике пока единственная платформа для этого - это Windows.

NET Runtime состоит из нескольких частей. Одна из них - это **Common Language Runtime**. Это, говоря кратко, **некоторый набор стандартов, которые должны поддерживать все языки платформы .NET**. Например, в предыдущих версиях Visual Studio была такая проблема, что разные языки по-разному хранили данные одного по идее типа. Так, скажем, тип целого в языке Visual Basic занимал два байта, а в Visual C++ четыре байта. Это порождало кучу проблем при совместном использовании языков. Так вот, **Common Language Runtime как раз в частности и определяет стандартные для всех языков .NET типы данных**. И уже есть гарантии, что целый тип в одном языке будет в точности соответствовать одноименному типу в другом.

Еще одна важная часть NET Runtime - это **набор базовых классов**. Их очень много, порядка несколько тысяч. Кроме того, эти классы относятся не к конкретному языку, а к NET Runtime. Т. е. мы получаем набор классов, общий для всех языков .NET, что достаточно удобно.

Именно NET Runtime берет на себя некоторые рутинные функции. Например, в нем организована **сборка мусора**. И если раньше программисту приходилось самому освобождать объекты, созданные динамически, то теперь эту задачу берет на себя среда NET Runtime.

Еще одно свойство среды NET Runtime - это **проверка типов**. Означает это следующее: когда программа выполняется, то в принципе некоторой функции можно передать параметр неправильного типа. Скажем вместо целого подставить действительное число. Языки типа C++ свои параметры функций не проверяют, в результате чего записанная переменная большего размера может повредить чужую область памяти, и программа может просто рухнуть. Еще классический пример на эту тему - это выход за пределы массива. В среде NET Runtime же такое невозможно. Среда NET Runtime сама позаботится о проверке типов.

Существует несколько языков для среды NET Runtime. В настоящее время это:

- Visual C#,
- Visual Basic
- Visual C++
- Visual J#.

Быстрое начало. Консольное приложение

В этом разделе мы рассмотрим несколько примеров простых программ на C# для того, чтобы быстро начать писать и отлаживать несложные программы. Для этого начнем с консольного приложения на C#. На этих простых примерах мы познакомимся:

- С основами работы в среде Visual Studio Net.
- Со структурой консольной программы.
- С организацией ввода-вывода.

- С некоторыми важными классами.

Напишем первую программу, которая будет выводить на экран слова «Привет мир!». Итак, за дело.

Для разработки приложений будем использовать среду разработки Visual Studio Net 2008 (русифицированную). Чтобы создать консольное C#-приложение необходимо:

1. Открыть меню **Файл|Создать|Проект...**
2. В открывшемся диалоге в секции **Типы проектов** выбрать **Visual C#**.
3. В секции **Шаблоны** выбрать **Консольное приложение**.
4. В поле **Имя** ввести имя приложения
5. В поле **Расположение** ввести путь к решению
6. В поле **Решение** выбрать **Новое решение**, если проект будет сохранен в новой папке и выбрать **Добавить в решение**, если проект будет сохраняться в уже созданной папке.
7. Ввести **Имя решения**, если создается новое решение (см. рис 1).

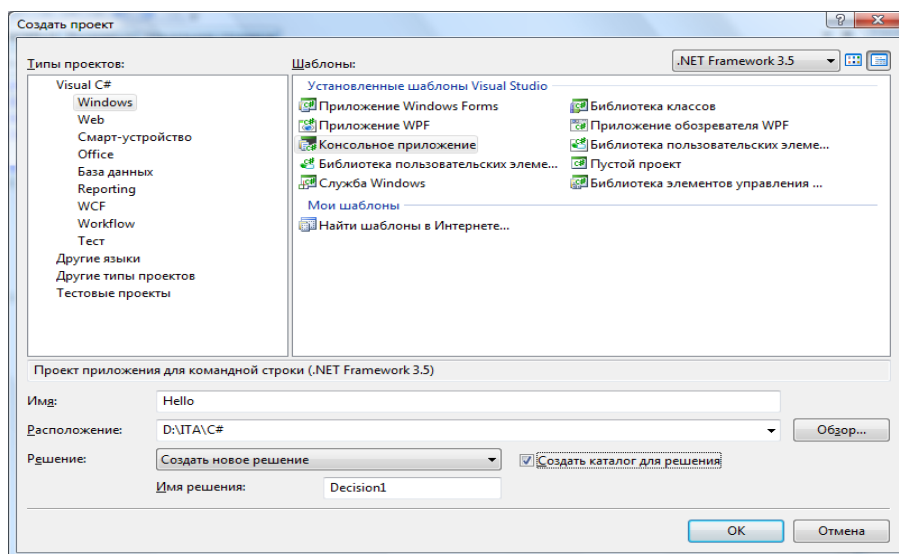


Рис. 1

В результате наших действий будет сгенерирована следующая простая программа:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace Hello
{
    class Program
    {
        static void Main(string[] args)
        {

        }

    }
}
```


Анализ первой программы

Программа C# сохраняется в файле с расширением .cs. Рассмотрим ее структуру. Ключевое слово **using** позволяет подключать пространства имен. Пространство имен обеспечивает способ хранения одного набора имен отдельно от другого:

```
using System;
using System.Collections.Generic;
using System.Text;
```

Ключевое слово **using** означает, что программа использует имена в заданном пространстве имен. Далее определяется пространство имен нашего приложения, это имя совпадает с названием папки, в которой оно и будет сохранено:

```
namespace Hello
```

Далее определяется класс **Program**, ведь все с чем мы будем иметь дело в C# - это классы или объекты, определенных классов. Класс **Program** содержит статический метод **Main**, который является точкой входа в программу. Ключевое слово **void** при описании метода говорит о том, что данный метод не возвращает значение. О том, что такое статический метод, мы поговорим несколько позже. Именно внутри этого метода будет находиться программный код, который мы должны написать. Наша программа не будет делать ничего иного, как только выводить на экран слова "Привет мир!". Итак требуемый код будет иметь вид:

```
Console.WriteLine("Привет мир!");
```

Класс **Console** предоставляет методы **Write** и **WriteLine** для вывода сообщений на экран. Отличие метода **WriteLine** от метода **Write** заключается в том, что после вывода сообщения на экран производится переход на следующую строку. Итак приведем еще раз полностью код нашей программы с комментариями:

```
namespace Hello
{
    class Program
    {
        static void Main(string[] args)    // точка входа в программу
        { // тело программы
            Console.WriteLine("Привет мир!");
        } // конец описания метода
    } // конец описания класса
} // конец программы
```

Запуск программы на выполнение осуществляется одним из следующих способов:

- Меню **Отладка** | **Начать отладку (F5)** – с отладкой.
- Меню **Отладка** | **Запуск без отладки (CTRL + F5)** – без отладки.

Комментарии в коде программы

C#, как и многие другие языки программирования, позволяет вводить комментарии в исходный код программы, при этом компилятор игнорирует содержание комментария. В комментарии описывается или объясняется операция, выполняемая в программе. В приложениях комментарии обычно объясняют, как работают определенные части программы, или какие данные содержатся в какой-либо переменной.

В С# поддерживается два вида комментариев: *однострочные* и *многострочные*. Многострочный комментарий состоит из нескольких строк и находится между символами `/*` и `*/`. Весь текст, находящийся между этими символами игнорируется компилятором. Например:

```
/* это простое
   консольное приложение
*/
```

Однострочный комментарий начинается с символов `//` и заканчивается с окончанием строки. Например:

```
int price; // переменная хранит цену товара
```

Принято общее правило: многострочный комментарий программист использует для больших заметок, а однострочные – для коротких объяснений, не требующих много места.

Ввод информации. Метод `ReadLine` класса `Console`

Рассмотрим пример посложнее. Напишем консольное приложение, которое запрашивает у пользователя имя и выводит его на экран. Для ввода воспользуемся методами класса **Console**: **Read** и **ReadLine**. Методы **Read** и **ReadLine** возвращают введенное пользователем значение в виде символьной переменной. Разница между методами заключается в том, что метод **ReadLine** выполняет еще и переход на следующую строку. Важно, что оба метода возвращают значение строкового типа. Программный код, решающий нашу задачу, приведен ниже.

```
namespace Hello
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Введите свое имя:");

            // объявление переменной s как переменной класса string,
            // т.к. метод ReadLine - возвращает строку
            string s = Console.ReadLine();
            Console.WriteLine("Здравствуйте, {0}", s);

            // следующий оператор позволяет организовать задержку, чтобы можно
            // было посмотреть результат

            Console.ReadLine();
        }
    }
}
```

Эта программа мало чем отличается от предыдущей. Новой для нас является только следующая строка:

```
Console.WriteLine("Здравствуйте, {0}", s);
```

"Здравствуйте, {0}" это строка форматирования, где `{0}` это подстановочное выражение, вместо него будет подставлен параметр, следующий за строкой форматирования, т.е. переменная `s`. Подстановочных выражений в строке форматирования может быть несколько, нумерация выражений начинается с нуля.

Рассмотрим еще один пример. Требуется ввести два целых числа и вывести их сумму на экран. Ниже приведен программный код с комментариями, решающий данную задачу:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

namespace Console2
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Введите первое число:");
            String s1 = Console.ReadLine();

            // строку s1 необходимо преобразовать в целый формат, для того,
            // чтобы выполнить операцию сложения. Это делается, например,
            // с помощью метода ToInt16 класса Convert,
            // который переводит строку в 2-х байтовое целое число.
            int a = Convert.ToInt16(s1);

            Console.WriteLine("Введите второе число:");
            String s2 = Console.ReadLine();

            int b = Convert.ToInt16(s2);
            int sum = a + b;

            Console.WriteLine("Сумма = {0}", sum);
            Console.ReadLine();
        }
    }
}
```

Метод **ReadLine** возвращает введенные данные в виде строки. Для подсчета суммы чисел необходимо выполнить преобразование введенных строк в числа. Подобные преобразования выполняются с помощью методов класса **Convert**. В табл. 1 приводятся некоторые важные методы класса **Convert**:

Таблица 1

Метод	Пояснение
ToInt	Переводит аргумент в целое число
ToDouble	Переводит аргумент в дробное число
ToChar	Переводит аргумент в символьный тип
ToString	Переводит аргумент в строку
ToInt16	Выполняет преобразование строки в число размером не более 2-х байт.
ToInt32	Выполняет преобразование строки в число размером не более 4-х байт.
ToInt64	Выполняет преобразование в число размером не более 8-ми байт.
ToSingle, ToDouble	Выполняют преобразование строки в дробное число.

В данном примере мы не учли, что вводимые числа могут быть дробными. Ниже приведен пример кода, который это учитывает:

```
Console.WriteLine("Введите 1 число:");
String s1 = Console.ReadLine();
Single a = Convert.ToSingle(s1);
```

```

Console.WriteLine("Введите 2 число:");
String s2 = Console.ReadLine();
Single b = Convert.ToSingle(s2);

Single sum = a + b;
Console.WriteLine("Сумма = {0}", sum);

Console.ReadLine();

```

Замечание 1: при вводе дробного числа разделитель целой и дробной части – точка.

Замечание 2: Все встроенные типы данных C# поддерживают статический метод *Parse*, поэтому преобразование строки в число можно выполнять еще и так:

```

Int n=int.Parse(string s);
Double k = Double.Parse("3.4");
Console.WriteLine("k={0}", k);

```

В следующем задании требуется ввести значения катетов прямоугольного треугольника и посчитать длину гипотенузы.

Решение. Для извлечения корня квадратного будем использовать класс **Math** и его статический метод *Sqrt*, который выполняет извлечение корня квадратного из аргумента. Класс **Math** инкапсулирует свойства и методы необходимые для выполнения математических операций.

```

double a, b, c;
// Длина гипотенузы
Console.WriteLine("Введите значение первого катета");
string s1 = Console.ReadLine();
a = Convert.ToDouble(s1);

Console.WriteLine("Введите значение второго катета");
s1 = Console.ReadLine();
b = Convert.ToDouble(s1);

c = Math.Sqrt(a * a + b * b);

Console.WriteLine("Длина гипотенузы = {0}", c);

Console.ReadLine();

```

Правила написания операторов

В C# символ точки с запятой означает окончание оператора. То есть каждый отдельный оператор должен заканчиваться точкой с запятой. C# не распознает окончание строки, как окончание оператора, только точка с запятой может завершить оператор.

Поэтому не имеет значения в каком месте строки вы помещаете точку с запятой. Например, для C# строки кода

```

x = y;
y = y + 1;
Console.WriteLine("x=" + x + " y=" + y);

```

будут означать тоже самое, что и строка

```

x = y; y = y + 1; Console.WriteLine("x=" + x + " y=" + y);

```

Более того, отдельные элементы оператора могут быть помещены в разные строки. Так, допустимо следующее расположение:

```

Console.WriteLine("Это очень длинный оператор " +
    "x=" + x + " y=" + y +

```

" поэтому рабиваем его на несколько строк");

Такой способ разбивки строк используется для улучшения читабельности программ. К тому же можно увидеть, как эта строка будет выглядеть при выводе.

Использование отступов в коде программы

При написании программ следует использовать отступы. Использование или отсутствие отступов не влияет на работоспособность программ, но делает программы более читабельными.

Контрольные вопросы

1. Как создать консольное приложение?
2. Какова структура консольного C#-приложения?
3. С какого метода начинается выполнение программы на C#?
4. Что такое void? Что обозначает слово static?
5. Каким образом выполняется преобразование строк в числа?
6. Как завершаются все операторы в C#? Справедливо ли утверждение, что все операторы должны начинаться и заканчиваться на одной строке?
7. Обязательно ли в C#-программе использовать оператор using System?

Задания для самостоятельной работы

Используя консольные приложения решить следующие задачи.

1. Ввести 2 вещественных числа, округлить их до целых (использовать метод Round класса Math). Результат вывести на экран
2. Ввести значение аргумента, вывести значение функции sin для этого аргумента. Следует иметь в виду, что аргумент задается в радианах. То же самое проделать с функцией cos.
3. Ввести значение текущего курса доллара. Ввести сумму в долларах. Выполнить пересчет введенного количества долларов в рубли и вывести результат. Ввести сумму в рублях. Вывести результат в долларах по текущему курсу.
4. Написать программу вычисления стоимости покупки яблок и апельсинов, при этом требуется вводить стоимость 1 кг яблок и апельсинов, а также количество купленных яблок и апельсинов. Рекомендуемый вид экрана:

Вычисление стоимости покупки

Стоимость 1 кг яблок:

Куплено яблок (кг):

Стоимость 1 кг апельсинов:

Куплено апельсинов (кг):

Всего стоимость покупки:

5. Написать программу расчета стоимости поездки на дачу на автомобиле (туда и обратно). Исходными данными являются: расстояние до дачи(км), количество бензина, которое потребляет автомобиль на 100 км пробега, цена 1 литра бензина. Рекомендуемый вид экрана:

Вычисление стоимости поездки на дачу

Расстояние до дачи (км):

Расход бензина (литров на 100 км пробега):

Цена литра бензина:

Типы данных и операторы

Переменные и константы

Переменная это данные, которое может менять свое значение в процессе выполнения программы. Константа свое значение менять не может.

Все переменные и константы, которые используются в программе, обязательно должны быть объявлены. При объявлении переменной необходимо указывать ее имя и тип. В C# переменная может быть объявлена в любом месте программы. При объявлении переменной для нее выделяется память.

Синтаксис объявления переменной:

`<тип> {<имя переменной>| <список переменных>;}`

Например:

```
int a;  
long b;  
double c, d, f;
```

Синтаксис объявления константы:

`Const <тип> <имя> = <значение>;`

Где const – ключевое слово

Например:

```
const double pi = 3.1415;  
const int Number = 42;
```

Замечание. При описании синтаксиса используются следующие обозначения:

- `<имя>` – вместо угловых скобок подставляется конкретное значение;
- `[...]` – квадратные скобки позволяют задавать необязательные элементы;
- `{...|...|...}` – фигурные скобки задают выбор одного из нескольких вариантов.

Именованное. Идентификаторы

Идентификаторами являются имена методов, переменных или других определенных пользователем элементов. Какие имена считать правильными? Правильное имя (идентификатор) должно подчиняться следующим требованиям:

1. Имя должно начинаться с буквы или символа подчеркивания.
2. Содержать только латинские буквы, цифры и подчеркивание. Знаки подчеркивания используются для улучшения читабельности имен.
3. Имя чувствительно к регистру букв, т.е. *Rembo* и *rembo* – это разные имена.
4. Имя не должно совпадать с ключевым словом.

Хороший стиль программирования предполагает использование имен идентификаторов, которые отражают значение или особенности применения данного элемента.

Хотя в качестве имени нельзя использовать ключевое слово, перед ним можно поставить символ @, что позволит использовать ключевое слово в качестве действительного идентификатора. Например, *@if* является идентификатором. Фактически, идентификатором является *if*, а @ будет игнорироваться. Использование таких идентификаторов не рекомендуется, за исключением случаев, когда это необходимо для специальных целей.

Оператор присваивания

Цель этого оператора - задание переменной некоторого значения. Синтаксис оператора:

<переменная> = <значение>;

При этом тип переменной должен быть совместим с типом значения.

Оператор присваивания позволяет создавать «цепочку присваивания», которую хорошо демонстрирует следующий пример:

```
int a, b, c;  
a = b = c = 50; // переменным a, b, c присваивается значение 50
```

Типы данных C#

Язык C# является языком строгой типизации, это означает, что все операции в C# контролируются компилятором на предмет совместимости типов данных. Такая строгая проверка типов данных помогает предотвратить ошибки и повышает надежность программ.

Итак, что же такое тип данных? Тип данных задает множество правильных значений переменной и набор операций, которые над ней можно выполнять. C# включает две основные категории типов данных: обычные или базовые типы и ссылочные типы. К ссылочным типам относятся классы, о которых несколько позже. А теперь рассмотрим базовые типы C#. Числовые типы целые и дробные приведены в табл. 2 и табл. 4.

Таблица 2

Целые числа

Тип C#	Размер в байтах	Пояснение
byte	1	Байт, от 0 до 255
sbyte	1	Байт со знаком, от -128 до 127
short	2	Короткое целое со знаком, от -32768 до 32767
ushort	2	Короткое целое без знака, от 0 до 65535
int	4	Целое со знаком, от -2147483648 до +2147483647
uint	4	Целое без знака, от 0 до 4294967295
long	8	Длинное целое со знаком, от -9223372036854775808 до 9223372036854775807
ulong	8	Длинное целое без знака, от 0 до 18446744073709551615

Таблица 4

Дробные числа		
Тип C#	Размер в байтах	Пояснение
float	4	Вещественное число одинарной точности, от 1.5E-45 до 3.4E+38
double	8	Вещественное число двойной точности, от 5E-324 до 1.7E+38
decimal	8	Вещественное число фиксированной точности (от 1E-28 до 7.9E+28)

Наиболее интересным является тип данных *decimal*. Тип данных *decimal* устраняет ошибки округления и может представлять числа с точностью до 28 десятичных разрядов. Этот тип данных предназначен для использования в денежных вычислениях. Ниже приведен пример использования типа данных *decimal*. Литерал типа *decimal* должен заканчиваться символом *m* или *M*. Например:

```
decimal dec1 = 110.333M, dec2 = 99.678m;
decimal dec3 = dec1 - dec2;
Console.WriteLine(" dec3=" + dec3);
```

На экран будет выведено
dec3 = 10.655.

Над числовыми типами данных определены арифметические операторы, приведенные в табл. 5:

Таблица 5

Оператор	Описание
+	Сложение
-	Вычитание
*	Умножение
/	Деление. Важно: результатом деления целых чисел является целое число.
%	Остаток от деления
++	Инкремент
--	Декремент

Оператор инкремента добавляет к своему операнду единицу, а оператор декремента отнимает от операнда единицу. Таким образом, следующие операторы эквивалентны:

```
a++; и a = a + 1;
b--; и b = b - 1;
```

Эти операторы могут иметь либо префиксную (до операнда), либо постфиксную (после операнда) запись. Например:

```
b = ++a;
```

Префиксная запись означает, что значение переменной *a* сначала увеличится на 1, а затем произойдет присвоение полученного значения переменной *b*.

```
b = a++;
```

Постфиксная запись означает, что сначала значение переменной *a* будет присвоено переменной *b*, а затем произойдет увеличение *a* на единицу.

Над числами определены операции следующие операции сравнения (см. табл. 6).

Таблица 6

Оператор	Значение
==	Равно
!=	Не равно
>	Больше
<	Меньше
>=	Больше или равно
<=	Меньше или равно

Из констант, переменных, обращений к функциям, операций и скобок можно строить выражения. Примеры выражений:

$(a + b) * c$; $a * 2.2$; $(a + b + c) / 3$; $c \% 10$.

Последовательность действий при вычислении выражений определяется приоритетами операций и обычными правилами, принятыми в математике.

Символьный тип данных(char)

Таблица 7

Тип C#	Размер в байтах	Пояснение
char	2	Символ (Unicode)

В отличие от других языков программирования (таких как C++, в которых для представления символа выделяется 8 битов), в C# используется набор символов Unicode, в котором представлены символы всех языков мира, поэтому для хранения данных этого типа используется 2 байта. Стандартный 8-ми битовый набор ASCII является частью Unicode. Символьные литералы заключаются в одинарные кавычки. Например:

```
char ch;
ch = 'X';
Console.WriteLine("Значение переменной ch=" + ch);
```

Но следующий фрагмент кода работать не будет:

```
char On;
On = 10;
```

так как символьный тип нельзя совмещать с числовым типом.

Логический тип данных(bool)

Таблица 8

Тип C#	Размер в байтах	Пояснение
bool	1	Логический тип

Логический тип данных – *bool*. Для переменных этого типа определены два значения: *false* и *true*. Не существует способа преобразования переменной этого типа в целочисленные значения. Например, значение 1 не преобразуется в значение *true*, а значение 0 не преобразуется в *false*. Например:

```
bool b;
b = 0; // это ошибка!
b = true; // это правильно!
```

Над логическими переменными определены следующие операции (см. табл. 9):

Таблица 9

Оператор	Описание	Пример
&&	Логическое <i>И</i> . Результат равен <i>true</i> , только если оба операнда равны <i>true</i>	(x==8) && (y==5)
	Логическое <i>ИЛИ</i> . Результат равен <i>false</i> , только если оба операнда равны <i>false</i>	(y>8) (y<5)
!	Отрицание. Изменяет логическое значение на противоположное	if(!(a==b))...

Где операция «==» означает сравнение двух операндов. Знак же одинарного равенства (=) используется для присваивания. Для знака «==» существует парный знак «!=» ("не равно").

Строковый тип данных

Таблица 10

Тип C#	Размер в байтах	Пояснение
string	-	строка

Переменные этого типа содержат текст. Например:

```
string str = "Привет!!!";
```

Обратите внимание, что строковые литералы заключаются в двойные кавычки.

Для строк определена единственная операция сцепление (конкатенация) строк, которая обозначается символом: +. Например:

```
string s1 = "Hello ", s2 = " Mary!";
s1 = s1 + s2;
Console.WriteLine(s1);
```

Контрольные вопросы

1. Что такое переменная? Что такое константа? Нужно ли описывать данные программы перед использованием? Чем константа отличается от переменной?
2. Что такое идентификатор? Какие правила используются в языке C# для задания идентификаторов?
3. Что задает тип данных? Как описать переменную? Как описать константу?
4. Какие числовые типы данных C# Вам известны? Какие операции определены на числовом типе данных?
5. С помощью какого оператора можно изменить значение переменной?
6. Какое ключевое слово используется в C# для целочисленных данных? Что обозначает ключевое слово *double*? Для чего используется тип данных *decimal*?
7. Что такое инкремент (декремент)?

Задания для самостоятельной работы

1. Правильные ли это имена переменных: 1asd, Чаша, @for, program, err type, int_type, Prog1, abc, ABC, pr1? Можно проверить на компьютере.
2. Напишите операторы C#, позволяющие описать следующие переменные:
 - В 2-х переменных требуется хранить трехзначные целые числа.
 - В двух переменных будут храниться дробные числа.
 - В переменной требуется хранить количество лет человека.

- В трех переменных будет храниться зарплата человека, стоимость 1 булки хлеба, 1 брикета мороженого.
- В переменной будет храниться результат деления 2-х целых чисел.
- В переменной будет храниться 5-тизначное целое число.

Форматирование вывода

Рассмотрим более подробно форматированный вывод с помощью оператора **WriteLine**. Синтаксис следующий:

WriteLine("строка форматирования",arg1,arg2,...)

Строка форматирования – задает формат вывода. Строка форматирования состоит из символьных данных и спецификаторов, определителей формата, которые задаются так:

{номер аргумента: формат}

Чаще всего формат используется для вывода дробных чисел. При этом задается шаблон. Для этого в спецификаторе формата указывается номер аргумента (нумерация с нуля), затем после двоеточия при помощи символа # указываются позиции выводимых цифр. В следующем примере:

```
Console.WriteLine("Длина окружности = {0:0.##}, площадь окружности = {1:0.##}",
                  sum1, area);
```

первое подстановочное выражение задано для вывода переменной sum1, второе – для вывода переменной area. При этом шаблон вывода «#.##» означает, что числа будут округлены до двух знаков после точки.

Например: ввести три числа, вывести среднее арифметическое этих чисел.

```
// ввести три числа, вывести их среднее арифметическое
double c, d, e;
string s1, s2, s3;

Console.WriteLine(" Введите 3 числа ");
s1 = Console.ReadLine();
s2 = Console.ReadLine();
s3 = Console.ReadLine();

c = Convert.ToDouble(s1);
d = Convert.ToDouble(s2);
e = Convert.ToDouble(s3);
double f1 = (c + d + e) / 3;
Console.WriteLine("ср арифм = {0:0.##}", f1);
```

Составные операторы присваивания

В С#, так же как и в C/C++, используются составные операторы присваивания, в которых арифметические операторы совмещены с оператором присваивания. Например, выражение:

a = a + b;

аналогично выражению

a += b;

Таким образом, в С# имеются следующие составные операторы присваивания:

+= ++ *= /=

`%=`

`&=`

`|=`

`^=`

Составные операторы по сравнению с их «обычными» аналогами имеют два преимущества. Во-первых, они более компактны, во-вторых, их использование позволяет ускорить компиляцию кода (т.к. операнд оценивается только один раз).

Преобразование типа в операциях присваивания

В операциях присваивания часто переменной одного типа присваивается значение переменной другого типа. Например, переменной типа *float* можно присвоить значение переменной типа *int*, как показано в следующем примере:

```
int i = 10;
float f;
f = i;
```

Если в операции присваивания используются совместимые типы, то тип переменной, находящейся справа от оператора, автоматически преобразуется к типу переменной, находящейся слева от оператора.

Но не все типы данных являются совместимыми. Например, *bool* и *int* не совместимы.

Преобразования бывают двух видов: *явные* и *неявные*. Неявные преобразования выполняются автоматически в случае, если:

- два типа совместимы;
- конечный тип(слева) больше исходного типа (справа).

При выполнении этих двух условий происходит *расширяющее преобразование*. Например, количества битов, выделенных под тип *int* всегда достаточно для хранения значений типа *byte*, а так как оба эти типа целочисленные, то к ним может быть применено автоматическое (неявное) преобразование.

Хотя существует неявное преобразование типа переменной из *long* в *double*, обратное неявное преобразование осуществить нельзя, т.к. оно не будет расширяющим.

В дополнение в сказанному добавим, что не существует неявного преобразования между типами *decimal* и *float* или *double*, либо неявного преобразования переменных числового типа в переменные типа *char* и *bool*.

В случаях, когда не работает неявное преобразование, применяется явное преобразование или приведение типов. *Приведение типа* это инструкция компилятору для преобразования одного типа в другой. Операция приведения имеет следующий синтаксис:

(тип) выражение

Тип указывает к какому типу должно быть приведено выражение. Например, если необходимо преобразовать выражение *x/y* к типу *int*, следует написать:

```
int d = (int) (x / y);
```

Когда в результате приведения типа происходит *сужающее преобразование*, информация может быть утеряна. Например, при преобразовании типа данных *int* в *byte*:

```
int i = 258;
byte b;
b = (byte) i;
Console.WriteLine("b={0}", b);
```

Будет выведено: *b = 2*.

Аналогичным образом преобразования типов выполняются в выражениях. В выражениях существует возможность совмещения двух и более различных типов при условии, что они являются совместимыми.

В качестве примера приведем решение следующей задачи: введите 3-х значное целое число, выделите цифры сотен, десятков и единиц этого числа. Выведите результат.

```
int n;
byte dec, sot, ed;    // для хранения десятков, сотен, единиц

Console.WriteLine("Введите 3-х значное число:");
n = Int16.Parse(Console.ReadLine());

ed = (byte)(n % 10);  // выделяем единицы
n = (n - ed) / 10;
dec = (byte)(n % 10); // выделяем десятки
sot = (byte)(n / 10); // выделяем сотни

Console.WriteLine("сотни {0}, дес {1}, единицы {2} ", sot, dec, ed);
Console.Read();
```

Контрольные вопросы

1. Какие операции над числами разрешены в C#?
2. Какие операции сравнения чисел Вы знаете? Как они обозначаются?
3. Как выполняется преобразование типов? Как преобразовать число в строку и строку в число?
4. Для чего используется логический тип данных? Какие значения может принимать переменная логического типа? Как задать переменную логического типа?
5. Какие операции определены на логическом типе данных?

Задания для самостоятельной работы

1. Ввести радиус круга. Вывести длину и площадь круга. Написать программу. Подсказка: число «π» вы найдете в классе *Math*.
2. Написать программу пересчета величины временного интервала, заданного в минутах, в величину, выраженную в часах и минутах. Рекомендательный вид экрана:
Введите временной интервал (в минутах): 150
150 минут это 2 ч. 30 минут.
3. Написать программу, которая преобразует введенное с клавиатуры дробное число в денежный формат. Например: 23,7 это 23 руб. 70 копеек.
4. Введите 3-х значное целое число, выделите цифры сотен, десятков и единиц этого числа. Выведите результат. Какое число получится, если поменять местами цифры сотен и единиц. Вывести результат.
5. Сила тяжести на поверхности Луны составляет приблизительно 17 процентов от силы тяжести на поверхности Земли. Напишите программу, которая бы вычисляла, каким будет ваш вес на Луне.

Операторы ветвлений: if и switch

Для реализации разветвляющегося алгоритма используются операторы *if* и *switch*. Оператор *if* служит для разветвления программы на два направления. Если некоторое условие выполняется, то выполняется одна ветка программы, если условие не выполняется, то выполняется другая ветка программы. Синтаксис оператора имеет вид:

```
if (условие)
{ <группа команд1>
}
[ else
  { <группа команд2>
  }
]
```

Ветвь *else* не является обязательной. Если группа команд состоит из одного оператора, то фигурные скобки можно не использовать. В следующем примере определяется, какое число ввел пользователь:

```
Console.WriteLine("Введите целое число ");
int k = Int32.Parse(Console.ReadLine());
if (k / 2 == 0)
{
    Console.WriteLine("Четное число");
}
else
{
    Console.WriteLine("Нечетное число");
}

Console.ReadLine();
```

Можно использовать несколько вложенных операторов *if*. При этом необходимо помнить, что во вложенных операторах *if-else* вторая часть оператора (*else*) всегда относится к ближайшему оператору *if*. Например:

```
int a = 8, b = 7;
if (a == b)
    Console.WriteLine("равны");
else
    if (a > b) // последний else относится к этому if
        Console.WriteLine(" a больше b");
    else Console.WriteLine("a меньше b");

Console.ReadLine();
```

Оператор *switch* позволяет программе выбрать нужное действие из перечня вариантов. Синтаксис оператора следующий:

```
switch (<выражение>)
{
    case <константа 1>:
        группа операторов1
        break;
    case <константа 2>:
```

```

        группа операторов2
        break;
    ...
[ default:
    группа операторов3
    break;]
}

```

Значение выражения последовательно сравнивается с каждой константой из списка. При совпадении значения с одной из констант выполняется ассоциированная с ней последовательность операторов.

Выражение в операторе должно быть целочисленного (*char*, *byte*, *short* или *int*), либо строкового типа *string*. Выражения с плавающей точкой запрещены. Константы должны быть литералами и иметь тип, совместимый с типом выражения. Ветвь *default* не является обязательной.

Оператор *switch* может иметь любое число вариантов (*case*), но не может быть двух вариантов с одинаковыми значениями. Выполнение оператора начинается с выбранного варианта и продолжается до выполнения оператора *break*. *Break* завершает выполнение оператора *switch*. *Break* требуется после выполнения любого *case*-утверждения, включая утверждение *default*. Пример использования оператора:

```

Console.WriteLine("Введите оценку за экзамен ");
int k = Int32.Parse(Console.ReadLine());

switch (k)
{
    case 1:
    case 2:
        Console.WriteLine("Неудовлетворительно");
        break;
    case 3:
        Console.WriteLine("Удовлетворительно");
        break;
    case 4:
        Console.WriteLine("Хорошо");
        break;
    case 5:
        Console.WriteLine("Отлично");
        break;
    default:
        Console.WriteLine("Ошибка");
        break;
}

```

В приведенном примере в зависимости от введенного пользователем числа на экран выводится та или иная оценка. Если число *k* не лежит в промежутке от 1 до 5, то выполняются операторы в ветке *default* и выводится надпись "Ошибка". Обратите внимание на оператор *break*. Если его не написать, то будут выполняться операторы из следующей ветки *case* до строки с *break*'ом (т. е. в данном примере если пользователь введет 1 или 2, то программы выведет "Неудовлетворительно"). Обратите внимание, что если в некоторой ветке *case* или *default* есть операторы, то написание *break* обязательно. Так, в следующем фрагменте кода есть ошибки:

```

case 1:
    Console.WriteLine("Совсем неудовлетворительно");

```

```

        //Ошибка! Тут пропущен break
    case 2:
        Console.WriteLine("Неудовлетворительно");
        break;    ...
        ...
default:
    Console.WriteLine("Неудовлетворительно");
    break;

```

Контрольные вопросы

1. Можно ли использовать несколько вложенных операторов *if*? К какому оператору *if* в этом случае относится ветка *else*?
2. Какого типа должно быть выражение в операторе *switch*?
3. Обязателен ли оператор *break* после любого *case*-утверждения?
4. Могут ли быть два одинаковых *case* в операторе *switch*?
5. Что означает ключевое слово *default* в операторе *switch*?

Задания для самостоятельной работы

1. Введите порядковый номер дня недели. Выведите название дня недели (понедельник, вторник и т.д.).
2. Ввести пять чисел. Вывести минимальное число.
3. Написать код, выполняющий проверку того, что введено трехзначное число.
4. Введите четыре вещественных числа. Определите можно ли построить четырехугольник с длинами сторон, равными этим числам.
5. Ввели 6-ти значное число. Является ли оно счастливым, т.е. равна ли сумма первых 3-х цифр сумме 3-х последних.
6. Ввели 6-ти значное число. Является ли оно перевертышем, т.е. одинаково читается как справа налево, так и слева направо.
7. Поле шахматной доски определяется парой (m, n) натуральных чисел (m от 1 до 8, n от 1 до 8). Первое число – номер строки (номер горизонтали), второе число – номер столбца (номер вертикали). Даны натуральные числа k, l, m, n , каждое из которых не больше 8. Определить:
 - а) являются ли поля (m,n) и (k,l) полями одного цвета;
 - б) угрожает ли полю (m,n) ферзь, находящийся на поле (k,l) .
 - в) угрожает ли конь, стоящий на поле (k,l) полю (m,n) .

Объектно-ориентированный подход в программировании

В настоящее время большое распространение получила технология объектно-ориентированного программирования. Большинство современных языков программирования используют объектно-ориентированную парадигму.

Этот подход в программировании возник, когда появилось понимание того, что программный код тесно связан с теми данными, которые он обрабатывает. Кроме того, упаковка программного кода в объекты, позволяет посмотреть на программу не как на совокупность программного кода, в виде процедур, функций и данных, а как на взаимодействие различных объектов, каждый из которых представляет собой совокупность

программного кода и данных. Такое понимание программы ближе к представлению предметной области в виде объектов реального мира.

Основоположниками объектного подхода в программировании считаются норвежцы Оле Джохан Дал и Кристен Ньюгорт, авторы языка Симула, история которого началась в 1962 году. Авторам этого языка пришла в голову идея объединить данные с процедурами, их обрабатывающими. Ими же были также придуманы термины "класс" и "объект". Тогда же возникла и технология наследования.

Объектно-ориентированный подход помогает справиться с такими сложными проблемами, как

- сложность программного обеспечения;
- надежность программного обеспечения;
- возможность модификации отдельных компонентов программного обеспечения без изменения остальных его компонентов;
- возможность повторного использования отдельных компонентов программного обеспечения.

Систематическое применение объектно-ориентированного подхода позволяет разрабатывать хорошо структурированные, надежные в эксплуатации, достаточно просто модифицируемые программные системы. Этим объясняется интерес программистов к объектно-ориентированному подходу и объектно-ориентированным языкам программирования. Объектно-ориентированный подход является одним из наиболее интенсивно развивающихся направлений теоретического и прикладного программирования.

Основные понятия ООП

Ключевыми понятиями этой технологии являются понятия объекта и класса. Под классом понимается некая совокупность объектов реального мира, обладающих общими свойствами и общим поведением. Например, класс «Собаки» - объединяет вообще всех собак. Класс «Книги» - объединяет все книги. Класс «Студенты» - объединяет вообще всех студентов. Класс «Планеты» - объединяет все планеты, например, солнечной системы.

Объект представляет собой конкретный экземпляр какого-либо класса, например, собака «Шарик» является объектом класса «Собаки», книга «Три мушкетера» - это объект класса «Книги», а студент Иванов Вася является объектом класса «Студенты», планета «Земля» - объект класса «Планеты».

При описании класса задаются свойства объектов класса и их поведение. Свойства могут задавать внешний вид объекта, его состояние и описываются как данные. Поведение объектов задается с помощью программного кода. При этом разные объекты одного класса могут иметь разные значения свойств, но одно и то же поведение. Например, различные планеты солнечной системы могут иметь разное положение на экране, разную окраску, разный размер, но двигаются они по одним и тем же законам.

Определение класса в С#

Класс – это тип (шаблон), в соответствии с которым создаются экземпляры объектов. Например класс СТУДЕНТ, класс СОБАКА, класс ЖИВОТНОЕ, класс КНИГА, и т.д.

Объект – это экземпляр класса. Объект строится строго в соответствии с той структурой, которая задана типом, или классом.

Объект имеет свойства и поведение. Например, собака Дружок может иметь окрас, возраст, кличку и т.д. Это ее свойства. Собака Дружок относится к классу СОБАКА. У собаки есть поведение, она подает голос, она бежит, кусается, просит кушать и т.д. Свойства объекта описываются как данные, поведение задается как программный код, так

называемые методы. В операторах языка программирования это имеет строгое описание в соответствии с принятым синтаксисом.

Класс - тип

Объект = Данные + Программный код

Давайте сначала спроектируем классы, а затем строго в соответствии с синтаксисом C# опишем их и проверим как это работает на компьютере.

Создадим класс СОБАКА со свойствами Окрас, Возраст, Кличка. Поведение собаки характеризуется следующими методами: «ПодатьГолос», «Представиться» (например, наша собака умеет говорить, и хочет рассказать о себе все), метод «Расти» (будет увеличивать возраст собаки на 1 год).

Итак, все свойства и методы класса СОБАКА(Dog) приведены в табл. 11:

Таблица 11

Свойства и методы	Описание
Name	Свойство, кличка собаки
Clr	Свойство, окрас собаки
Age	Свойство, возраст собаки
Presentation	Метод, собака рассказывает о себе
Voice	Метод, собака подает голос
Grow	Метод, собака взрослеет на 1 год

Описание класса Dog выглядит следующим образом:

```
// Определение класса Dog
class Dog
{ // Свойства
    public string Clr; // Цвет собаки
    public string Name; // Кличка собаки
    private int Age; // возраст собаки

    // конструктор
    public Dog(string clr, string name, int age)
    {
        this.Clr = clr;
        this.Name = name;
        this.Age = age;
    }

    // Методы
    // собака подает голос
    public void Voice()
    {
        Console.WriteLine("Гав, Гав, ...");
    }

    // Собака представляется
    public void Presentation()
    {
        Console.WriteLine("Меня зовут {0}, мне {1} лет", this.Name, this.Age);
    }

    // Собака подросла на 1 год
    public void Grow()
    {
        this.Age++;
    }
}
```

```

    }
} // конец описания класса

```

При описании свойств используются модификаторы доступа: **private**, **public**, еще есть **protected**.

Private – означает закрытое свойство, т.е. видимое только внутри методов данного класса.

Public – открытое свойство, т.е. свойство доступное для внешних объектов, видимое из вне. Если делаем свойство типа **public**, тогда можно к нему обращаться так:

```
Bobik.Clr="Желтый";
```

Где **Bobik** – это объект класса **СОБАКА**.

Такое обращение к свойству типа **Private** вызовет ошибку.

Protected – свойства, которые доступны потомкам класса. Дело в том, что *private*-свойства у потомков не видны. А *protected* видны.

В классе **Dog** присутствует метод с одноименным названием **Dog**. Этот метод имеет специальное назначение и называется **конструктор**. Конструктор вызывается автоматически один раз при создании экземпляра объекта. Часто основное назначение конструктора состоит в инициализации значений свойств объекта.

Создадим два объекта класса **Dog**:

```

// Определение переменных типа Dog
Dog Bobik, Sharik;
// Создание экземпляров класса (объектов)
Bobik = new Dog("черный", "Бобик", 3);
Sharik = new Dog("коричневый", "Шарик", 4);
// Вызов методов объектов
Bobik.Presentation();
Bobik.Voice();
Bobik.Grow();
Bobik.Presentation();
Sharik.Presentation();
Sharik.Voice();
Console.ReadLine();

```

Основные принципы ООП

Основных принципов объектно-ориентированного программирования три :

- Инкапсуляция
- Наследование
- Полиморфизм

Рассмотрим их более подробно.

Инкапсуляция

Этот принцип по другому еще называется *сокрытие данных*, и означает, что доступ к данным должен выполняться не напрямую, а через методы объекта. Данные как бы находятся в капсуле из методов, которые позволяют проконтролировать правильно ли изменяется свойство(данные объекта).

Данные объекта бывают двух видов: поля и свойства. Свойство это поле плюс два метода, первый позволяет читать из поля (*get block*), а второй позволяет и писать в поле (*set block*). Когда выполняется доступ к свойству, т.е. чтение или запись, на самом деле работает соответствующий метод.

Например, сделаем *public*-данное *Clr* в нашем классе *Dog* закрытым, т.е. с модификатором доступа *private*:

```
private string Clr; // окрас собаки
```

Защитим данное *Clr* двумя методами, т.е. сделаем его свойством. Определим для него открытый метод *Cols*, который содержит два блока: блок *get* и блок *set*. После этого свойство *Clr* будет защищено своими методами.

```
// Определение класса Dog
class Dog
{
    public string Name; // Кличка собаки
    private int Age; // возраст собаки
    private string Clr; // окрас собаки

    // демонстрация инкапсуляции
    public string Cols
    {
        get { return col; }
        set { col = value; }
    }

    // конструктор
    public Dog(string clr, string name, int age)
    {
        this.Clr=clr;
        this.Name = name;
        this.Age = age;
    }
    . . .
}
```

На данный момент наши блоки ничего сложного не делают, блок *get* извлекает данные, а блок *set* изменяет данные. Но программный код блоков может быть более сложным.

Если создать экземпляр класса СОБАКА:

```
Dog Bobik;
// Создание экземпляра объекта
Bobik = new Dog("черный", "Бобик", 3);
Bobik.Cols = "соль с перцем";
```

То доступ к свойству *Clr* будет выполняться не на прямую, а через метод *Cols*.

Наследование

Одним из основных принципов ООП является принцип наследования. Назначение наследования в повторном использовании кода. Можно создавать наследников класса, при этом наследуются все свойства и методы предка и добавляются новые свойства и методы. Это очень мощный принцип, который позволяет не изобретать «велосипед», а взять все, что нужно у предка и добавить новое.

Давайте на простом примере рассмотрим наследование.

Пусть имеем абстрактный класс ЖИВОТНОЕ(ANIMAL). Абстрактный класс это такой класс, который не может иметь экземпляров, т.е. нельзя создавать объекты этого класса. Действительно, нет просто животных. От этого класса произведем два класса наследника: СОБАКА и КОТ (рис. 2).

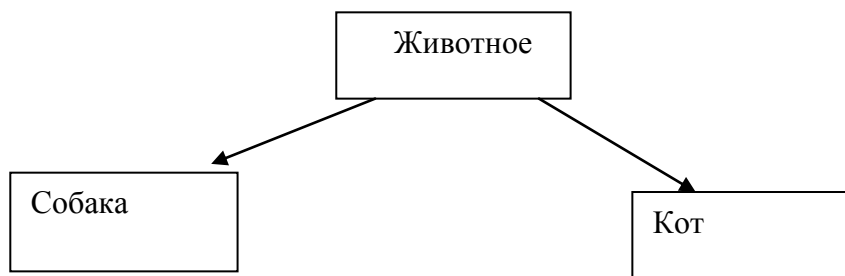


Рис. 2

В табл. 12 приведены свойства и методы класса ANIMAL:

Таблица 12

Свойства и методы	Описание
Name	Свойство, кличка животного
Clr	Свойство, окрас животного
Age	Свойство, возраст животного
Presentation	Метод, животное рассказывает о себе
Voice	Метод, животное подает голос
Grow	Метод, животное взрослеет на 1 год

Причем метод *Voice* виртуальный, т.е. он получит конкретную реализацию у потомков, будет переопределен. Действительно, СОБАКА лает (Гав, гав...), а КОТ мяукает (Мяу, мяу...).

Классы СОБАКА и КОТ, которые являются наследниками класса ЖИВОТНОЕ, содержат только конструктор и переопределенный метод *Voice*. Все остальные свойства и методы наследуются от предка. Программный код, реализующий нашу идею, приведен ниже:

```

class Program
{
    static void Main(string[] args)
    {
        // создание экземпляров
        Dog Bobik = new Dog("Рыжий", "Бобик", 3);
        Cat Murzik = new Cat("Полосатый", "Мурзик", 5);

        // Вызов метода экземпляра
        Bobik.Presentation();
        Bobik.Voice();

        Murzik.Presentation();
        Murzik.Voice();
        Console.ReadLine();
    }

    // абстрактный класс Животное
    abstract class Animal
    {
        public string Clr; // окрас животного
        public string Name; // Кличка животного
        protected int Age; // возраст животного

        // виртуальный метод, это значит, что у потомков он будет переопределен
        public virtual void Voice()
        {

```

```

    }

    // Животное представляется
    public void Presentation()
    {
        Console.WriteLine("Меня зовут {0}, мне {1} лет, я имею {2} окрас",
            this.Name, this.Age, Clr);
    }

    // Животное подросло на 1 год
    public void Grow()
    {
        this.Age++;
    }

} // конец описания класса Animal

// Определение класса Dog, наследника класса Animal
class Dog : Animal
{
    // конструктор
    public Dog(string clr, string name, int age)
    {
        this.Clr = clr;
        this.Name = name;
        this.Age = age;
    }

    // собака подает голос, override означает что, метод
    //переопределен
    public override void Voice()
    {
        Console.WriteLine("Гав, Гав, ...");
    }
} // конец описания класса Dog

// Определение класса Cat, наследника класса Animal
class Cat : Animal
{
    public Cat(string clr, string name, int age)
    {
        this.Clr = clr;
        this.Name = name;
        this.Age = age;
    }

    // кошка подает голос
    public override void Voice()
    {
        Console.WriteLine("Мяу, Мяу, ...");
    }
}

}

```

В самом деле, в классе *Dog*, точно так же как и в классе *Cat*, описаны только два метода – конструктор и переопределенный метод *Voice*. Остальные методы: *Presentation* и *Grow* наследуются от предка.

Полиморфизм

Третий принцип ООП называется – полиморфизм (поли – много, морф – форма), можно перевести как разнообразие. Этот принцип означает, что родственные объекты

могут выполнять схожие по смыслу действия разными способами. Т.е. у потомков поведение уже может быть изменено. Иногда этот принцип называют сокрытие реализации, т.е. за одним и тем же заголовком метода, реализующего поведение объекта, скрывается разная реализация метода у предка и у потомка.

Для реализации полиморфизма используются ключевые слова **virtual** и **override**. Если базовый класс определяет метод, который должен быть замещен в производном классе, этот метод должен быть определен как виртуальный (при помощи слова **virtual**). В нашем примере метод *Voice* в классе *Animal* определяется как **virtual**. Это значит, что в классах наследниках он будет замещен одноименным методом, но с другим содержанием.

В классе наследнике этот метод определяется с ключевым словом **override**. В примере выше был приведен программный код, реализующий полиморфизм. В самом деле, метод *Voice*, определенный у предка, как виртуальный, у потомка получил реализацию. При этом у разных потомков одноименный метод получил разную реализацию.

Статические данные и методы

Статические данные аналогичны статическим переменным в языках программирования. Обычные данные каждый объект хранит отдельно и независимо от других объектов класса. В отличие от обычных данных статические данные совместно используются всеми объектами класса, в котором эти данные были определены. Вместо локальных копий переменной в каждом из объектов существует единственная копия статической переменной, которой пользуются все объекты данного класса.

Статический метод (*static method*) может быть вызван напрямую через уровень класса без создания экземпляра. По этой причине метод *Main* всегда объявляется как *static* (статический), для того чтобы этот метод мог выполняться сразу еще до создания первого экземпляра класса.

Создадим в нашем классе *Dog* статическую переменную *NumberDogs*, в которой будет храниться количество созданных экземпляров данного класса.

```
private static int NumberDogs; // статическое свойство количество собак
```

Т.к. переменная *NumberDogs* является скрытой (*private*), то необходимо создать статический метод *GetNumber*, который извлекает значение статической переменной *NumberDogs*:

```
public static int GetNumber ()  
{ return NumberDogs; }
```

В конструктор *Dog* добавим оператор:

```
NumberDogs++;
```

Таким образом, при создании каждого нового объекта, значение переменной *NumberDogs* будет увеличиваться на 1.

Использование статического метода:

```
int num = Dog.GetNumber();  
Console.WriteLine("число собак= {0} ", num);
```

Как видно отсюда не нужно создавать экземпляр класса, для того чтобы выполнить статический метод.

Примеры статических методов можно найти у классов *Console*, *Convert*, *Math*.

Задания для самостоятельной работы

1. Создать класс «КОРОВА», который является наследником класса «ЖИВОТНОЕ». В классе КОРОВА определить конструктор и переопределить метод «Подать голос», корова говорит «МУ-МУ».

2. Создать класс *Person* с полями:

Fio – фамилия, имя, отчество

Adres – адрес,

Tel – телефон.

Написать для этого класса конструктор, который инициализирует свойства класса. Создать метод *printPerson*, который выводит значения всех свойств объекта типа *Person*. Создать два экземпляра класса *Person*.

Фио= Иванов И И

Адрес = Русская 21, 19

Тел = 347-45-13

Фио = Петров П П

Адрес = Цветной 14,12

Тел = 347-46-90

Вызвать методы *printPerson* для каждого экземпляра. Создать класс *Student*, который является наследником класса *Person*. Дополнить его свойствами:

- Номер зачетки
- Факультет
- Курс
- Год поступления
- Год выпуска

Переопределить метод *printPerson* в классе *Student*. Создать экземпляр(объект) класса *Student*. Будет ли для него работать конструктор класса *Person*? Если нет, создать конструктор для класса *Student*. Добавить метод *NextCurs*, который проверяет, если курс > 5, то курс = «Выпуск», иначе курс++ (перевод на следующий курс).

Контрольные вопросы

1. Что такое класс? Что такое объект? Из каких элементов состоит объект?
2. Что задают данные объекта? Что задают методы объекта?
3. Основные принципы ООП? Перечислить и кратко охарактеризовать.
4. Каков синтаксис описания класса?
5. Как создать экземпляр объекта? Что такое конструктор?
6. Каков синтаксис получения доступа к свойству?
7. Что означает модификатор доступа *public*?
8. Что означает модификатор доступа *private*?
9. Что означает модификатор доступа *protected*?
10. Каков синтаксис вызова метода?

Среда разработки Visual Studio Net

Среда разработки VS состоит из следующих основных элементов:

- Линейки меню с выпадающими подменю;
- Кнопочных панелей;
- Окна проектирования приложения.

Кроме этих основных элементов можно использовать следующие вспомогательные элементы:

- **Обозреватель решений** – окно проекта (сборки), в котором видна вся структура проекта (сборки), т.е. из каких файлов он состоит.
- **Свойства** – окно свойств, в котором видны свойства выбранного объекта.
- **Панель элементов** – панель готовых инструментов для конструирования приложения.
- **Обозреватель объектов** – браузер объектов, позволяет просматривать иерархию объектов данного приложения.

Есть и другие вспомогательные элементы и окна, которые доступны через меню «Вид» (рис. 3).

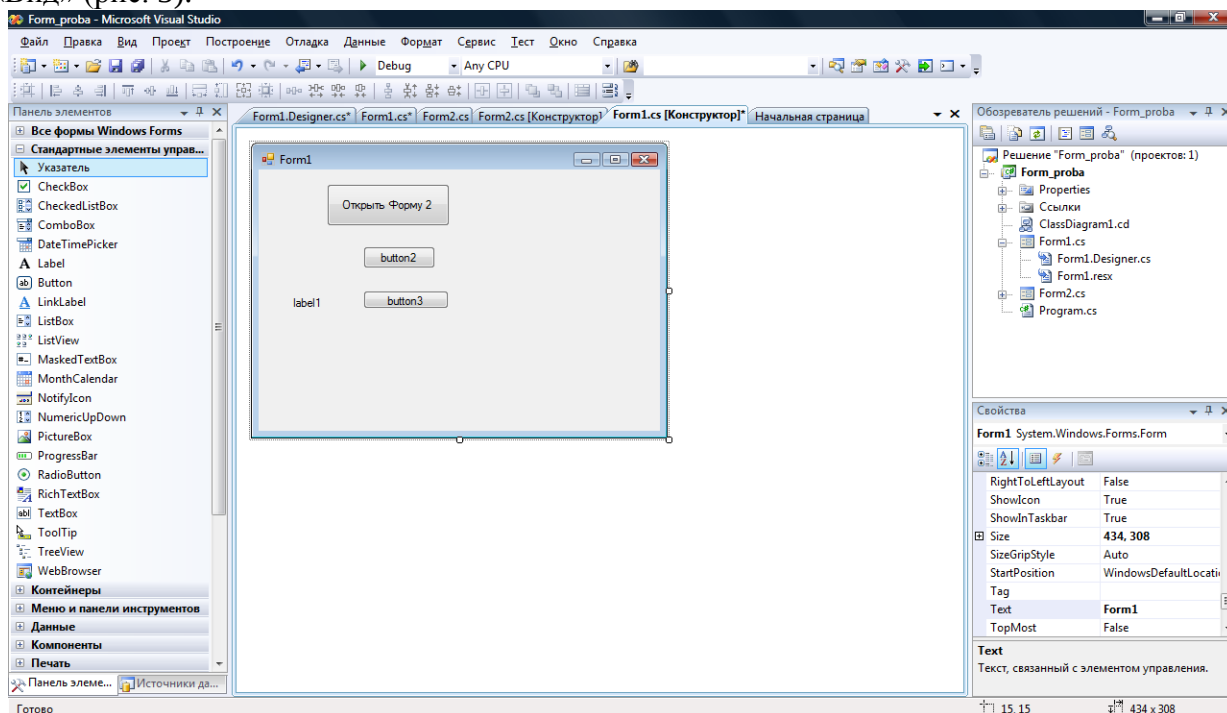


Рис. 3

Создание приложения Windows Forms

Приложение Windows Forms позволяет создавать многооконный пользовательский интерфейс, состоящий из нескольких окон или форм. Для создания такого приложения необходимо последовательно выбрать следующие пункты меню:

Файл | Создать | Проект...

Открывается диалог «Создать проект» (см. рис. 3).

В секции «*Типы проектов*» необходимо выбрать *Visual C#* и далее *Windows*. В секции «*Шаблоны*» выбрать «*Приложение Windows Forms*», как показано на рис. 4.

В поле «*Имя:*» задается имя проекта, в поле «*Расположение*» задается папка в которой будет располагаться исходный код нашего приложения.

Поле «*Имя решения*» – имя папки, в которой будет размещен проект.

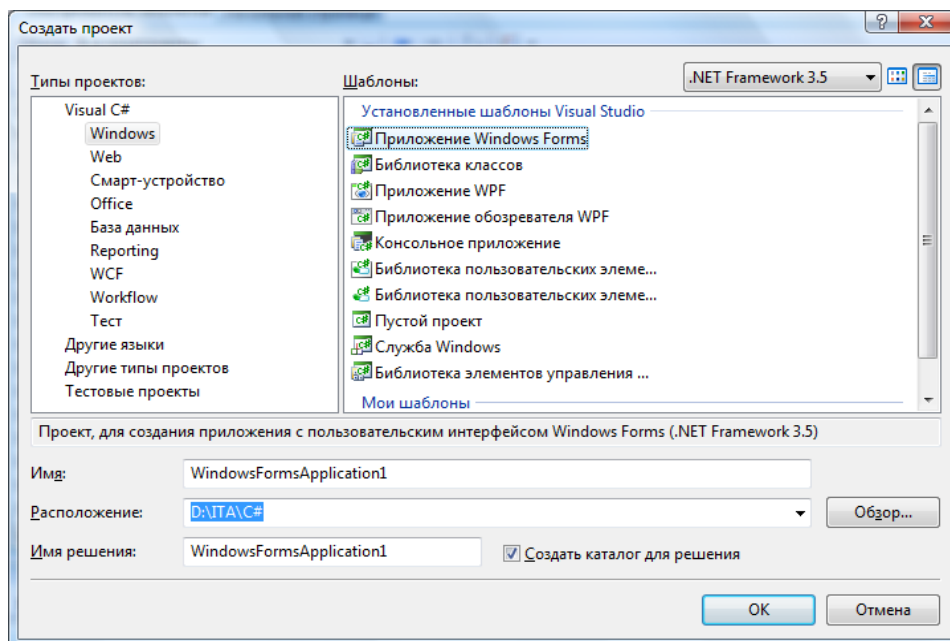


Рис. 4

Windows-приложение создается в виде проекта. Проект состоит из нескольких файлов разного назначения. В табл. 13 приводится описание некоторых файлов проекта:

Таблица 13

Имя файла	Описание
*.sln	файл сборки проекта
*.suo	файл опций проекта
*.csproj	Проект
*.cs	программные файлы на C#
	содержит графическое представление формы

Вся работа с приложениями Windows Forms выполняется через меню «*Файл*», а именно создание, открытие, сохранение.

Работа с формами

Приложение может состоять из одной или нескольких форм. С точки зрения пользователя форма это просто окно, с точки зрения программиста форма это объект, наследник класса *Form*.

Создание формы

Для создания формы необходимо выполнить последовательно следующие пункты меню *Проект | Добавить форму Windows...*

В открывшемся диалоге (см. рис. 5) в секции «*Категории*» выбрать «*Windows Form*», в секции «*Шаблоны*» - «*Форма Windows Forms*». В поле «*Имя*» ввести имя формы.

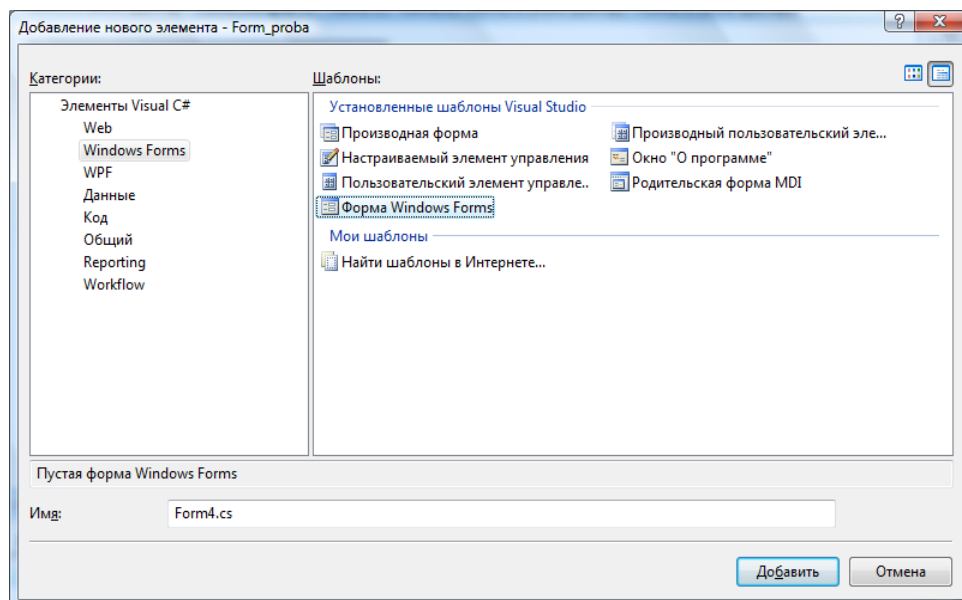


Рис. 5

С каждой формой связаны графическое представление и программный файл. Программный файл имеет расширение *.cs и имеет следующий вид:

```
using System.Drawing;
using System.Text;
using System.Windows.Forms;

namespace app1
{
    public partial class Form2 : Form
    {
        public Form2()
        {
            InitializeComponent();
        }
    }
}
```

Откуда видно, что форма это наследник класса Form. Ключевое слово «partial» позволяет определению класса, структуры или интерфейса быть распределенным по нескольким файлам. Метод *InitializeComponent* позволяет инициализировать компоненты (элементы управления), помещенные в форму.

В окне «Обозревателя решений» можно видеть все формы, входящие в сборку(решение), просматривать их, переключаться между ними (рис. 6).

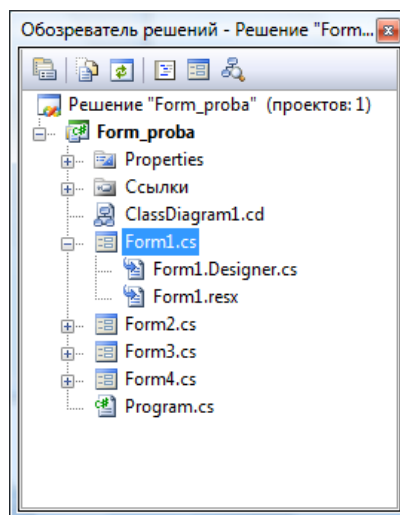


Рис. 6

Основные свойства и методы формы

Рассмотрим основные свойства формы (см. табл. 14)

Таблица 14

Свойство	Описание
Name	имя формы, как объекта класса, позволяет ссылаться на форму в программе
Text	задает заголовок окна формы
BackColor	задает цвет фона
StartPosition	задает стартовая позиция окна
Size	задает размер окна

По способу открытия формы бывают *модальные* и *немодальные*. *Модальная* форма это, по сути, диалоговое окно, которое открывается поверх всех окон и не позволяет получать доступ к другим окнам приложения до тех пор, пока не будет закрыто. *Немодальное* окно позволяет получать доступ к любым другим окнам приложения.

Основные методы формы приведены в табл. 15.

Таблица 15

Метод	Описание
ShowDialog	Окно открывается как модальное
Show	Окно открывается как обычное, не модальное
Close	Закреть окно

Форму можно открыть, например, по кнопке. Поместим кнопку на форму (см. рис. 7). На событие Click кнопки напомним следующий код:

```
// создание объекта формы
Form2 frm2 = new Form2();
// открытие формы как немодальной
frm2.Show();
```

Для открытия формы, как модального окна используется следующий код:

```
frm2.ShowDialog();
```

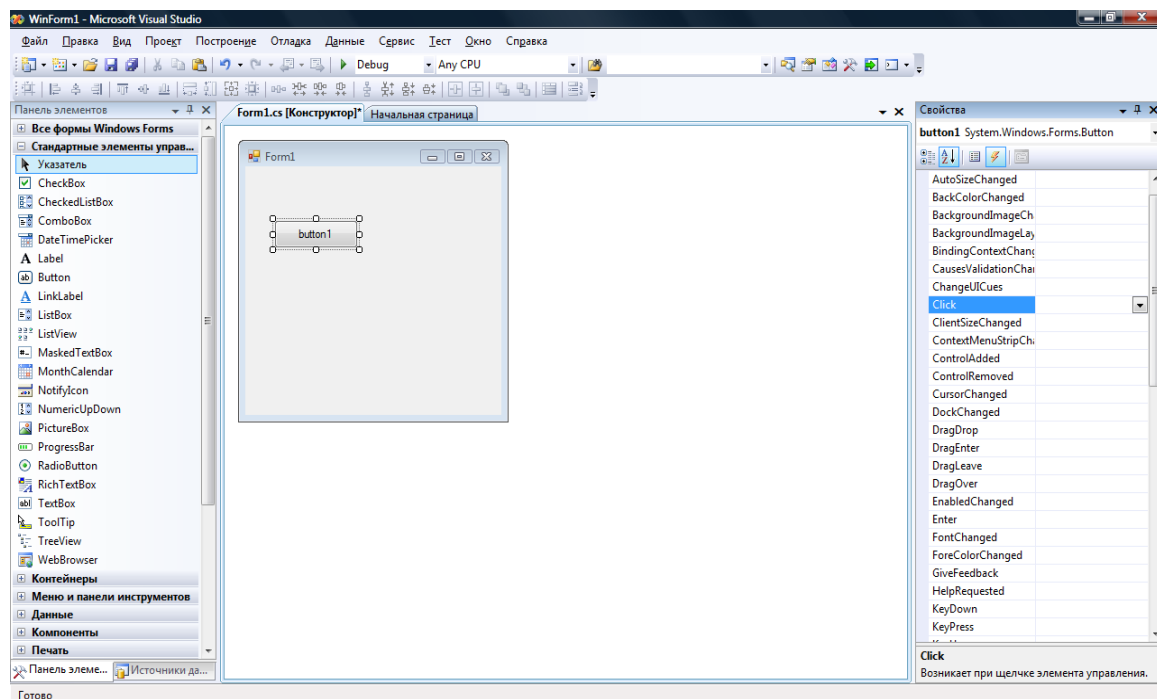


Рис. 7

Если форма открыта как *модальная*, то код, выполняющийся непосредственно за методом `ShowModal()` будет выполнен только, когда форма закроется. В то время как для *не модальной* формы код выполнится сразу, как только форма будет выведена на экран.

Заккрыть форму можно либо так:

```
this.Close();
```

где `this` – это ссылка на текущее окно, а `Close` – это метод окна. Либо так:

```
Close();
```

С формой связаны следующие файлы:

- ✓ `Form1.Designer.cs` – хранит графическое представление формы
- ✓ `Form1.cs` – содержит программный код, связанный с формой.
- ✓ `Form1.resx` – ресурсный файл, содержит рисунки, иконки приложения.

Задание: Добавить к проекту форму «Окно «О программе» (см. рис. 8).

Открыть ее по кнопке:

```
AboutBox1 about = new AboutBox1();
about.Show();
```

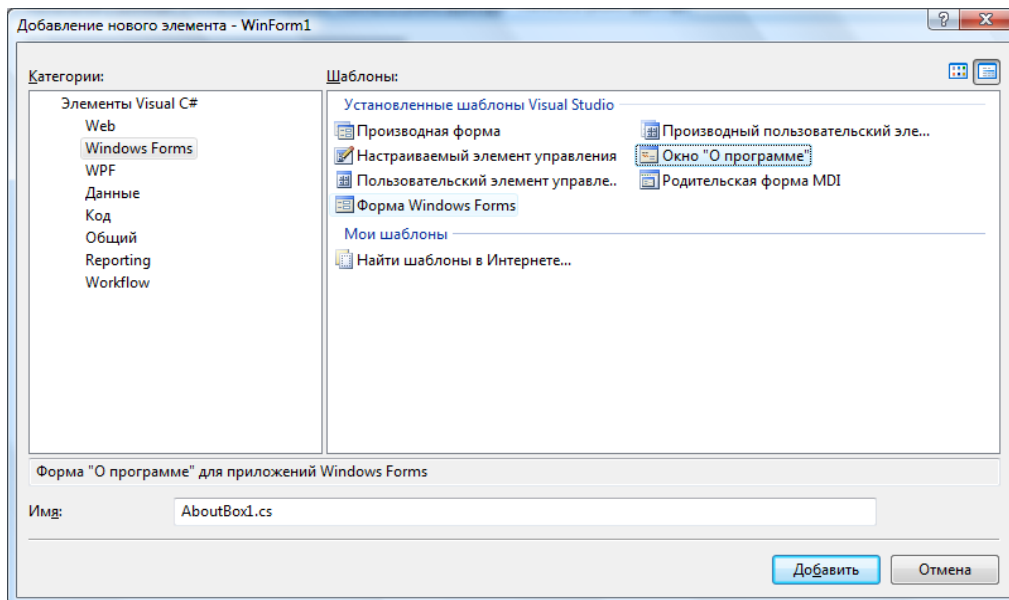


Рис. 8

Внешний вид формы приведен на рис. 9.

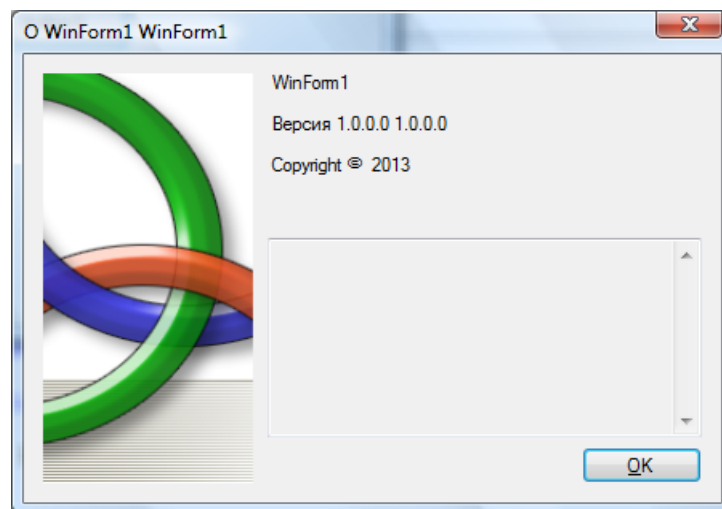


Рис. 9

Способность форм ссылаться друг на друга

Часто требуется из одной формы получить доступ к объектам или элементам управления другой формы. Например, из формы Form2, которая открывается по кнопке из формы Form1 нужно изменить объект label1, который находится в форме Form1. Как это сделать?

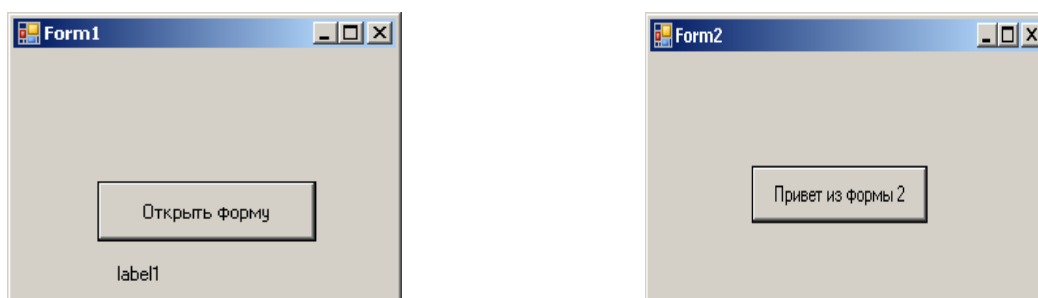


Рис. 10

Для этого в форме 2 создается ссылка *frm1* на форму 1:

```
public Form1 frm1;
```

Ссылка *frm1* получает свое значение в момент, когда мы из формы 1 открываем форму 2, например, по кнопке:

```
frm2 = new Form2();  
frm2.frm1 = this;  
frm2.Show();
```

Теперь из формы 2 доступ к элементу *label1* формы 1 получить очень просто:

```
frm1.label1.Text = "привет из формы 2";
```

Аналогично выполняется доступ к элементам формы 2 из формы 1. Полностью код формы 1 и формы 2 приведен ниже:

Код формы 1:

```
public partial class Form1 : Form  
{  
    public Form1()  
    {  
        InitializeComponent();  
    }  
    // создаем ссылку на объект формы 2 на уровне класса  
    public Form2 frm2;  
    . . .  
    // при нажатии на кнопку открывается форма 2 и ей передается указатель на  
    // форму 1  
    private void button2_Click(object sender, EventArgs e)  
    {  
        frm2 = new Form2();  
        frm2.frm1 = this;  
        frm2.Show();  
    }  
}
```

Код формы 2

```
public partial class Form2 : Form  
{  
    public Form2()  
    {  
        InitializeComponent();  
    }  
    public Form1 frm1;  
    . . .  
    // далее все просто ...  
    private void button1_Click(object sender, EventArgs e)  
    {  
        frm1.label1.Text = "привет из формы 2";  
    }  
}
```

Важно: в определении функций и переменных, к которым мы хотим получить доступ, должен присутствовать модификатор доступа **public**, а для элементов управления свойство **Modifiers** должно быть установлено в **Public**.

Контрольные вопросы

1. Что такое проект? Из каких основных файлов он состоит?

2. Что такое форма? Как добавить форму к проекту?
3. Как из одной формы по кнопке открыть другую форму? Какие изменения должны быть внесены в модуль вызывающей формы?
4. Как из одной формы сослаться на объект, находящийся в другой форме?
5. Как откомпилировать приложение?
6. Как запустить приложение на выполнение?
7. Как приостановить выполнение приложения?
8. Как прервать выполнение приложения, например в случае зависания?
9. Как получить доступ к опциям проекта?
10. Как открыть существующий проект?
11. Как открыть проект из числа недавно использовавшихся проектов?
12. Как сохранить все файлы, входящие в проект?
13. Как закрыть весь проект со всеми файлами, входящими в него?
14. Как добавить к проекту существующую форму?

Элементы управления форм

В данном разделе мы познакомимся с основными элементами управления форм: кнопками, флажками, меню, списками и т.д. Элементы управления формы могут быть как визуальными, т.е. видимыми, так и не визуальными, т.е. видимыми только на этапе проектирования приложения. Все элементы управления располагаются на **Панели элементов**. **Панель элементов** состоит из нескольких вкладок:

- **Стандартные элементы управления** – содержит основные элементы управления форм, такие как метки, списки, кнопки и т.д.
- **Контейнеры** – элементы контейнеры.
- **Меню и панели инструментов** – меню и кнопочные панели.
- **Данные** – элементы для работы с данными (базами данных).
- **Компоненты** – различные вспомогательные компоненты, такие как Timer, ImageList, Process, DirectorySearcher.
- **Печать** – элементы для печати.
- **Диалоговые окна** – для создания стандартных диалоговых окон Windows.
- **Отчеты** – компоненты для создания отчетов.

Каждый элемент управления является объектом соответствующего класса и обладает набором свойств, методов и событий. Свойства можно задавать либо на этапе проектирования через окно свойств, либо программным путем. Некоторые свойства доступны только на этапе выполнения программы. События задают поведение объекта при наступлении определенных событий в системе и доступны через окно свойств. Все элементы управления обладают одним общим свойством *Name*, которое позволяет ссылаться на элемент управления в программе. Начнем с самых простых элементов управления.

Элемент Label

Этот элемент находится на вкладке «Стандартные элементы управления» и позволяет выводить текст в окне. Основные свойства приведены в табл. 16.

Таблица 16

Свойство	Описание
Name	Задает имя объекта, используется для ссылки на объект в программе
Text	Строка, задает тот текст, который будет выведен в окне
Autosize	Если свойство равно true, то рамка объекта растягивается по размеру

	текста.
Font	Задаёт шрифт (размер, стиль, подчеркивание, зачеркивание)
Image	Задаёт рисунок, связанный с текстом
TextAlign	Задаёт выравнивание текста относительно рамки
BackColor	Задаёт цвет фона

Элемент Textbox

Этот элемент также располагается на вкладке «Стандартные элементы управления», это поле редактирования, которое позволяет пользователю вводить текст. Основные свойства элемента приведены в табл. 17:

Таблица 17

Свойство	Описание
Text	Строка, содержит то, что ввел пользователь
TextAlign	Задаёт выравнивание текста относительно рамки
Lines	Свойство типа массив, содержит многострочный текст, который введен в поле.
MultiLine	Если свойство равно true, то элемент будет показывать многострочный текст
Location	Задаёт расположение элемента относительно окна
Font	Шрифт текста
BackColor	Цвет фона
ForeColor	Цвет текста
WordWrap	Если свойство равно true, то введенный текст разбивается на несколько строк по пробелам
PasswordChar	Задаёт символ, маскирующий ввод
ReadOnly	Если равен true, элемент только для чтения
ScrollBars	Позволяет добавить полосу скроллинга, если равен true

Список основных событий, связанных с этим элементом приведен в табл. 18:

Таблица 18

Событие	Описание
Enter	Наступает, когда элемент получает фокус ввода
Leave	Наступает, когда элемент теряет фокус ввода
TextChanged	Наступает, когда меняется свойство Text
Click	Наступает, при щелчке на элементе управления

Элемент Button

Это кнопка. Основное событие Click – щелчок на кнопке. Основные свойства приведены в табл. 19:

Таблица 19

Свойство	Описание
Text	Текст на кнопке
TextAlign	Выравнивание текста относительно рамки
Size	Размер кнопки
Font	Шрифт
ForeColor	Цвет букв
BackColor	Цвет фона
Image	Картинка на кнопке

Выполним несложное упражнение, создадим «Простой калькулятор». Пользователь вводит в поля формы два числа. По кнопке «Вычислить» выполняется подсчет суммы, произведения, разности и отношения этих чисел.

Создадим приложение Windows Forms. Поместим на форму элементы управления как показано на рис. 11.

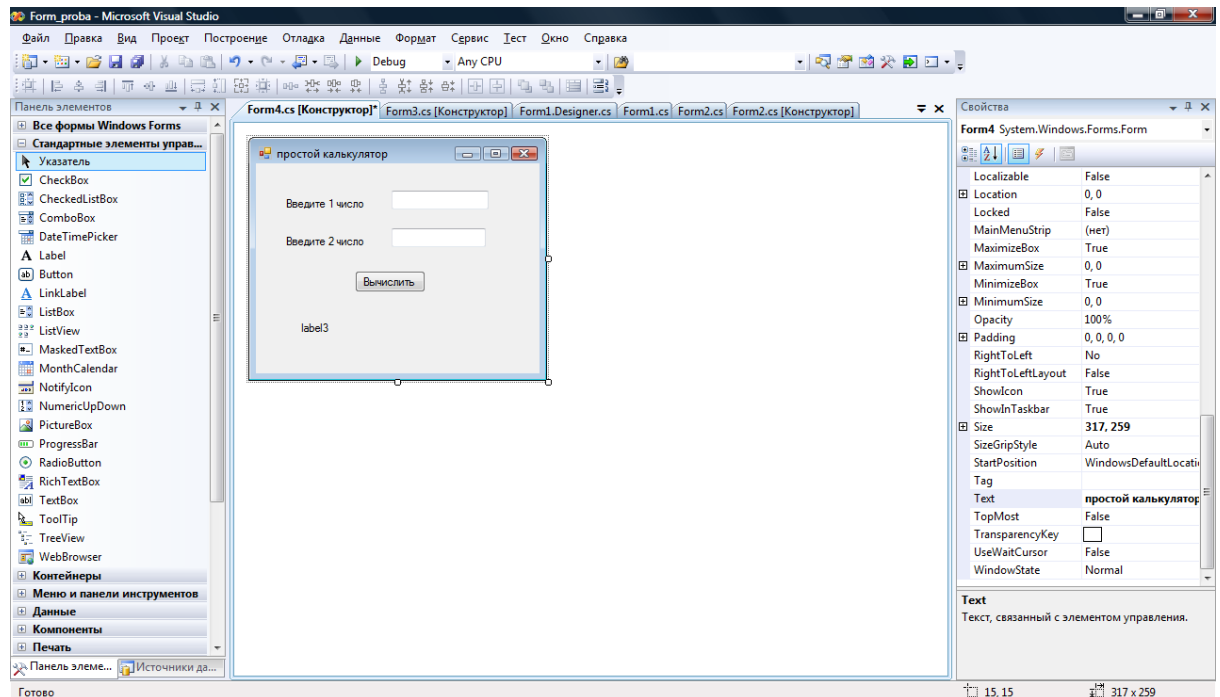


Рис. 11

На событие Click кнопки «Вычислить» напишем следующий код:

```
private void button1_Click(object sender, EventArgs e)
{
    double c, d, sum, multi, div, sub;

    // преобразование строки в тип double
    c = double.Parse (textBox1.Text);
    d = double.Parse (textBox2.Text);
    sum = c + d;
    multi = c * d;
    sub = c - d;

    if (d != 0) div = c / d; else div = 0;
    label3.Text = "Сумма = " + sum.ToString() +
    " Разность = " + sub.ToString() +
    " Произведение = " + multi.ToString() +
    " Отношение = " + div.ToString();
}
```

Вид формы на этапе выполнения приведен на рис. 12.

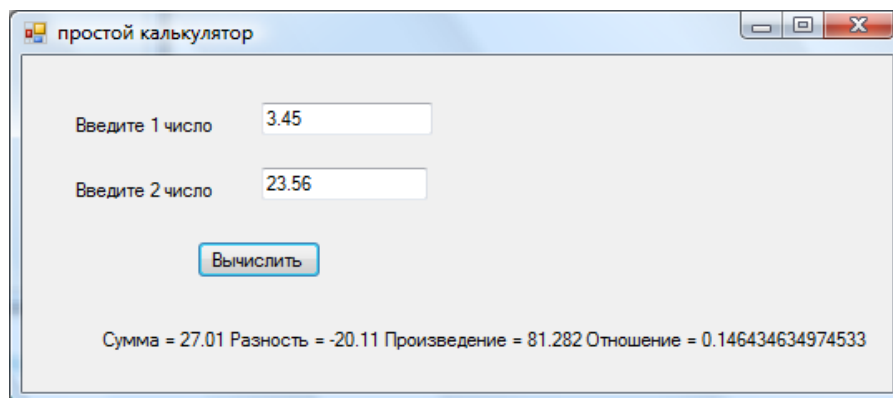


Рис. 12

Класс MessageBox

Класс *MessageBox* используется для выдачи сообщений пользователю в окне. Статический метод класса *Show*, позволяющий выдавать это диалоговое окно на экран, является перегруженным и имеет 21 вариант реализации. Рассмотрим один из вариантов. Упрощенный синтаксис приведен ниже:

Show(<текст в окне>[,<заголовок окна>,<набор кнопок>,<иконка>, <выбранная по умолчанию кнопка >])

Метод *Show* возвращает значение типа *DialogResult*, которое представляет собой код выбранной пользователем кнопки.

Рассмотрим более подробно параметры метода *Show*. Все параметры, кроме первого, являются необязательными. Первый и второй параметры метода строкового типа. Остальные параметры представляют собой перечислимые типы (*Enum*), которые с помощью набора констант задают кнопки, иконки в окне и кнопку по умолчанию.

Тип *MessageBoxButtons* задает набор кнопок в окне. Набор констант типа *MessageBoxButtons*, приведен в табл. 20:

Таблица 20

Значение	Описание
AbortRetryIgnore	Окно содержит кнопки Abort, Retry и Ignore.
OK	Окно содержит кнопку OK.
OKCancel	Окно содержит кнопки OK и Cancel.
RetryCancel	Окно содержит кнопки Retry и Cancel.
YesNo	Окно содержит кнопки Yes и No.
YesNoCancel	Окно содержит кнопки Yes, No, и Cancel.

Тип *MessageBoxIcon* задает иконку в окне. Набор констант типа *MessageBoxIcon*, приведен в табл. 21:

Таблица 21

Значение	Описание
Asterisk	Сообщение содержит символ, состоящий из маленькой буквы I в кружке
Error	Пиктограмма ошибочного сообщения
Exclamation	Пиктограмма содержит восклицательный знак
Hand	Пиктограмма содержит символ, состоящий из белого крестика в кружке на красном фоне.
Information	Информационная пиктограмма
None	Ничего не содержит.

Question	Пиктограмма содержит вопросительный знак
Stop	Пиктограмма содержит символ, состоящий из белого крестика в кружке на красном фоне.
Warning	Пиктограмма содержит символ, состоящий восклицательного знака в треугольнике на желтом фоне.

Перечисление `MessageBoxDefaultButton` определяет кнопку, выбранную по умолчанию. В таблице приведены члены перечисления:

Таблица 22

Значение	Описание
Button1	Первая кнопка в окне сообщения является кнопкой по умолчанию
Button2	Вторая кнопка в окне сообщения является кнопкой по умолчанию
Button3	Третья кнопка в окне сообщения является кнопкой по умолчанию

Метод `Show` возвращает значение типа `DialogResult`, который предоставляет набор констант, сигнализирующих какая из кнопок выбрана пользователем (см. табл. 23):

Таблица 23

Значение	Описание
Abort	Нажата кнопка Abort.
Cancel	Нажата кнопка Cancel.
Ignore	Нажата кнопка Ignore.
No	Нажата кнопка No.
None	Ничего не нажато.
OK	Нажата кнопка OK.
Retry	Нажата кнопка Retry.
Yes	Нажата кнопка Yes .

Покажем на примерах использование класса `MessageBox`.

Пример 1. Выдача сообщения в окне

```
MessageBox.Show("Текст", "Заголовок окна");
```

Вид окна на рис. 13:

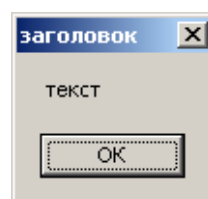


Рис. 13

Пример 2. Использование иконки в окне:

```
MessageBox.Show("Hello!", "Окно ввода", MessageBoxButtons.OK,
    MessageBoxIcon.Exclamation);
```

Вид окна на рис. 14:

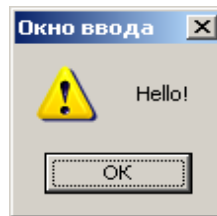


Рис. 14

Пример 3. Использование нескольких кнопок

```
// удалить текущий элемент
if (MessageBox.Show("Вы уверены?", "Вопрос", MessageBoxButtons.OKCancel,
    MessageBoxIcon.Question) == DialogResult.OK)
    listBox1.Items.RemoveAt(listBox1.SelectedItem);
```

Вид окна на рис. 15:

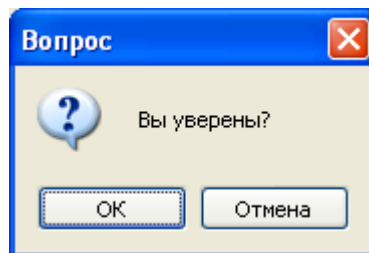


Рис. 15

Пример 4. Кнопка «Отмена», выбранная по умолчанию:

```
if (MessageBox.Show("Вы уверены?", "Вопрос", MessageBoxButtons.OKCancel,
    MessageBoxIcon.Question,
    MessageBoxDefaultButton.Button2) == DialogResult.OK)
{
    MessageBox.Show("ok");
}
```

Вид окна на рис. 16:

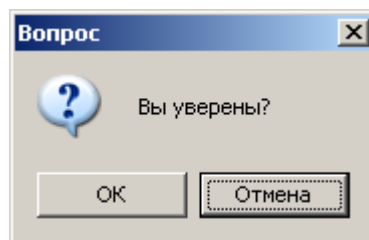


Рис. 16

Списки ListBox и ComboBox

Всего предлагается три вида списков:

- ComboBox
- ListBox
- ListView.

Список ComboBox это комбинированный список, т.е. поле редактирования с выпадающим списком. С помощью поля редактирования можно вводить новые элементы в список. ListBox – это прокручиваемый список, новые элементы в список вводить нельзя.

Элемент ListView позволяет выводить списки 5-ю различными способами. Этот многофункциональный элемент мы рассмотрим несколько позже.

Элементы ListBox и ComboBox имеют много общих свойств и методов. Общие свойства списков приведены в табл. 24, общие методы в табл. 25.

Таблица 24

Свойство	Описание
Items	Свойство-коллекция, содержит значения элементов списка Имеет набор встроенных методов для работы со списком: 1. Add(<что вставлять>) – добавить элемент в конец списка 2. Insert(<номер куда вставлять>, <что вставлять>) – вставить элемент в нужную позицию списка 3. Remove(<номер элемента>) – удаление элемента 4. Clear() – удалить все элементы
SelectedIndex	Задаёт номер выделенного элемента, нумерация с нуля. Если свойство равно -1(минус 1) то ни один элемент не выбран.
Sorted	Если равен true, элементы отсортированы
DropDownStyle	Задаёт стиль списка: DropDown – обычный комбинированный список DropDownList – прокручиваемый список, т.е. без поля редактирования.

Таблица 25

Название метода	Описание
Clear	Очистить список

Элементы в список можно вводить либо на этапе проектирования через окно свойств, либо программным путем. Рассмотрим второй вариант: внесение изменений в список программным путем.

Выполним несложное упражнение. Создадим прокручиваемый список типа ListBox и несколько кнопок, с помощью которых будут добавляться, вставляться, удаляться элементы списка. Внешний вид формы на этапе проектирования приведен на рис. 17:

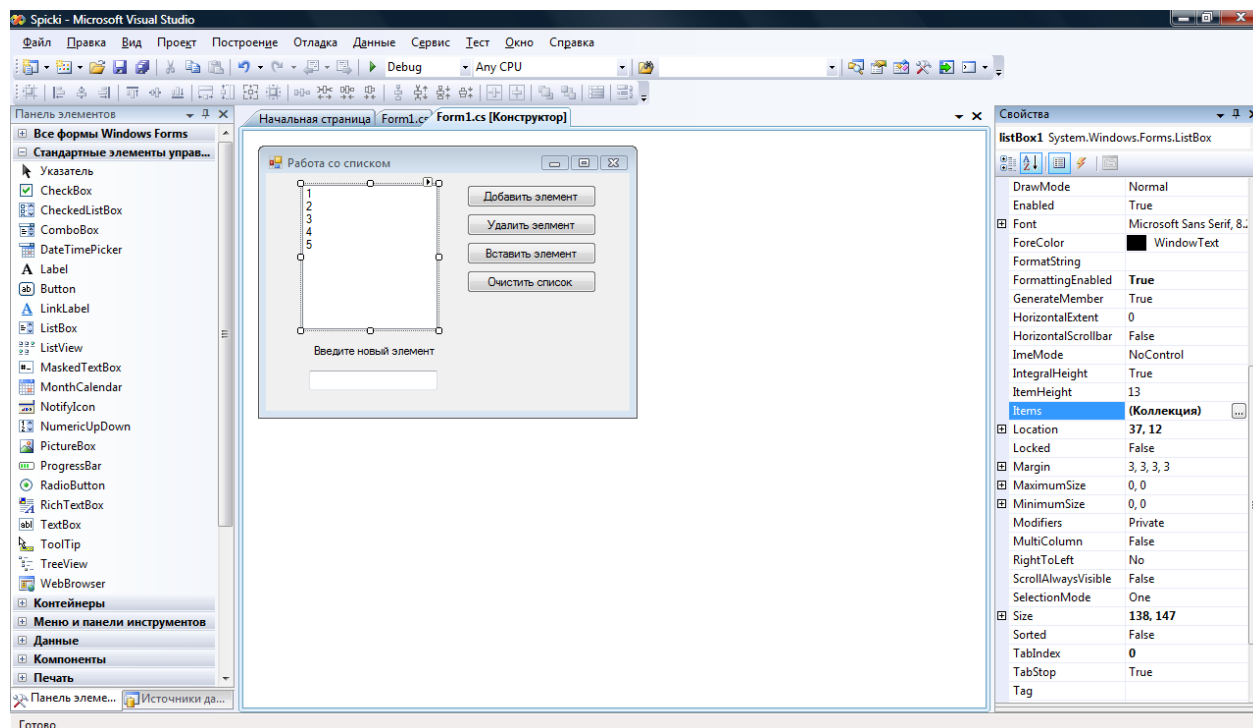


Рис. 17

Новый элемент списка будет вводиться с помощью элемента `textBox1`. По кнопке «Добавить» элемент добавляется в конец списка. По кнопке «Удалить» удаляется выделенный элемент списка. По кнопке «Вставить» новый элемент вставляется перед выделенным. По кнопке «Очистить» все элементы списка удаляются. Программный код, реализующий данные функции кнопок приведен ниже:

```
private void button2_Click(object sender, EventArgs e)
{
    // добавить новый элемент
    listBox1.Items.Add(textBox3.Text);
}

private void button3_Click(object sender, EventArgs e)
{
    // удалить выбранный элемент
    if (MessageBox.Show("Вы уверены?", "Вопрос", MessageBoxButtons.YesNo,
        MessageBoxIcon.Question) == DialogResult.Yes)
        listBox1.Items.RemoveAt(listBox1.SelectedIndex);
}

private void button5_Click(object sender, EventArgs e)
{
    // очистить список
    listBox1.Items.Clear();
}

private void button4_Click(object sender, EventArgs e)
{
    // вставить элемент перед текущим
    listBox1.Items.Insert(listBox1.SelectedIndex, textBox3.Text);
}
```

Контрольные вопросы

1. Как написать текст в окне?
2. С помощью какого элемента управления можно ввести информацию с клавиатуры?

3. Для чего используются кнопки? Как задать подпись на кнопке? Как задать картинку для кнопки?
4. Что такое событие? Что такое обработчик события? Какие события кнопки вам известны?
5. Какие виды списков Вы знаете? Чем они отличаются?

Задания для самостоятельной работы

Используя рассмотренные выше элементы управления форм создать приложения Windows Forms с удобным пользовательским интерфейсом для выполнения следующих заданий.

1. Создать простой калькулятор на 4 операции. Примерный внешний вид формы приведен на рис. 18:

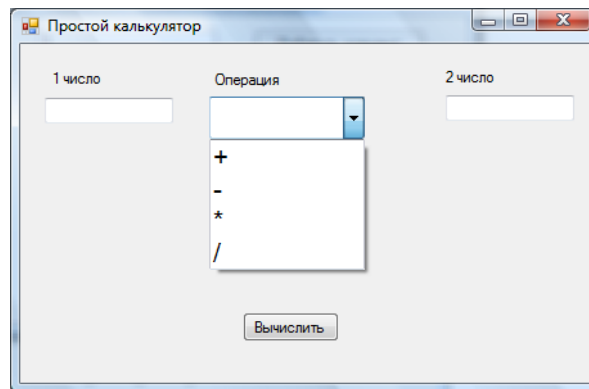


Рис. 18

2. Заданы тарифы на оплату электроэнергии. За первые 100 киловатт-час (квт. час) плата А рублей за каждый квт. час. За следующие 100 квт. час (т.е. от 101 до 200 квт. час) оплата В руб. за каждый квт. час. А если выше 200, то оплата С руб. Составьте алгоритм, вычисляющий, сколько надо заплатить за использование N квт. часов электроэнергии.
3. Электронные часы показывают Р часов, М минут, S секунд. Напишите программу, вычисляющую время в часах, минутах и секундах, которое будут показывать часы через 2 минуты 10 секунд.
4. В кассе имеются только 3-х и 5-ти рублевые купюры (как это было в 1980 году). Составить программу, которая бы "выплачивала" такими купюрами любую сумму более 7 рублей.
5. Ввести коэффициенты квадратного трехчлена, вывести значения корней квадратного уравнения. В случае если корни не существуют выдать сообщение.

Циклы

Цикл со счетчиком for

Цикл *for* используется, как правило, если известно число повторов (шагов) цикла. В таких циклах имеется переменная, которая называется счетчиком цикла. Оператор цикла состоит из двух частей: заголовка и тела. Синтаксис оператора *for* имеет следующий вид:

```
for ([<начальные значения>; <условия выхода из цикла>; <инкремент>])
{ <тело цикла>
}
```


В части *начальные значения* инициализируются переменные, которые будут использоваться в теле цикла. В части *условия выхода из цикла* задается условие выхода из цикла. *Инкремент* задает шаг, на который будет изменяться значение счетчика цикла. Эти три основных элемента цикла должны быть разделены точкой с запятой. Любой из этих элементов является необязательным и может быть опущен.

Цикл *for* выполняет оператор или блок операторов до тех пор, пока условие выхода из цикла не станет равным *false*. В теле цикла должен быть код, влияющий на условие, иначе получим бесконечный цикл.

В случае зависания программы (зацикливания) нажмите Shift+F5.

Примеры использования оператора *for* приведены ниже.

Пример 1

Вычисляется значение факториала чисел от 1 до 5. Обратите внимание переменная счетчик может быть объявлена внутри цикла. В этом случае ее область видимости ограничена рамками цикла.

```
int fact = 1;
for (int i = 1; i <= 5; i++)
    fact *= i;
    MessageBox.Show("fact= " + fact);
```

Пример 2

Вычисляются числа от 3 до 100, кратные трем:

```
string str = "";
for (int i = 3; i <= 100; i += 3)
    str += i.ToString() + " ";
    MessageBox.Show("числа, кратные трем от 3 до 100 - " + str);
```

Пример 3

Демонстрирует цикл без тела, который подсчитывает сумму чисел от 1 до 10:

```
int i, sum;
for (i = 1, sum = 0; i <= 10; sum += i, i++) ;
```

Пример 4

Демонстрирует бесконечный цикл:

```
for ( ; ; )
{ . . .
}
```

В этом случае в теле цикла должен быть оператор, выполняющий выход из цикла по условию.

Цикл с предусловием – while

Циклы с предусловием используются тогда, когда выполнение операторов тела цикла зависит от некоторого условия, которое проверяется перед очередным шагом цикла, поэтому цикл носит название цикла **с предусловием**. Синтаксис цикла *while* приведен ниже:

```
while <условие>
{ <операторы тела цикла>
}
```

Операторы тела цикла выполняются до тех пор, пока условие истинно. При использовании цикла с предусловием необходимо понимать, что:

1. значение условия выполнения цикла должно быть определено до начала цикла;
2. если условие входа в цикл ложно, то происходит выход из цикла;
3. хотя бы один из операторов тела цикла должен влиять на условие выполнения цикла, иначе цикл будет повторяться бесконечное число раз;
4. цикл может не выполниться ни разу, если условие выполнения цикла ложно.

Пример 1. Цикл `while` используется для вывода букв латинского алфавита:

```
char ch;
ch = 'a';
while (ch <= 'z')
{
    Console.Write(ch + " ");
    ch++;
}
```

Пример 2. Вывести квадраты чисел от 1 до 1000

```
int i = 1;
while (i * i < 1000)
{
    Console.WriteLine("{0} ", i * i);
    i++;
}
```

Цикл с постусловием `Do`

Цикл с постусловием напоминает цикл с предусловием. Отличие заключается в том, что проверка выхода из цикла выполняется в конце цикла. Синтаксис цикла имеет следующий вид:

```
do
{ <тело цикла>
} while (усл);
```

Операторы тела цикла будут выполняться пока условие **истинно**. Цикл `do..while` выполнится хотя бы один раз, т.к. проверка условия выполняется в конце цикла.

Пример 1. В следующем примере цикл выполняется до тех пор, пока пользователь не введет символ `*`:

```
char ch;
do
{
    Console.WriteLine("введите символ, после чего нажмите ENTER:");
    ch = (char)Console.Read(); // считывание символа
} while (ch != '*');
```

Использование операторов `Continue` и `Break` в циклах

В теле цикла могут использоваться операторы `break` и `continue`.

Оператор `break` обеспечивает выход из циклов и продолжает выполнение программы с оператора, непосредственно следующего за циклом. Например:

```
int n = 100;
for ( ; ; )
{
    n -= 2;
    if (n == 0) break;
}
```

В данном учебном примере в теле бесконечного цикла выполняется изменение переменной *n*, если значение *n* становится равным 0, то с помощью оператора *break* выполняется выход из цикла.

Оператор *continue* позволяет перейти на начало цикла. В следующем примере печатаются все целые числа от 1 до 10 кроме числа 5:

```
for (int n = 1; n <= 10; n++)
{
    if (n == 5) continue;
    Console.Write("{0} ", n);
}
```

Случайное число. Класс Random

При отладке широкого класса алгоритмов ввод значений переменных с клавиатуры, либо программным путем с помощью инициализации переменных является очень трудоемким. Значения переменных можно получать во время выполнения программы с помощью генерации случайных чисел, для этого используется класс Random. В табл. 26 приведены конструкторы класса Random

Таблица 26

Название	Описание
Random ()	Создает экземпляр класса, основываясь на значении текущего времени.
Random (Int32)	Создает экземпляр класса, основываясь на начальном значении, заданном в качестве входного параметра

Некоторые методы класса Random приведены в табл. 27:

Таблица 27

Метод	Описание
Next ()	Возвращает случайное не отрицательное целое число от 0 до int.MaxValue. Инициализация датчика случайных чисел основывается на системном времени.
Next (Int32)	Возвращает случайное не отрицательное целое число, не превышающее значение аргумента.
Next (Int32, Int32)	Возвращает целое число, равномерно распределенное в интервале от 1-го параметра до 2-го параметра.
NextDouble()	Возвращает дробное число от 0.0 до 1.0

Пример 1. В результате выполнения этого кода будут выданы случайные числа от 0 до 8:

```
// создается объект класса Random
Random rnd = new Random();
for (int i = 1; i < 10; i++)
{
    Console.Write("{0} ", rnd.Next(9));
}
```

Контрольные вопросы

1. Что такое цикл? Какие виды циклов используются в C#?
2. Для чего используется цикл со счетчиком? Как работает цикл со счетчиком? Какой оператор C# реализует цикл со счетчиком?
3. В каких случаях следует использовать цикл с предусловием? Какой оператор языка C# реализует цикл с предусловием? Может ли цикл с предусловием не выполниться

ни разу? В каких случаях? Может ли цикл с предусловием выполняться бесконечное число раз? В каких случаях?

4. Цикл с постусловием: в каких случаях используется? Какой оператор языка C# позволяет реализовать цикл с постусловием? Может ли цикл с постусловием не выполниться ни разу?
5. Что необходимо предпринять, чтобы прервать программу, которая заиклилась (зависла)?
6. Какой оператор позволяет немедленно выйти из цикла? Какой оператор позволяет перейти на начало цикла?

Задания для самостоятельной работы

- 1) Задать 5 случайных целых чисел с помощью класса Random. Вывести все эти числа на экран. Вывести значение среднего арифметического этих чисел. Найти минимальное число. Определить, получаются одинаковые или разные последовательности?
- 2) Вывести все квадратные числа до ста, т.е. числа, являющиеся точными квадратами до 100. Каждое число с новой строки.
- 3) Вывести все "треугольные" числа, т.е. числа вида $n(n+1)/2$ (где n =целое) до 100. Каждое число с новой строки.
- 4) Вывести на страницу 100 случайных двузначных целых чисел по 10 в ряд в ячейках таблицы.
- 5) Найти сумму 100 первых чисел ряда: $1+1/2+1/3+1/4+\dots$
- 6) Напишите программу, которая вычисляет частичную сумму ряда:
$$1-1/3+1/5-1/7+1/9-\dots$$

и сравнивает полученное значение с $\pi/4$ (при суммировании достаточно большого количества членов этого ряда, величина частичной суммы приближается к $\pi/4$).
- 7) Найти сумму ряда чисел $1+2*2+3*3+\dots+n*n$, где n вводится с клавиатуры. Вывести результат в окне.
- 8) Найти сумму ряда чисел: $1+1/2+1/3+\dots+1/n$, где n вводится с клавиатуры. Вывести результат в окне.
- 9) По введенному числу M распечатать все 3-х значные десятичные числа, сумма цифр которых равна M . Посчитать также количество таких чисел или сообщить, что таких чисел нет.
- 10) Ежемесячная стипендия студента составляет A рублей, а расходы на проживание превышают стипендию и составляют B рублей в месяц. Рост цен ежемесячно увеличивает расходы на проживание на 3%. Напишите программу расчета необходимой суммы денег, которые нужно единовременно попросить у родителей, чтобы можно было прожить учебный год(10 месяцев), используя только эти деньги и стипендию. Создайте удобный интерфейс для ввода информации.
- 11) У студента имеются накопления S рублей. Ежемесячная стипендия составляет A рублей, а расходы на проживание превышают стипендию и составляют B рублей в месяц. Рост цен ежемесячно увеличивает расходы на 3%. Напишите программу расчета времени (в месяцах), которое может прожить студент, используя только эти накопления и стипендию. Подумайте, изменится ли алгоритм, если инфляция не постоянна.
- 12) Дан текст программы на языке C. Требуется найти три способа изменить в нем один символ так, чтобы программа стала печатать 20 звездочек. Изменить — это значит не добавить, не убрать и не переставить, а именно поменять.

```
main()  
{ int i, N=20;
```

```

        for (i = 0 ; i < N ; i++)
        { printf("*"); }
    }

```

Массивы

Одномерные массивы

Под массивом понимается однотипный набор данных, объединенных общим именем. Доступ к элементу массива выполняется по имени массива и порядковому номеру элемента или индексу. Нумерация элементов начинается с 0. Данные в массиве организованы так, что ими легко манипулировать, в этом и заключается главное достоинство массивов.

Перед использованием массив необходимо объявить. Массивы в C# несколько отличаются от других C-подобных языков. Массив в C# это тип, производный от класса `System.Array`. Поэтому все массивы в C# обладают общим набором членов. Индексы массива могут содержать элементы любого встроенного типа данных C#. Массивы могут быть как одномерными, так и многомерными.

Для объявления одномерного массива используется следующий синтаксис:

```
<тип>[ ] <имя массива> = new <тип>[размер] ;
```

Например:

```
int[] temp = new int[3];
```

В этом примере объявляется массив с именем `temp`, состоящий из 3-х элементов типа `int`.

Так как массив представляет собой объекты, то процесс создания массива состоит из двух этапов. На первом этапе объявляется имя массива – или переменная ссылочного типа. На втором этапе для массива выделяется память и переменной массива присваивается ссылка на эту область памяти, т.е. с помощью оператора `new` память массиву выделяется автоматически. Например:

```
double[] sample; // объявление переменной ссылочного типа
sample = new double[10];
```

Только после выполнения второго оператора переменной `sample` присваивается ссылка на массив.

Следующие объявления массивов являются ошибочными:

```
int k[ ]; //Неверно!
int k[3]; //Неверно!
int[3] k; //Неверно!
```

Инициализация массива

Инициализацию элементов массива можно выполнять несколькими способами:

1. Вручную с помощью нескольких операторов присваивания, как показано в следующем примере.

```
temp[0] = -5; temp[1] = 4; temp[2] = 55; //Задаем элементы массива
```

Инициализировать массив сразу при его создании. Общий синтаксис инициализации одномерного массива выглядит так:

```
<тип>[ ] <имя массива> = { знач1, знач2, ... значN }
```

Элементам массива значения присваиваются по очереди слева направо, начиная с элемента с индексом 0. Например:

```
int[] nums = { 99, 23, 45, 13, 78, 46, 63, 8, 2, 55, 0, 10 };
```

2. С помощью случайных чисел. В следующем примере массив из 10 целых чисел инициализируется случайным способом числами не больше 100.

```
int[] nums = new int[10];
Random rnd = new Random();
string str = "";
for (int i = 0; i < 10; i++)
{
    nums[i] = rnd.Next(100);
    str += nums[i].ToString () + " ";
}
```

Рассмотрим пример: требуется написать программу, выводящую пропись трехзначного числа, например число 234 должно выводиться как «двести тридцать четыре». Внешний вид формы приведен на рис. 19:

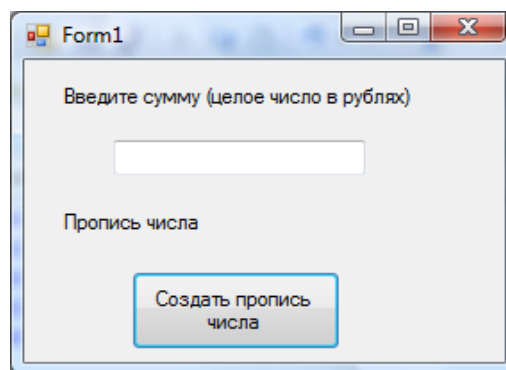


Рис. 19

Программный код на кнопку «Создать пропись числа» имеет следующий вид:

```
private void button1_Click(object sender, EventArgs e)
{
    // массив единиц
    string[] ed =
    { "", "один", "два", "три", "четыре", "пять", "шесть", "семь", "восемь", "девять" };
    // массив десятков
    string[] des = { "", "десять", "двадцать", "тридцать", "сорок",
    "пятьдесят", "шестьдесят", "семьдесят", "восемьдесят", "девяносто" };
    // массив сотен
    string[] sotni = { "", "сто", "двести", "триста", "четыреста",
    "пятьсот", "шестьсот", "семьсот", "восемьсот", "девятьсот" };
    // массив чисел от 10 до 19
    string[] teen = { "десять", "одиннадцать", "двенадцать", "тринадцать",
    "четырнадцать", "пятнадцать", "шестнадцать", "семнадцать", "восемнадцать",
    "девятнадцать" };
    int num; byte c_ed, c_des, c_sotni, temp;
    num = Int16.Parse(textBox1.Text);
    c_ed = (byte) (num % 10);
    temp = (byte) (num / 10);
    c_des = (byte) (temp % 10);
    c_sotni = (byte) (temp / 10);

    // пропись формируется в переменной txt
    string txt = "";
    txt += sotni[c_sotni] + " ";
    if (c_des == 1)
        txt += teen[c_ed];
}
```

```

else
{
    txt += des[c_des];
    if (c_ed > 0) txt += " " + ed[c_ed];
}
label2.Text = "пропись числа: " + txt;
}
}

```

Пропись числа выводится в элементе *label2*.

Многомерные массивы

Многомерным называется массив, который имеет две и более размерности, а доступ к отдельному элементу выполняется с помощью указания двух и более индексов. Самым простым является двумерный массив, который можно рассматривать, как информационную таблицу, первый индекс которой указывает номер строки, а второй – номер столбца. Например, объявляется двумерный массив размерностью 3 строки и 4 столбца.

```
int[ , ] table = new int[3,4];
```

В первой части оператора с помощью запятой в квадратных скобках [,] указывается, что создается переменная ссылочного типа для двумерного массива. С помощью оператора `new` происходит выделение памяти для массива при выполнении второй части оператора.

Для доступа к элементу массива необходимо указать два индекса, разделив их запятой:

```
table[2,3] = 100;
```

В этом примере первый индекс обозначает номер строки, а второй – номер столбца.

Ниже приведен фрагмент программы, выполняющий инициализацию такого массива числами от 1 до 12:

```

for (int i = 0; i < 3; i++)
    for (int j = 0; j < 4; j++)
        table[i, j] = i * 4 + j;

```

Инициализировать многомерные массивы можно, поместив список значений каждой размерности в отдельный блок, заключенный в фигурные скобки. Синтаксис инициализации многомерного массива выглядит так:

```

<тип>[ , ] <имя массива> = {
    {знач, знач, ... знач},
    {знач, знач, ... знач},
    ...
    {знач, знач, ... знач}
}

```

Здесь слово «знач» обозначает присваиваемое значение. Каждый внутренний блок присваивает значения элементам соответствующей строки. Блоки, содержащие значения, разделены запятыми. Например, предыдущий массив `table` можно инициализировать еще и так:

```

int[,] table = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9,10,11,12}
};

```

В следующем примере двумерный массив инициализируется случайными значениями. Созданные значения элементов массива отображаются в элементе *label*. Внешний вид формы приведен на рис. 20:

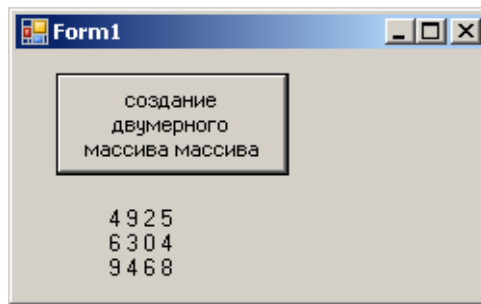


Рис. 20

Программный код на кнопку выглядит так:

```
int t, i, j;
int[,] table = new int[3, 4];
string str = "";
Random rnd = new Random();
for (i = 0; i < 3; i++)
{
    for (j = 0; j < 4; j++)
    {
        table[i, j] = rnd.Next(10);
        str += " " + Convert.ToString(table[i, j]);
    }
    str += "\n";
}
label1.Text = str;
```

Невыровненные (ступенчатые) массивы

Невыровненный или ступенчатый массив это внешний массив, состоящий из внутренних массивов разной длины. Следовательно, невыровненный массив можно использовать для создания таблицы со строками разной длины.

При объявлении невыровненного массива для указания каждой размерности используется отдельная пара квадратных скобок. Например, двумерный невыровненный массив объявляется следующим образом:

```
type [ ][ ] array_name = new type[size][ ];
```

size обозначает количество строк в массиве. Размерность же строк не указывается и память под них не выделяется. Для каждой строки необходимо выполнение отдельного оператора *new*. Такая технология позволяет определять строки различной длины. Например:

```
int[ ][ ] jagged = new int[3][ ];

jagged[0] = new int[2];
jagged[1] = new int[3];
jagged[2] = new int[4];
```

После выполнения этих операторов, массив *jagged* состоит из трех строк, первая содержит 2 элемента, вторая – 3 элемента, третья – 4 элемента.

Доступ к элементу можно получить следующим образом:

```
jagged[2][1] = 10;
```


Обратите внимание, что для доступа к элементам прямоугольного массива применяется иная форма доступа.

Самая важная и интересная возможность у ступенчатых массивов - это их "**не прямоугольность**". Так, в приведенном выше примере в первой "строке" массива два целых числа, а во второй - три. Часто это оказывается очень к месту.

Использование цикла `foreach`

Цикл `foreach` используется для обработки элементов коллекции. Коллекция это набор объектов. В C# определены несколько типов коллекций, одним из которых является массив. Синтаксис цикла `foreach`:

```
foreach (<тип> <переменная> in <коллекция>)
{ операторы тела цикла
}
```

При каждом шаге цикла переменная будет принимать очередное значение элемента из коллекции.

<тип> <переменная> - это тип и имя итерационной переменной, которая будет принимать значение элементов из коллекции. Обратите внимание, что при работе с массивом итерационная переменная доступна только для чтения.

Пример 1. В данном примере переменная `i` принимает значение очередного элемента массива `m`, в теле цикла суммируются все элементы массива, а сумма записывается в переменную `sum`.

```
int[] m = { -5, 4, 10 };
int sum = 0;
foreach (int i in m)
{
    sum += i;
}
```

Пример 2. Вывод элементов массива по порядку

```
int[] intArray = { 2, 4, 6, 8, 10, -2, -3, -4, 8 };
foreach (int i in intArray)
{
    System.Console.WriteLine(i);    // Вывод элементов массива по порядку
}
```

Контрольные вопросы

1. Что такое массив? Для чего используются массивы?
2. Как описать одномерный массив?
3. Для чего используется цикл `foreach`? Приведите пример использования цикла `foreach`.
4. Как получить доступ к элементу одномерного массива?
5. Как определяется размер массива?
6. Как описать двумерный массив? Как получить доступ к элементу двумерного массива?
7. Что такое не выровненный или ступенчатый массив?

Задания для самостоятельной работы

1. Создать массив, состоящий из `N` различных целых случайных двузначных положительных чисел. Написать программу определяющую:

- максимальное и минимальное число из этой последовательности;
 - сумму всех чисел из этой последовательности;
 - среднее арифметическое чисел этой последовательности.
2. Результат сдачи экзамена группой из N студентов находится в массиве *Rezultat*. Посчитать количество студентов, сдавших экзамен на 2,3,4 и 5. Создать значения элементов массива *Rezultat* случайным образом.
 3. Написать программу подсчета суммы всех отрицательных элементов заданного целочисленного массива размером 30 элементов. Если отрицательных элементов нет, то сообщить об этом. Значения элементов массива задаются случайным образом.
 4. Создать целочисленный массив из 28 элементов. Элементы массива создаются случайным образом и могут принимать значения от 0 до 100 – процент выполнения учащимися домашних заданий по информатике. Для получения положительной оценки за год требовалось набрать не менее 40 баллов. Написать программу, которая находит и выводит минимальный балл среди учащихся, получивших за год положительную оценку. Гарантируется, что в классе хотя бы один учащийся получил за год положительную оценку.
 5. Создать вещественный массив из 50 элементов. Элементы массива создаются случайным путем и могут принимать произвольные значения. Напишите программу, которая находит и выводит номер первого отрицательного элемента массива или сообщение, что такого элемента нет.
 6. Дан целочисленный массив из 30 элементов. Элементы массива могут принимать произвольные значения. С клавиатуры вводится целое число X. Напишите программу, которая находит и выводит наименьший номер элемента массива, равного X, или сообщение, что такого элемента нет.

Обработка исключений

Исключение представляет собой ошибку, происходящую во время выполнения программы. С помощью подсистемы обработки исключений для C# можно обрабатывать такие ошибки не вызывая краха программы.

Обработка исключений ускоряет процесс обработки ошибок, поскольку программа может содержать блок кода, именуемый обработчиком исключений. Этот блок кода будет выполняться автоматически в случае возникновения каких-либо ошибок. При появлении ошибки происходит ее обработка с помощью обработчика ошибок.

Другая причина, по которой важна обработка исключений состоит в том, что в C# определены стандартные исключения для наиболее часто встречающихся ошибок, таких как деление на ноль, выход индекса за границы диапазона, и т.д. Поэтому для устранения подобных ошибок в программе нужно только следить за возникновением соответствующих исключений и обрабатывать их.

Класс *System.Exception*

В языке C# исключения представлены классами. Все классы исключений могут быть унаследованы от встроенного класса исключений *Exception* из пространства имен *System*. В C# определены несколько встроенных исключений, например *DividedByZeroException* в случае деления на 0, *IndexOutOfRangeException* в случае выхода за границы индекса массива.

Обработка исключений в C# выполняется с помощью 4-х ключевых слов: *try*, *catch*, *throw*, *finally*. Рассмотрим их подробнее.

В блок *try* помещаются операторы программы, за выполнением которых необходимо следить на предмет возникновения исключений.

Возникающее исключение перехватывается и обрабатывается блоком *catch*. Для генерирования исключения применяется ключевое слово *throw*. В блок *finally* помещается код, который всегда выполняется при выходе из блока *try*.

Блоки *try* и *catch* работают совместно. Нельзя задать блок *try* и не задать *catch*, и наоборот.

Синтаксис:

```
try {  
    // блок кода, для которого выполняется мониторинг ошибок  
}  
catch (Exception1 exc)  
{ // обработчик исключения 1  
}  
catch (Exception2 exc)  
  
{ // обработчик исключения 2  
  
}
```

Exception1 и *Exception2* задают типы исключений. После генерации исключения происходит перехват его оператором *catch*. Тип исключения определяет выполняемый оператор *catch*. Если возникшее исключение соответствует типу исключения, указанному в операторе *catch*, то выполняется блок команд этого оператора, а остальные блоки игнорируются. Указание объекта *exc* не является обязательным. Операторы *catch* вызываются только в том случае, если генерируется исключение.

В следующем простом примере демонстрируется перехват исключения в случае выхода за пределы индекса массива, действительно элемента с индексом 10 не существует.

```
int[] nums = new int[5];  
// инициализация элементов массива  
...  
try  
{ nums[10]=15;  
  Console.WriteLine("Этот оператор не будет работать");  
}  
catch(IndexOutOfRangeException)  
{  
  Console.WriteLine("Выход за пределы диапазона индекса!");  
}
```

В следующей программе выполняется деление элементов одного массива на элементы другого массива. В случае деления на 0 возникает исключение *DivideByZeroException*, которое обрабатывается блоком *catch*.

```
int[] num1 = { 8, 3, 12, 5 };  
int[] num2 = { 2, 0, 3, 0 };  
for (int i=0; i<num1.Length ; i++)  
    try {  
        Console.WriteLine (num1 [i] + "/" + num2 [i] + "=" +  
            num1[i]/num2[i]);  
    }  
    catch (DivideByZeroException)  
    { Console.WriteLine ("Нельзя делить на 0");  
    }  
}
```

Результат выполнения данной программы приведен на рис. 21:

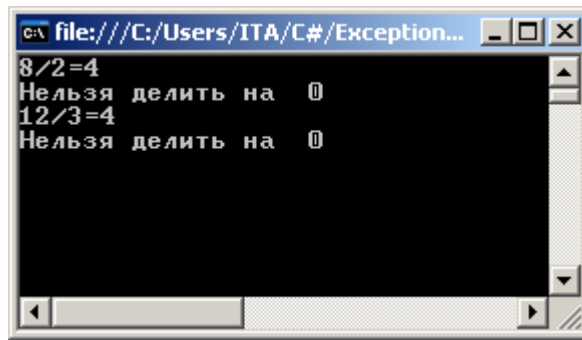


Рис. 21

Работа со строками. Классы String и StringBuilder

Класс String

Строки в C# - это объекты, экземпляры класса System.String. Вообще говоря, в C# есть тип string, но класс System.String является более продвинутым, так что его использование часто оказывается более оправданным и простым. Самым простым способом создания объекта типа String является использование строкового литерала:

```
String str = "это строка!";
```

Можно также создать объект типа String из массива типа char. Например:

```
char[] stroka = { 's', 'e', 'z', 'a', 'm' };
```

Класс String имеет множество методов и свойств, некоторые из которых перечислены ниже.

Свойство Length. Возвращает длину строки. Например:

```
String s="Hello";
int k=s.Length; //переменная k равна 5
```

Метод Compare. Статический метод, сравнивающий две строки. Метод возвращает 0, если строки равны, отрицательное значение, если первая строка меньше второй и положительное значение, если первая строка больше второй (больше и меньше в алфавитном смысле). Например:

```
String s1 = "cream", s2 = "ice", s3 = "nothing";
//Напечатает 0, т. к. строка сравнивается сама с собой
Console.WriteLine(String.Compare(s1, s1));
//Выдаст 1, т. к. "ice" ,больше "cream"
Console.WriteLine(String.Compare(s2, s1));
//Выдаст -1, т. к. "ice" меньше "nothing"
Console.WriteLine(String.Compare(s2, s3));
```

Метод Equals, возвращает true, если строки равны, false - если не равны. Может быть как статическим, так и не статическим. Пример использования:

```
String s1="apple", s2="orange";
// статический метод
Console.WriteLine(String.Equals(s1, s2).ToString());
// метод экземпляра
Console.WriteLine(s1.Equals(s2).ToString());
```

Метод Substring. Позволяет извлечь из строки подстроку. Метод имеет 2 параметра. Первый параметр задает номер начальной позиции подстроки (нумерация с нуля), второй - количество символов подстроки. Метод возвращает подстроку. Пример использования:

```
String s1 = "подстрока", s2;  
s2 = s1.Substring(3, 5);  
  
Console.WriteLine(s2); //будет выведено "строк"
```

Метод Insert. Вставляет в строку другую строку. Метод возвращает новую строку. Первый параметр задает позицию, с которой будет вставлена строка (нумерация с нуля), второй параметр задает вставляемую строку. Пример использования:

```
String s1 = "цена = руб", s2;  
s2 = s1.Insert(7, "213 ");  
Console.WriteLine(s2); //будет выведено "цена = 213 руб"
```

Метод IndexOf возвращает позицию подстроки в строке. Если подстрока не найдена, то возвращает -1(минус 1). Пример использования:

```
String s1 = "Hello Mary!", s2 = "Mary", s3 = "Alex";  
Console.WriteLine(s1.IndexOf(s2)); //будет выведено 6  
Console.WriteLine(s1.IndexOf(s3)); //будет выведено -1
```

Метод LastIndexOf возвращает позицию последнего вхождения подстроки в строку. Синтаксис метода аналогичен синтаксису метода IndexOf. Например:

```
String s1 = "один два три один", s2 = "один";  
int k= s1.LastIndexOf(s2);  
Console.WriteLine(k); // будет выведено 13
```

Метод Replace производит замену подстроки в строке. Первый параметр задает подстроку, которая будет заменена, второй параметр задает подстроку замены. В следующем примере подстрока «bc» будет заменена на подстроку «q» везде, где она будет найдена:

```
String s1 = "цена = 100 руб", s2 = "руб", s3;  
s3 = s1.Replace(s2, "$");  
Console.WriteLine(s3); //будет выведено цена = 100 $
```

Методы EndsWith и StartsWith. Проверяют, не завершается ли или не начинается ли строка с или на заданную строку соответственно. Пример использования:

```
String s1 = "array";  
if (s1.StartsWith("ar"))  
    Console.WriteLine("Строка начинается на \"ar\"");  
Else  
    Console.WriteLine("Строка не начинается на \"ar\"");
```

Методы ToUpper и ToLower переводят строку в верхний или нижний регистр соответственно. Пример использования:

```
String s1 = "ПРИВЕТ";  
s1 = s1.ToLower();  
Console.WriteLine(s1); // будет выведено «привет»
```

Методы Trim, TrimEnds и TrimStart. Удаляют пробельные символы из начала и конца строки (метод Trim), только с конца строки (метод TrimEnds) и только с начала строки (метод TrimStart). Пример использования:

```
String s1 = "  Merry Cristmas!  ";  
System.Console.Write(s1.Trim());
```

Заметьте, что при изменении строки старый экземпляр класса `System.String` фактически уничтожается, и создается новый с тем же именем и измененным содержанием. Это означает, что при интенсивной работе со строками программа может работать не так быстро, как хотелось бы. Если мы не хотим, чтобы каждый раз создавался новый экземпляр класса, то вместо класса `System.String` лучше использовать класс `StringBuilder`, который мы рассмотрим ниже.

Класс `StringBuilder`

Рассмотрим еще один класс для работы со строками - `StringBuilder`. Он принадлежит пространству имен `System.Text`.

Этот класс работает быстрее, чем класс `String`, так как при изменении строки, созданной как экземпляр класса `String`, каждый раз создается новый экземпляр класса, а старый уничтожается. При использовании же класса `StringBuilder` расходов на создание-уничтожение экземпляра класса нет - мы работаем всегда с одним экземпляром.

Обратите внимание, что для этого класса мы не можем использовать простое присваивание:

```
StringBuilder s="abc"; //Неправильно!
```

В этом случае мы должны действовать так:

```
StringBuilder s=new StringBuilder("abc"); //Правильно
```

У класса `StringBuilder` нет статических методов. Все его методы - динамические. Ниже перечислены основные свойства и методы класса `StringBuilder`:

Свойство `Length`. Возвращает длину строки. Пример использования:

```
int k=s.Length;
```

Свойство только для чтения **`MaxCapacity`**. Возвращает максимальное количество символов, которые можно записать в объект типа `StringBuilder`. Пример использования:

```
System.Console.WriteLine(s.MaxCapacity);
```

Метод `Append` позволяет добавить новую строку к существующей. Пример использования:

```
StringBuilder s1=new StringBuilder("Cogito ");
StringBuilder s2=new StringBuilder("ergo ");
s1.Append(s2);
s1.Append("sum");
System.Console.WriteLine(s1); //Напечатается "Cogito ergo sum"
```

Метод `Equals` служит для сравнения двух строк. Возвращает `true`, если строки равны и `false` в противном случае. Пример использования:

```
if(s1.Equals(s2))
    System.Console.WriteLine("Строки равны");
else
    System.Console.WriteLine("Строки не равны");
```

Метод `Insert` вставляет символы в заданную позицию (нумерация с нуля). Первый параметр задает позицию вставки, второй вставляемую строку. Например:

```
StringBuilder s1=new StringBuilder("abcde");
s1.Insert(2, "xyz");
System.Console.WriteLine(s1); //будет выведено "abxyzcde"
```

Метод Remove удаляет символы из строки. Первый параметр метода задает номер позиции, второй количество удаляемых символов. Например:

```
StringBuilder s1=new StringBuilder("abcde");
s1.Remove(1, 2);
System.Console.WriteLine(s1); // будет выведено "ade"
```

Метод Replace позволяет заменять подстроку в строке. Первый параметр метода задает подстроку, которая будет заменена везде, где встречается, второй – задает строку замены. Например:

```
StringBuilder s=new StringBuilder("abcdeabcde");
s.Replace("abc", "ZZZ");
System.Console.WriteLine(s); //Напечатается "ZZZdeZZZde"
```

Для примера разберем следующую задачу. Напишите программу-телеграф, которая принимает от пользователя сообщение и выводит его на экран в виде последовательности точек и тире. Вывод точек и тире можно сопроводить звуковым сигналом соответствующей длительности. Не различать строчные и прописные буквы. Азбука Морзе для букв русского алфавита приведена ниже:

А "-."	Б "-..."	В ".---"	Г "--."
Д "-.."	Е "."	Ж "...-"	З "-.."
И ".."	Й ".---"	К "-.-"	Л "-.."
М "--"	Н "-."	О "---"	П "-.-"
Р ".-."	С "..."	Т "-"	У "-."
Ф "-.-"	Х "...."	Ц "-.-."	Ч "----"
Ш "----"	Щ "--.-"	Ъ "-.-"	Ы "-.-"
Ь "-.-"	Э "-.-"	Ю "-.-"	Я "-.-"

Код программы приведен ниже:

```
// Азбука Морзе
string[] Morze = {"-.", "-...", ".---", "--.", "-..", ".",
                 "...-", "-..-", ".-.-", "-.-", "-.-.",
                 "-.-", "-.", "-.-.", "-.-.",
                 "-.-.", "-..-", "-.-.", "-.-.", "-.-.", "-.-.", "-.-.", "-.-.",
                 "-.-.", "-.-.", "-.-.", "-.-.", "-.-.", "-.-.", "-.-.", "-.-.",
                 "-.-.", "-.-.", "-.-.", "-.-."};

int[] Kol = new int[32];
// введенный пользователем текст
string str = textBox1.Text;
// в этой строке будем формировать код Морзе
StringBuilder str_Morze = new StringBuilder("");
int kod;
int dist = 0; // расстояние от буквы «а» до очередной буквы текста

// цикл по текстовому сообщению
for (int i = 0; i < str.Length; i++)
{
    kod = (int)str[i];
    if (kod < (int)'a') dist = kod - (int)'A';
```

```

        else dist = kod - (int)'a';
        str_Morze.Append(Morze[dist]); Kol[dist]++;
    }
    label2.Text = "Текст Морзе  " + str_Morze.ToString ();

```

Создается строковый массив с именем Morze, содержащий коды букв азбуки Морзе. Воспользуемся тем, что расстояние от буквы «а»(или «А») до любой другой буквы русского алфавита есть индекс, указывающий на соответствующую букву в массиве Morze. Далее все просто.

Контрольные вопросы

1. С помощью какого типа данных можно представить один символ данных?
2. Как описать символ данных? Как присвоить символу нужное значение?
3. Как задать строку данных? Какие классы для работы со строками Вы знаете? Кратко охарактеризовать каждый класс.
4. Перечислить основные методы класса String.
5. Перечислить основные методы класса StringBuilder.
6. Как строке присвоить значение?

Задания для самостоятельной работы

1. Введите строку русскими строчными буквами. Преобразуйте введенные строчные буквы в прописные и выведите строку.
2. Перевести введенный символ в код и наоборот. Создать удобный пользовательский Windows-интерфейс.
3. Дано арифметическое выражение в виде строки символов. Определить:
 - а. Сколько раз в этом выражении встречается каждый из символов '+', '-', '*', '/',
 - б. Сколько всего символов.
4. Дан русский текст, максимальная длина которого 255 символов.
 - а. Определить число слов в тексте, если слово это набор символов, ограниченный пробелами слева и справа.
 - б. В этом тексте заменить все вхождения слова «Маша» на слово «Саша».
 - с. Подсчитать сколько раз в тексте встречается каждая буква русского алфавита.
5. Зашифровать введенный текст, заменив каждый символ, стоящий через один символ от данного в таблице кодировки. Исходное разбиение на строки должно быть сохранено.
6. Ввести строку, состоящую из строчных русских букв. Преобразовать строку в прописные русские буквы.
7. Проверить, является ли введенная строка числом.
8. Вывести на экран 1-ю часть таблицы перекодировки (от 0 до 128).
9. По заданному многочлену и заданному значению переменной x вычислить значение многочлена. Многочлен задается в виде суммы одночленов. Между одночленами находится знак + или минус. Одночлен записывается так:

$$[\text{<коэффициент>} *] x [\text{^<степень>}]$$
 или

$$\text{<коэффициент>}$$
 Где <коэффициент> - натуральное число, не превышающее 100, x – символ переменной (всегда маленькая латинская буква x).
 <степень> - натуральное число, не превышающее 4. Параметры, взятые в квадратные скобки, могут быть опущены.

Примеры многочленов:

- $8*x + 5$
- $-2+x^1 - 3*x^2 +x^2 + 100*x^3 - 2*x$

Побитовые операторы And, Or, Xor, Not

Таблица истинности для побитовых операторов выглядит следующим образом (табл. 28):

Таблица 28

p	q	p&q	p q	p ^ q	~p
0	0	0	0	0	1
1	0	0	1	1	0
0	1	1	1	1	1
1	1	1	1	0	0

Оператор AND

Побитовый оператор AND (&) можно представить, как средство для сброса битов. То есть, если какой-либо бит в одном из операндов имеет значение 0, то соответствующий ему бит результата всегда будет иметь значение 0. Например:

11010011 &
10101010 =
10000010

Можно использовать оператор & для преобразования символов нижнего регистра в символы верхнего регистра с помощью переустановки шестого бита в 0 (сброс 6-го бита). Как определено в стандартном наборе символов Unicode/ASCII, значение символов нижнего регистра больше соответствующих символов верхнего регистра на число 32. Следовательно, чтобы преобразовать символы нижнего регистра в символы верхнего регистра, достаточно сбросить шестой бит, как это сделано в следующей программе, которая преобразует текст из нижнего регистра в верхний (рис. 22).

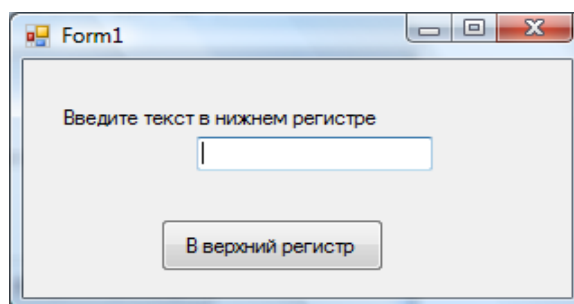


Рис. 22

Обработчик на кнопку «В верхний регистр» имеет следующий вид:

```
private void button1_Click(object sender, EventArgs e)
{
    string txt = textBox1.Text;
    string txt2 = "";
```

```

char ch;
for (int i = 0; i < txt.Length; i++)
{
    ch = txt[i];
    // 223 это 1101 1111, т.е отключен шестой бит
    ch = (char) (ch & 223);
    txt2 += ch;
}
textBox1.Text = txt2;
}

```

Число 223, используемое в операторе AND, имеет следующее двоичное представление: 1101 1111. Следовательно, операция AND оставляет без изменений все биты значения переменной ch, кроме шестого, который устанавливается в 0.

Операция используется также для того, чтобы определить установлен бит или нет. Например, в следующем примере проверяется, установлен ли 4-ый бит переменной status:

```

If (status & 8) Console.WriteLine("Четверный бит установлен! ");

```

В следующей программе такой подход используется для **вывода на экран двоичного представления числа типа byte** (рис. 23):

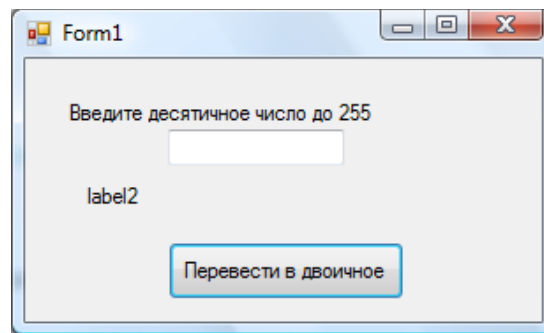


Рис. 23

Обработчик на кнопку «Перевести в двоичное»:

```

private void button1_Click(object sender, EventArgs e)
{
    byte num;
    byte t;    string txt="";
    num = byte.Parse(textBox1.Text);
    for (t = 128; t > 0; t /= 2)
    {
        if ((num & t) != 0) txt += "1";
        else txt += "0";
    }
    label2.Text = txt;
}

```

Задание: как изменить эту программу, чтобы она переводила в двоичное любое число до 65000.

Оператор OR

В противоположность оператору AND оператор OR (|) может использоваться для установки бит. Если хотя бы в одном из операндов бит установлен, то соответствующий бит в значении переменной также будет установлен.

11010011 OR
10101010 =
11111011

Для преобразования символов верхнего регистра в символы нижнего регистра мы можем задействовать оператор OR, как показано ниже:

```
string txt = textBox2.Text;
string txt2 = "";
char ch;
for (int i = 0; i < txt.Length; i++)
{
    ch = txt[i];
    // 32 это 0010 0000, т.е включаем шестой бит

    ch = (char)(ch | 32);
    txt2 += ch;
}
textBox2.Text = txt2;
```

Оператор XOR

При выполнении операции XOR бит устанавливается только в том случае, если сравниваемые биты различны. Оператор XOR имеет очень важное свойство, которое **позволяет использовать его для шифрования сообщений**. Если этот оператор применить к значениям X и Y, а затем к полученному результату и значению Y, то после выполнения второй операции будет получено значение X. То есть в последовательности операторов

$$R1 = X \oplus Y; \quad R2 = R1 \oplus Y;$$

Элемент R2 имеет то же значение, что и элемент X. Таким образом, результатом выполнения последовательности двух операций XOR с использованием одного значения будет первый операнд. Этот принцип можно использовать при создании простой программы для шифрования сообщений, в которой некоторое целочисленное сообщение будет являться ключом, используемым как для шифровки, так и для дешифровки сообщений с помощью оператора XOR.

Для шифровки сообщения оператор XOR применяется первый раз, в результате мы получаем зашифрованный текст. Для расшифровки оператор XOR применяется во второй раз, в результате мы получаем первоначальный текст. Например, разработаем следующую форму (рис. 24):

Рис. 24

```

public partial class Form1 : Form
{
    int key;
    public Form1()
    {
        InitializeComponent();
    }

    private void button1_Click(object sender, EventArgs e)
    {
        // шифрование сообщения
        string msg=textBox1.Text;
        string enc = "";
        key = Int16.Parse(textBox2.Text);
        for (int i = 0; i < msg.Length; i++)
        { enc+= (char) (msg[i]^key);
        }
        label2.Text=enc;
    }

    private void button2_Click(object sender, EventArgs e)
    {
        // расшифровать сообщение
        string decrypt="";
        string txt = label2.Text;

        for (int i = 0; i < txt.Length ; i++)
        { decrypt+=(char) (txt[i]^key);
        }
        label2.Text = decrypt;
    }
}

```

Подробнее об элементах управления форм

Рассмотрим еще несколько интересных элементов управления форм, так называемых *controls*. Элементы управления форм позволяют задавать режимы работы программы, отображать результаты вычислений, организовать диалог с пользователем. Начнем с самых простых элементов.

Флажок - элемент `CheckBox`

`Checkbox` это независимый переключатель или флажок, который может находиться только в двух состояниях «включено» и «выключено». Флажок находится на вкладке **Стандарные элементы управления**. Основные свойства приведены в табл. 29.

Таблица 29

Свойство	Описание
<code>Checked</code>	Если = true, то флажок включен
<code>Enabled</code>	Если = false, то флажок недоступен
<code>Text</code>	Подпись флажка

Радиокнопка - `RadioButton`

Радиокнопка это зависимый переключатель, это означает, что из нескольких радиокнопок во включенном состоянии может находиться только одна кнопка. Радиокнопка находится на вкладке **Стандарные элементы управления**. Радиокнопки, помещенные в контейнер работают, как группа зависимых переключателей, т.е. во включенном состоянии может находиться только одна кнопка. Контейнером для радиокнопок может служить форма, элементы GroupBox и Panel. Основные свойства приведены в табл. 30.

Таблица 30

Свойство	Описание
Checked	Если = true , то включен
Enabled	Если = false, то недоступен
Text	Подпись элемента

Рассмотрим пример. Даны 5 радио кнопок, как показано на рис. 25. С помощью одного обработчика события Click, общего для всех кнопок, определить текст выбранной кнопки.

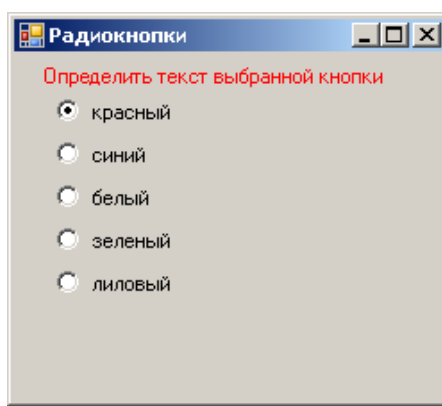


Рис. 25

Решение. Необходимо создать обработчик на событие Click любой из кнопок. Программный код обработчика будет иметь следующий вид:

```
private void radioButton1_Click(object sender, EventArgs e)
{
    MessageBox.Show((sender as RadioButton).Text);
}
```

Обработчику передается параметр `sender`, который указывает на объект источник события. Оператор `as` позволяет рассматривать данный объект, как объект типа `RadioButton`. Используя его свойство `Text`, мы и определяем текст выбранной кнопки.

Элемент GroupBox

Группирующий элемент, позволяет объединять элементы управления в группы. Находится на вкладке **Контейнеры** панели элементов. Для других элементов управления является контейнером. Основные свойства приведены в табл. 31.

Таблица 31

Свойства	Описание
Text	Подпись группы
Locked	Если = true, то элемент блокируется
Enabled	Если = false, то элемент недоступен

Элемент Panel

Это также группирующий элемент. Находится на вкладке **Контейнеры** панели элементов. Поддерживает полосы прокрутки, поэтому может использоваться для экономии места на пространстве формы. Например, если элементы, которые мы планируем разместить на форме там не помещаются, мы можем поместить их внутрь Panel и установить значение Autoscroll элемента Panel равным true. В результате пользователь получит возможность доступа к «неумещающимся» элементам управления с помощью полос прокрутки.

Таблица 32

Свойства	Описание
Autoscroll	Если = true, то задается полоса скроллинга
Locked	Если = true, то элемент блокируется
Enabled	Если = false, то элемент недоступен
BorderStyle	Задаёт стиль границы

Элемент NumericUpDown(вертушка)

Находится на вкладке **Стандарные элементы управления**. Основные свойства приведены в табл. 33.

Таблица 33

Свойства	Описание
Minimum	Минимальное значение
Maximum	Максимальное значение
Value	Текущее значение

Например, данный код позволяет получить текущее значение «вертушки»:

```
MessageBox.Show(numericUpDown1.Value.ToString());
```

Элемент MaskedTextBox

Задаёт поле редактирования с маской. Находится на вкладке **Стандарные элементы управления**. Маска позволяет ограничить ввод только заданными символами. Основные свойства приведены в табл. 34.

Таблица 34

Свойства	Описание
Mask	Маска
Text	В этом свойстве содержится выбор пользователя

Маска это строка, которая задаёт маскирующие элементы. Например, маска **000.00** – задаёт **шаблон** для ввода числа, который говорит о том, что число содержит 3 цифры целой части, далее следует разделитель целой и дробной части, затем две цифры дробной части.

Некоторые маскирующие элементы приведены в табл. 35:

Таблица 35

Маскирующий элемент	Описание
0	Обозначает цифру от 0 до 9, обязательный элемент
9	Обозначает цифру или пробел, необязательный элемент
#	Обозначает цифру или пробел, необязательный элемент.
L	Обозначает букву, обязательный элемент. Ограничивает ввод буквами a-

	z and A-Z. Эта маска эквивалентна регулярному выражению вида: [a-zA-Z].
?	То же, что и L, только необязателен. Эта маска эквивалентна регулярному выражению вида: [a-zA-Z]?.
&	Обозначает обязательный символ.
C	Обозначает необязательный символ.
A	Обозначает необязательную цифру или букву.
.	Задаёт точку в десятичном числе.
,	Разделитель тысяч
:	Разделитель, используется при задании времени
/	Разделитель даты
\$	Используется для задания денежных единиц

В табл. 36 показаны некоторые примеры масок.

Таблица 36

Маска	Описание
00/00/0000	Дата в формате день, месяц, год
(999)-000-0000	Для задания номера телефона

Элемент `CheckedListBox`

Список с флажками, относится к семейству списков. Находится на вкладке **Стандарные элементы управления**. Позволяет помещать обычные флажки внутри поля с полосами прокрутки. Рассмотрим основные свойства и методы этого элемента.

Свойство `Items` - это объект, который позволяет получить ссылку на набор элементов списка. Используя это свойство можно добавлять, удалять элементы, а также получать количество элементов. Объект имеет методы: `Add`, `Insert`, `Remove`, `Count`.

Например, следующий код позволяет добавить в список еще один элемент:

```
checkedListBox1.Items.Add("6");
```

Свойство `CheckedItems` - содержит коллекцию выбранных элементов списка. Используя цикл *foreach* можно пройти по всем членам коллекции. Следующий пример демонстрирует использование свойства *CheckedItems*. В результате выполнения этого кода получим значения выбранных элементов списка:

```
string str="";
foreach (object itemChecked in checkedListBox1.CheckedItems)
{
    str += " " + itemChecked.ToString();
}
MessageBox.Show("выбранные элементы " + str);
```

В другом примере определяется количество выделенных элементов списка:

```
int k = checkedListBox1.CheckedItems.Count;
MessageBox.Show("k=" + k);
```

Свойство `GetItemChecked` - возвращает *true*, если элемент с заданным индексом выбран, и *false* в противном случае. Например:

```
k=0;
string str="";
```

```

for (int i = 0; i < checkedListBox1.Items.Count; i++)
{
    if (checkedListBox1.GetItemChecked(i))
        str += " " + checkedListBox1.Items[i].ToString();
}

MessageBox.Show("k=" + k+ "str="+str);

```

На рис. 26 приведен внешний вид приложения, реализующий рассмотренные выше примеры.

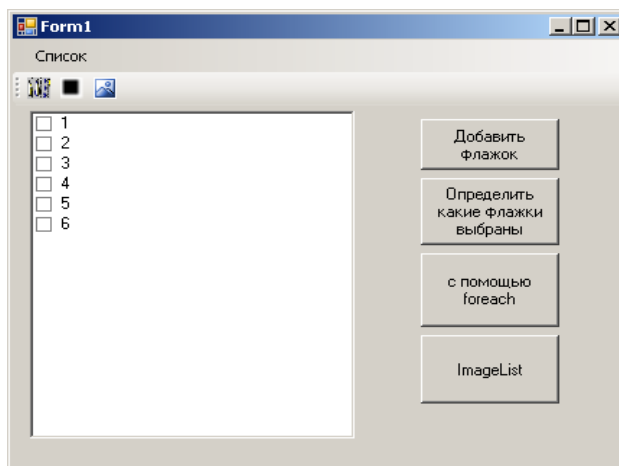


Рис. 26

Добавление элемента управления программно

Элемент управления может быть добавлен программным путем. Следующий код демонстрирует создание элемента `TextBox` программно:

```

private void button1_Click(object sender, System.EventArgs e)
{
    // создание объекта класса TextBox
    TextBox myText = new TextBox();
    // свойство Location задает позицию левого верхнего угла элемента в
    пикселах
    myText.Location = new Point(25,25);
    // добавление элемента управления в коллекцию элементов формы
    this.Controls.Add (myText);
}

```

Выравнивание элементов управления

Для выравнивания нескольких элементов управления необходимо:

1. Выделить их, очертив мышкой, либо выделить первый элемент, затем, удерживая клавишу **CTRL** выделить остальные.
2. Далее через меню **Формат** выровнять элементы по горизонтали, по вертикали, сделать одного размера.

Главное меню. Элемент `MenuStrip`

Элемент `MenuStrip` находится на вкладке «**Меню и панели инструментов**». Этот элемент позволяет создавать главное меню приложения с выпадающими подменю. Для создания главного меню приложения используется встроенный построитель меню (рис. 27).

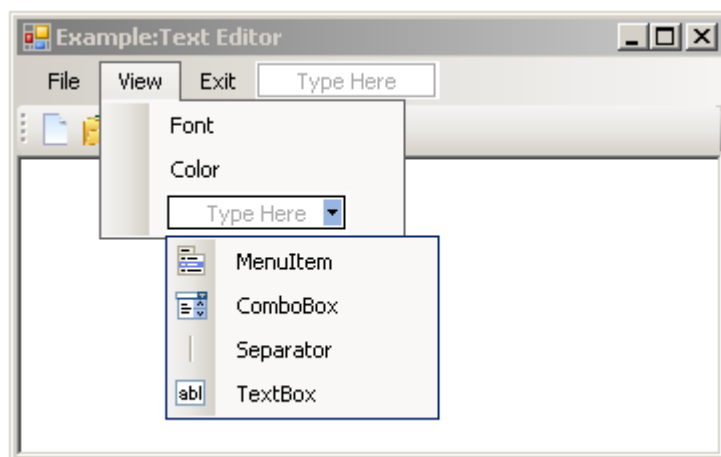


Рис. 27

В качестве элемента меню может выступать текстовый элемент (MenuItem), список (ComboBox), поле редактирования (TextBox) или разделитель (Separator).

На событие Click элемента меню пишется программный код, который будет выполняться при нажатии на пункт меню.

Элемент ToolStrip

Элемент ToolStrip находится на вкладке **«Меню и панели инструментов»** и позволяет создавать панель, состоящую из набора кнопок. Как правило, кнопочная панель располагается непосредственно под линейкой меню. Но с помощью свойства Dock положение кнопочной панели может быть изменено. На рис. 28 изображены возможные варианты расположения панели:

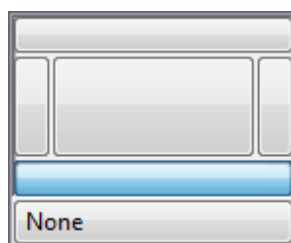


Рис. 28

Элемент ToolStrip состоит из кнопок, объектов типа ToolStripButton. Свойства кнопок приведены в табл. 37:

Таблица 37

Свойство	Описание
Image	содержит картинку на кнопке
ToolTipText	всплывающая подсказка, которой снабжается кнопка

Основное событие нажатие кнопки - Click.

Элемент DateTimePicker

Элемент управления DateTimePicker позволяет пользователю выбрать отдельный элемент в списке дат или времени. Находится на вкладке **Стандарные элементы**

управления. Когда компонент используется для представления даты, он состоит из двух частей: раскрывающегося списка с датой, представленной в виде текста, и сетки, которая появляется при нажатии кнопки со стрелкой вниз, расположенной рядом со списком. Внешний вид элемента приведен на рис. 29.

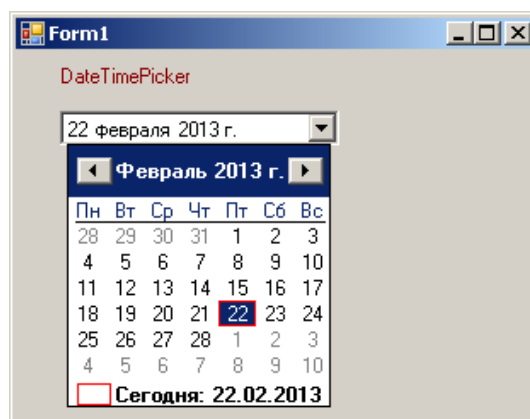


Рис. 29

У этого элемента множество интересных свойств, которые позволяют настроить внешний вид календаря на любой цвет и вкус. Но самые интересные **свойства** — это функциональные, которые приведены в табл. 38:

Таблица 38

Свойство	Описание
Format	формат отображения значения. Здесь можно указать одно из следующих значений: Long— длинный формат, в котором месяц указывается названием, а не числом; Short— короткий формат даты; Time— отображать время для редактирования; Custom— настраиваемый формат.
MaxDate	значение максимальной даты, за пределы которой нельзя выходить;
MinDate	значение минимальной даты, за пределы которой нельзя выходить;
CustomFormat	пользовательский формат даты
ShowUpDown	показывать в компоненте справа две кнопки со стрелками вверх и вниз, с помощью которых можно увеличивать и уменьшать текущую часть времени.
Value	Текущее значение DateTimePicker, значение типа DateTime

Чтобы получить выбранную пользователем дату, можно использовать следующий код:

```
DateTime dt= dateTimePicker1.Value;
```

В переменную класса DateTime сохраняется выбранное пользователем значение. Время в .NET представляется классом DateTime. Это не просто переменная, а класс с большим количеством свойств и методов.

Самым интересным событием компонента DateTimePicker можно назвать событие **ValueChanged**, которое генерируется каждый раз, когда вы изменяете дату или время внутри компонента. Но пользователь далеко не всегда может выбрать новое значение даты за один раз. Он может выбрать значение, подумать, а потом снова изменить его. Если вы хотите после изменения даты или времени тут же перерисовывать что-то, то это событие будет как раз кстати.

Если вам не нужно отслеживать изменения, которые делает пользователь, а вы хотите получить только окончательный выбор, то в этом случае может быть выгоднее обрабатывать событие **Leave**, которое генерируется, когда компонент теряет фокус ввода, т. е. когда пользователь переключается на работу с другим элементом управления

Следующий пример демонстрирует использование элемента DateTimePicker (рис. 30):

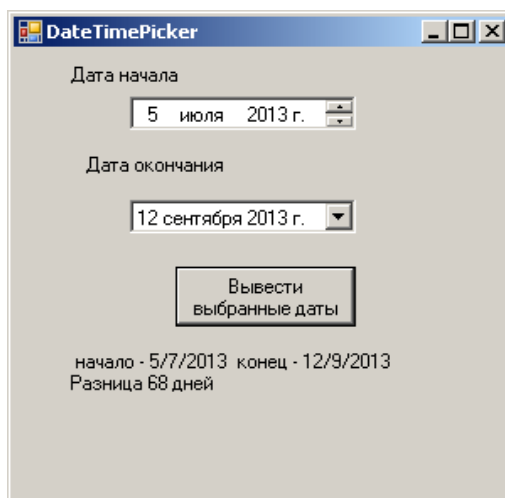


Рис. 30

В элементе label3 будет выведено выбранное пользователем значение даты. Разница в днях между датами высчитывается следующим образом:

```
DateTime startD = dateTimePicker1.Value;
DateTime finishD = dateTimePicker2.Value;
string start1 = startD.Day + "/" + startD.Month + "/" + startD.Year;
string start2 = finishD.Day + "/" + finishD.Month + "/" +
                finishD.Year;

TimeSpan ts = finishD - startD;

int DifDays = ts.Days;
label3.Text = " начало - " + start1 + " конец - " + start2 + "
              Разница " + DifDays.ToString() + " дней";
```

Элемент ListView

Элемент **ListView** позволяет отображать список элементов текста с рисунками(иконками) для идентификации типа элемента. Например, список файлов Windows Explorer'a аналогично отображает папки и файлы в виде дерева. Каждое изображение файла и папки сопровождается иконкой(картинкой) для определения типа файла или папки. Элементы списка могут отображаться одним из 5-ти различных способов: **Details**, **LargeIcon**, **List**, **SmallIcon**, **Tile**. Элементы могут отображаться как большие иконки, как маленькие иконки или как маленькие иконки в вертикальном списке.

Элементы могут иметь подписки(subitems) элементов, которые относятся к родительскому элементу. Представление **Details** позволяет отображать элементы и их подписки в виде сетки с заголовками столбцов, которая определяет тип информации подписка. Режим просмотра задается в свойстве **View**. В табл. 39 приведены значения свойства **View**.

Таблица 39

Значения	Описание
----------	----------

Details	Каждый элемент выводится в отдельной строке с последующей информацией о каждом элементе этой строки. Содержимое элементов данных ListView подразделяется в столбцы. Пользователь может изменять размер столбцов во время выполнения.
LargeIcon	Каждый элемент выглядит как большая иконка с подписью.
List	Каждый элемент выглядит как небольшая иконка с подписью справа. Элементы упорядочены в столбцы без заголовков.
SmallIcon	Каждый элемент выглядит как маленькая иконка с подписью справа.
Tile	Позволяет отображать элементы и их подписи как мозаику, которая содержит большие иконки, следующие за текстовой информацией.

На рис. 31 приведен пример внешнего вида списка в режиме просмотра **Details**. Здесь данные подразделяются на столбцы. Размеры столбцов можно изменять, можно задавать заголовки столбцов.

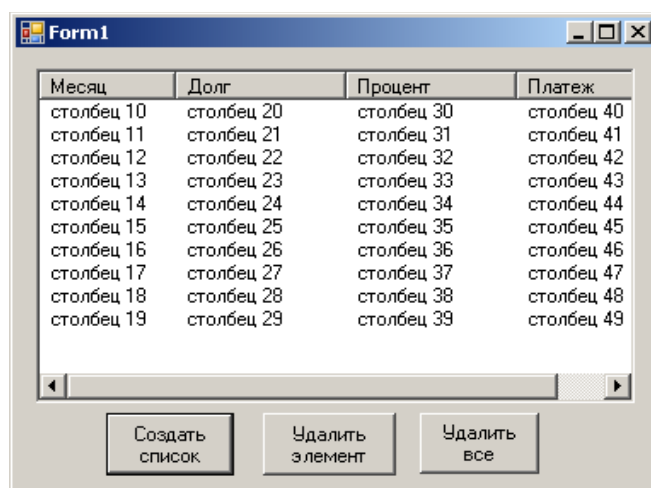


Рис. 31

На рис. 32 приведен пример режима **LargeIcon**:

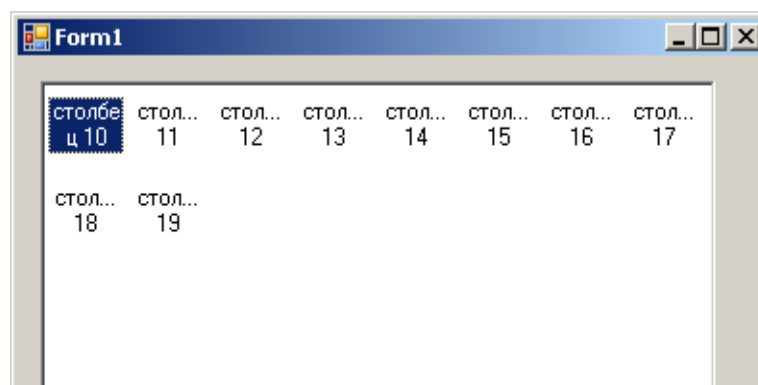


Рис. 32

На рис. 33 приведен пример внешнего вида в режиме **List**:

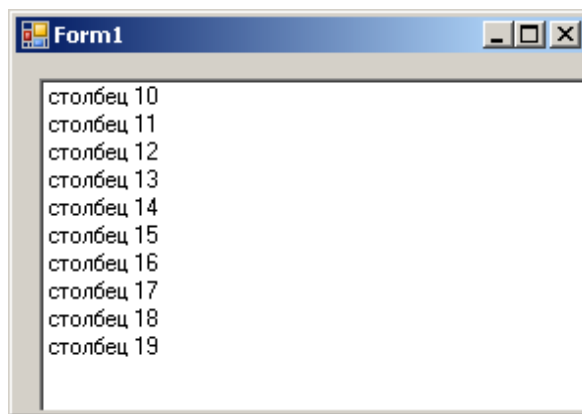


Рис. 33

На рис. 34 ниже приведен пример вида в режиме **SmallIcon**:

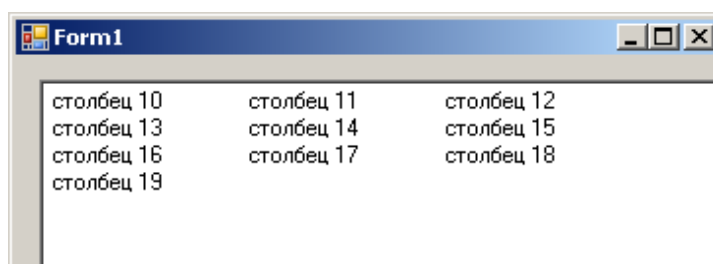


Рис. 34

Из списка **ListView** можно выбрать один или несколько элементов. Множественное выделение делается так же, как в списке **ListBox**. Дополнительно пользователь может активизировать выбранный элемент для выполнения задания. Например, можно использовать **ListView** для отображения файлов, которые приложение может открыть или использовать.

ListView может также отображать флажки (элемент **Checkbox**), для этого используется свойство **Checkboxes** списка.

Элемент **ListView** можно использовать для отображения информации из приложения, из базы данных, или текстового файла. **ListView** можно использовать для получения информации от пользователя, например выбрать список файлов. Список **ListView** предоставляет большое количество свойств, которые обеспечивают гибкость приложения и его поведение. Рассмотрим некоторые свойства элемента **ListView**.

Свойство **View** – задает тип представления списка **ListView**.

Свойства **LargeImageList**, **SmallImageList**, и **StateImageList** позволяют определить объекты **ImageList**, которые содержат изображения элементов, а в случае **StateImageList** это будут флажки (**Checkbox**'ы), которые отображаются, когда свойство **CheckBoxes** равно **true**.

Свойство **Columns** предоставляет доступ к коллекции **ListView.ColumnHeaderCollection**, в которой сохраняются заголовки, когда используется свойство **View** равное **Details**.

Элементы добавляются и удаляются из списка через свойство **Items**. Свойство **Items** обеспечивает доступ к коллекции **ListView.ListViewItemCollection**, которая предоставляет методы для манипулирования элементами списка.

Если необходимо предоставить пользователю возможность редактировать поля списка нужно использовать свойство **LabelEdit**.

Если список содержит большое количество элементов, то лучше видеть их в отсортированном виде. Для этого используется свойство **Sorting** (сортировка в алфавитном порядке).

Многие свойства списка **ListView** используются для типа **Details**. Свойство **AllowColumnReorder** позволяет менять порядок столбцов во время выполнения.

Свойство **FullRowSelect** – позволяет выбирать элементы и подэлементы списка одновременно.

Для вывода линий сетки в типе **Details** используется свойство **GridLines**.

Свойство **HeaderStyle** – задает типы заголовков списка.

Кроме свойств **ListView** имеет методы и события. Методы **BeginUpdate** и **EndUpdate** позволяют улучшить выполнение, когда добавляется много элементов в список, не позволяя перерисовывать список.

Для обеспечения функциональности при нажатии правой кнопки мыши используется метод **GetItemAt**.

Метод **EnsureVisible** может быть вызван для проверки того, что нужный элемент является видимым в списке.

Если свойство **LabelEdit** равно **true**, можно использовать события **BeforeLabelEdit** и **AfterLabelEdit** для проверки того, что было введено до и после.

В табл. 40 приведены некоторые члены класса **ListView** и показано в каких режимах представления они работают:

Таблица 40

Член класса ListView	Режим
Alignment (свойство)	SmallIcon или LargeIcon
AutoArrange (свойство)	SmallIcon или LargeIcon
AutoResizeColumn (метод)	Details
Columns (свойство)	Details или Tile
DrawSubItem (событие)	Details
FindItemWithText (метод)	Details , List , или Tile
FindNearestItem (метод)	SmallIcon или LargeIcon
GetItemAt (метод)	Details или Tile
Groups (свойство)	Все кроме List
HeaderStyle (свойство)	Details
InsertionMark (свойство)	LargeIcon , SmallIcon , или Tile

Рассмотрим примеры использования элемента **ListView**.

Пример 1.

Следующий код демонстрирует создание списка режима **Details**, с заголовками 4-х столбцов шириной 80, 100, 100, 100 пикселей, выравненных по левой границе (рис. 35).

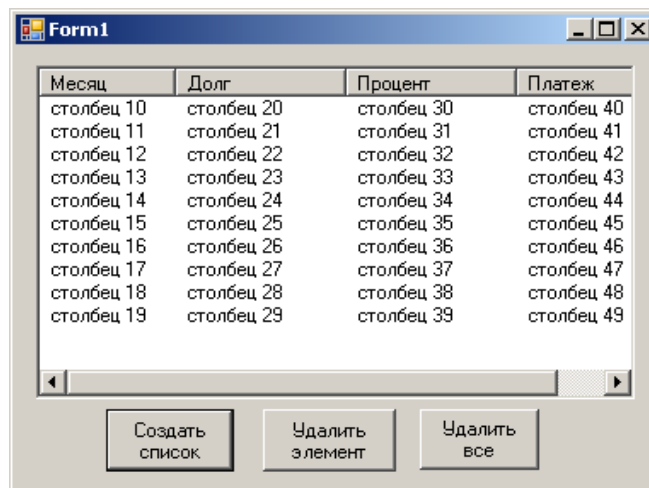


Рис. 35

Свойство **Column** объекта **ListView** имеет метод **Add**, который позволяет добавлять столбцы в список.

```
// Задание стиля списка типа Details.
listView1.View = View.Details;

listView1.Columns.Add("Месяц", 80, HorizontalAlignment.Left);
listView1.Columns.Add("Долг", 100, HorizontalAlignment.Left);
listView1.Columns.Add("Процент", 100, HorizontalAlignment.Left);
listView1.Columns.Add("Платеж", 100, HorizontalAlignment.Left);

// Добавление строк
for (int i = 0; i < 10; i++)
{
    listView1.Items.Add("столбец 1"+i );
    listView1.Items[i].SubItems.Add("столбец 2" + i);
    listView1.Items[i].SubItems.Add("столбец 3" + i);
    listView1.Items[i].SubItems.Add("столбец 4" + i);
}
```

Для удаления строк используется метод **RemoveAt** объекта **Items**, например:

```
listView1.Items.RemoveAt(1);
```

Для удаления всех элементов списка используется метод **Clear** объекта **Items**, например:

```
listView1.Items.Clear();
```

Пример 2.

В следующем примере создается элемент управления **ListView** с тремя объектами **ListViewItem**, для каждого из которых определяется подсписок, состоящий из трех элементов. Заголовки столбцов создаются с помощью объекта **ColumnHeader**. В примере используются следующие свойства объекта **ListView**:

- View
- LabelEdit
- AllowColumnReorder
- CheckBoxes
- FullRowSelect
- GridLines

- **Sorting**

Программный код приведен ниже:

```
private void CreateMyListView()
{
    // Создание нового элемента ListView
    ListView listView1 = new ListView();
    listView1.Bounds = new Rectangle(new Point(10,10), new
Size(300,200));

    // Задание стиля списка типа Details.
    listView1.View = View.Details;
    // Позволяет редактировать текст элементов
    listView1.LabelEdit = true;
    // Разрешает менять порядок следования столбцов
    listView1.AllowColumnReorder = true;
    // Выводит CheckBox'ы
    listView1.CheckBoxes = true;
    // Выбирать элементы и элементы подсписка
    listView1.FullRowSelect = true;
    // Вывод линий сетки
    listView1.GridLines = true;
    // Сортировка элементов в порядке возрастания
    listView1.Sorting = SortOrder.Ascending;

    // Создание элементов и трех наборов списков для каждого элемента
    ListViewItem item1 = new ListViewItem("item1",0);
    // Разместить элементы подсписков флажков
    item1.Checked = true;
    item1.SubItems.Add("1");
    item1.SubItems.Add("2");
    item1.SubItems.Add("3");
    ListViewItem item2 = new ListViewItem("item2",1);
    item2.SubItems.Add("4");
    item2.SubItems.Add("5");
    item2.SubItems.Add("6");
    ListViewItem item3 = new ListViewItem("item3",0);
    // размещение флажков около основных элементов
    item3.Checked = true;
    item3.SubItems.Add("7");
    item3.SubItems.Add("8");
    item3.SubItems.Add("9");

    // Создание заголовков для основных элементов и вторичных элементов
    listView1.Columns.Add("Item Column", -2, HorizontalAlignment.Left);
    listView1.Columns.Add("Column 2", -2, HorizontalAlignment.Left);
    listView1.Columns.Add("Column 3", -2, HorizontalAlignment.Left);
    listView1.Columns.Add("Column 4", -2, HorizontalAlignment.Center);

    //Добавление элементов в список
    listView1.Items.AddRange(new
ListViewItem[]{item1,item2,item3});

    // Создание двух объектов ImageList
    ImageList imageListSmall = new ImageList();
    ImageList imageListLarge = new ImageList();

    // Добавление элемента ListView в коллекцию элементов управления
    this.Controls.Add(listView1);
}
```

Результат выполнения данного кода приведен на рис. 36.

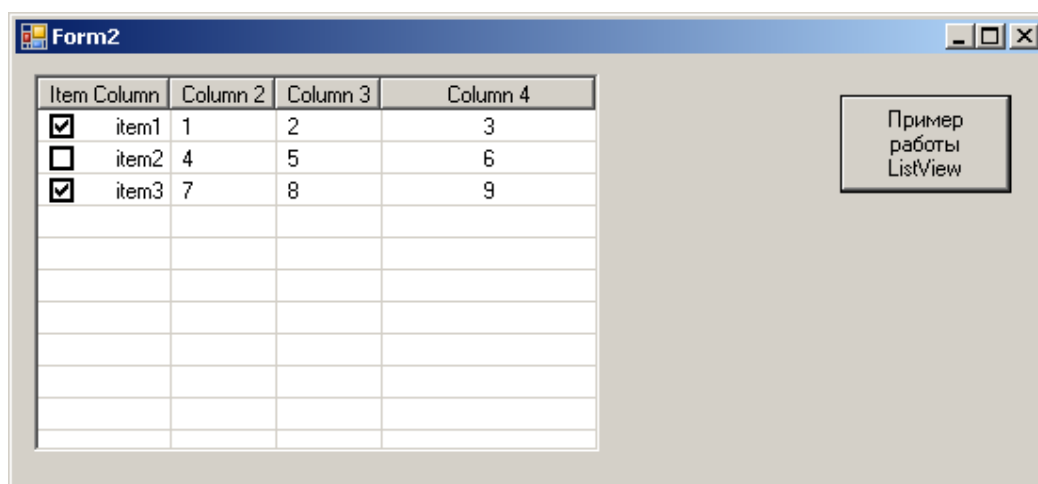


Рис. 36

Работа со стандартными диалоговыми окнами Windows

Стандартные диалоговые окна Windows располагаются в панели инструментов на вкладке «**Диалоговые окна**». Диалоговые окна позволяют использовать стандартные диалоги Windows в своем приложении. Предлагаются следующие диалоги:

- ColorDialog – диалог выбора цвета;
- FolderBrowserDialog – диалог выбора и просмотра папок;
- FontDialog – диалог выбора шрифта;
- OpenFileDialog – диалог открытия файла;
- SaveFileDialog – диалог сохранения файла.

Диалог открытия файла, класс OpenFileDialog

Элемент OpenFileDialog находится на вкладке «**Диалоговые окна**» панели инструментов и позволяет в своем приложении использовать стандартный Windows-диалог выбора файла. Основные свойства диалога приведены в табл. 41.

Таблица 41

Свойство	Описание
Title	Заголовок окна
InitialDirectory	Начальная папка, которая появляется в момент открытия окна
Filter	Будут отображаться только те файлы, которые указаны в этом свойстве.

Основной метод ShowDialog открывает окно диалога выбора файла.

Рассмотрим использование элемента на примере. Необходимо загрузить текстовый файл (*.txt или *.rtf) в элемент richTextBox, используя стандартный диалог выбора файла. Для этого поместим элемент richTextBox1 в область формы. Поместим элемент OpenFileDialog в область формы, как показано на рис. 37.

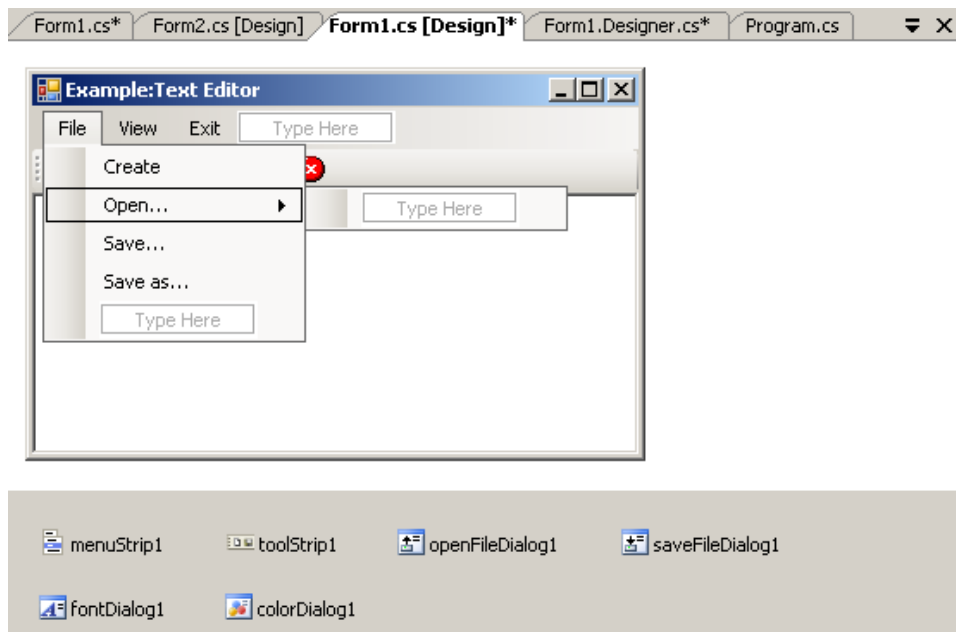


Рис. 37

Программный код открытия файла имеет следующий вид:

```
// открыть диалог выбора файла
if (openFileDialog1.ShowDialog() == DialogResult.OK)
{
    richTextBox1.LoadFile(openFileDialog1.FileName);
    FileN = openFileDialog1.FileName;
}
```

Если не использовать элемент OpenFileDialog панели инструментов, то программный код может иметь следующий вид:

```
// создание объекта
OpenFileDialog op = new OpenFileDialog();
op.Title = "заголовок окна";
// задание начальной папки
op.InitialDirectory = "d:/ita";
// фильтр на файлы
op.Filter = "Текстовые
файлы (*.txt) | *.txt | rich (*.rtf) | *.rtf | doc (*.doc) | *.doc";
// выполнить открытие диалога
if (op.ShowDialog() == DialogResult.OK)
{
    // обработка исключения
    try
    {
        richTextBox1.LoadFile(op.FileName);
    }
    catch
    {
        MessageBox.Show ("ошибка");
    }
}
```

В следующем примере с помощью диалога открытия файла выбирается рисунок и загружается в элемент управления типа PictureBox:

```
// Открыть диалог выбора файла
OpenFileDialog dlg = new OpenFileDialog();
dlg.Title="выбор картинки";
// Если пользователь выбрал файл
```

```

if (dlg.ShowDialog() == DialogResult.OK)
// То загрузить выбранный файл в элемент pictureBox1
    pictureBox1.Image= new Bitmap(dlg.OpenFile());
    //метод Dispose - освобождает ресурсы, занятые объектом dlg
dlg.Dispose();

```

Диалог выбора шрифта - класс FontDialog

Элемент открытия выбора шрифта FontDialog находится на вкладке «Диалоговые окна» панели инструментов и позволяет использовать стандартный Windows-диалог выбора шрифта в своем приложении.

Основное свойство Font содержит выбранный шрифт.

Основной метод ShowDialog открывает окно диалога выбора шрифта.

Например, с помощью диалога выбора шрифта меняем шрифт элемента label1:

```

if (fontDialog1.ShowDialog() == DialogResult.OK)
    label1.Font = fontDialog1.Font;

```

Рассмотрим еще один пример:

1. Создадим Windows Forms-приложение.
2. Главной форме дадим имя(Name) –WinDialogs.
3. В классе WinDialogs введем следующие переменные^

```

public partial class WinDialogs : Form
{
    private Font currFont = new Font("Times New Roman", 12);
    private SolidBrush br = new SolidBrush(Color.Black);
}

```

4. На событие Paint формы напишем обработчик

```

private void WinDialogs_Paint(object sender, PaintEventArgs e)
{
    Graphics g = e.Graphics;
    g.DrawString("Тестирование шрифта... ", currFont,br, 30, 50);
}

```

5. По кнопке запустить код:

```

if (fontDialog1.ShowDialog()== DialogResult.OK)
{
    currFont = fontDialog1.Font;
    Invalidate();
}

```

6. Изменить шрифт выделенного текста

```

// шрифт
if (fontDialog1.ShowDialog() == DialogResult.OK)
    richTextBox1.SelectionFont = fontDialog1.Font;

```

Этот код открывает стандартный диалог выбора шрифта, если пользователь выбирает ОК, то текущий шрифт меняется на выбранный и форма, на которой написан текст, перерисовывается.

Аналогично работают все диалоговые окна Windows.

Примеры разработки несложных приложений

Рассмотрим несколько несложных примеров разработки Windows-приложений на C# с использованием форм и элементов управления.

Конвертер валют

Следующее несложное приложение взято из книги Культина Н. «С# в примерах и задачах» и интересно тем, как выполняется контроль ввода данных в поля редактирования.

Требуется создать Windows-форму следующего вида (рис. 38). По кнопке «Пересчет» выполняется пересчет введенного количества рублей в доллары по текущему курсу. По кнопке «Завершить» форма закрывается.

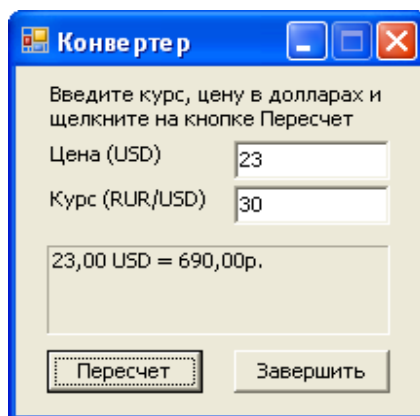


Рис. 38

Создание подобной простой формы не составляет труда. Цена и курс вводятся в поля *textBox1* и *textBox2* соответственно. В элемент *label4* выводится результат пересчета долларов в рубли по текущему курсу. По кнопке «Пересчет» выполняется пересчет. По кнопке «Завершить» приложение завершается.

Рассмотрим более подробно каким образом выполняется контроль ввода данных в поля редактирования. При нажатии любой клавиши выполняется событие *KeyPress* элемента *textBox*:

```
private void textBox1_KeyPress(object sender,
    System.Windows.Forms.KeyPressEventArgs e)
{
}
```

Событию *KeyPress* передается параметр *e*, это объект с набором методов. Разрешается вводить только цифры, запятую, как разделитель целой и дробной части, причем только в единственном экземпляре, использовать клавишу *back space* и стрелки вправо, влево. Программный код, выполняющий проверку того, что введено, имеет следующий вид:

```
if (!Char.IsDigit(e.KeyChar) &&
    !(Char.IsControl(e.KeyChar)))
{
    if (((e.KeyChar.ToString() == ",") &&
        (textBox1.Text.IndexOf(",") == -1)))
        e.Handled = true;
}
```

Рассмотрим этот код более подробно. Метод *KeyChar* объекта *e* возвращает код нажатой клавиши. Поэтому можно отследить какие клавиши нажимаются.

Использование класса *Char* позволяет определить, что нажато – цифра, буква, управляющая клавиши, или что-то еще. Метод *IsDigit* – возвращает истину, если нажата цифра. Метод *IsControl* возвращает истину, если нажата управляющая клавиша (к ним относятся *Enter*, *Esc*, *Back Space*). Метод *IsLetter* возвращает истину, если нажата буква. Метод *IndexOf* ищет подстроку, заданную в качестве аргумента в строке, в данном случае в *Text*, если не находит, то возвращает -1(минус 1).

Свойство *e.Handled* равное *true* означает, что нажатие отменяется.

Для решения этой задачи применяется следующая логика:

Если это не цифра и не управляющая клавиша, тогда

Если следующее утверждение не верно «это запятая и запятой еще нет в тексте» тогда отменить нажатие.

Ниже приведен программный код на событие Click кнопки «Пересчет»:

```
private void button1_Click(object sender, EventArgs e)
{
    double k; // курс доллара
    double usd; // цена в долларах
    double rub; // цена в рублях

    label4.Text = "";
    try
    {
        usd = Convert.ToDouble(textBox1.Text);
        k = Convert.ToDouble(textBox2.Text);
        rub = usd * k;

        label4.Text = usd.ToString("N") + " USD= " +
rub.ToString("C");
    }
    catch
    {
        if ((textBox1.Text == "") || (textBox2.Text == ""))
            MessageBox.Show("Ошибка в исходных данных. \n"
+ "Необходимо ввести данные в оба поля.",
"Конвертер",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
        else
            MessageBox.Show("Ошибка в исходных данных. \n" +
"Неверный формат данных в одном из полей.",
"Конвертер", MessageBoxButtons.OK,
            MessageBoxIcon.Error);
    }
}
```

Кнопочный калькулятор

Данное приложение также взято из книги Культина Н. «С# в примерах и задачах». Приложение демонстрирует создание компонентов программным путем. Кнопки калькулятора это элементы массива компонентов Button. Создание и настройка кнопок осуществляется в конструкторе формы, там же назначаются процедуры обработки события click. На все кнопки с цифрами назначается один обработчик события. Точно также на все кнопки с операциями назначается один обработчик события. Внешний вид формы приведен на рис. 39:

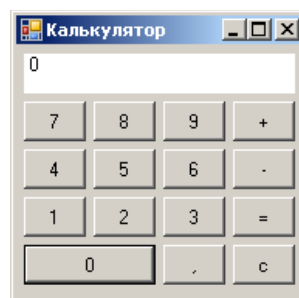


Рис. 39

Программный код имеет следующий вид:

```
namespace Calculator
{
    public partial class Form1 : Form
    {
        private const int
        bw = 40, bh=25, // ширина и высота кнопок
        bdx =5, bdy = 5; // расстояние между кнопками

        // массив цифровых кнопок и запятая
        private Button[] d = new Button[11];

        // массив операционных кнопок: +, -, =, C
        private Button[] op = new Button[4];

        private double ac; // аккумулятор
        private string co; // текущая операция
        private Boolean fd; // fd = true ждем первую цифру числа
                           // например, после того как была нажата кнопка +
                           // fd = false ждем ввода следующей цифры
                           // или нажатия кнопки операции

        public Form1()
        {
            InitializeComponent();
            int x,y; // координаты размещения кнопок

            // ширина и высота рабочей области формы
            this.ClientSize= new System.Drawing.Size(4*bw+5*bdx, 5*bh + 7*bdy);

            label1.SetBounds(bdx, bdy, 4 * bw + 3 * bdx, bh);
            label1.Text = "0";
            for (int i = 0; i <= 10; i++)
            {
                d[i] = new Button();
                // цифровые кнопки 0 - 9
                if (i < 10)
                {
                    d[i].Name = "Button" + Convert.ToString(i);
                    d[i].Text = i.ToString();
                    if (i != 0)
                    {
                        x = ((i - 1) % 3) * bw + ((i - 1) % 3 + 1) * bdx;
                        y = ((int)((9 - i) / 3) + 1) * bh + ((int)((9 - i) / 3) + 2) * bdy;
                        d[i].SetBounds(x, y, bw, bh );
                    }
                    else // кнопка 0
                        d[i].SetBounds(bdx, 4 * bh + 5 * bdy, 2 * bw + bdx, bh);
                }
                else
                {
                    // кнопка запятой
                    {
                        d[i].Name = "ButtonComma";
                        d[i].Text = ",";
                        d[i].SetBounds(2 * bw + 3 * bdx, 4 * bh + 5 * bdy, bw, bh);
                    }
                }

                // назначение процедуры обработки события нажатия кнопки
                this.d[i].Click += new System.EventHandler(this.ButtonN_Click);
                // добавление сформированной кнопки на форму
                this.Controls.Add(this.d[i]);
            }
        }
    }
}
```

```

    }
    // создание операционных кнопок
    for (int i = 0; i <= 3; i++)
    {
        op[i] = new Button();
        // кнопка +
        if (i == 0)
        {
            op[i].Name = "ButtonPlus";
            op[i].Text = "+";
        }
        // кнопка -
        if (i == 1)
        {
            op[i].Name = "ButtonMinus";
            op[i].Text = "-";
        }
        // кнопка =
        if (i == 2)
        {
            op[i].Name = "ButtonResult";
            op[i].Text = "=";
        }
        // кнопка с - очистить
        if (i == 3)
        {
            op[i].Name = "ButtonClear";
            op[i].Text = "c";
        }
    }
    op[i].SetBounds(3 * bw + 4 * bdx, (i + 1) * bh + (i + 2) * bdy, bw, bh);

    // назначение процедуры обработки события нажатия кнопки
    this.op[i].Click += new System.EventHandler(this.ButtonOp_Click);
    // добавление сформированной кнопки на форму
    this.Controls.Add(this.op[i]);
    } // конец цикла for

    // ждем первую цифру
    fd = true;
    co = "ButtonResult";
}
// нажатие цифровых кнопок или запятой
private void ButtonN_Click(object sender, System.EventArgs e)
{
    // нажатая кнопка
    Button btn_c = (Button)sender;
    if (btn_c.Name != "ButtonComma")
    {
        // цифры
        if (btn_c.Name != "Button0")
        {
            // кнопки 0..9
            if (fd)
            {
                label1.Text = btn_c.Text;
                fd = false;
            }
            else label1.Text += btn_c.Text;
        }
        else
        {
            // кнопка 0
            if (fd) label1.Text = btn_c.Text;
            if (label1.Text != "0")
                label1.Text += btn_c.Text;
        }
    }
}

```

```

    }
}
else
    // запятая
    if (fd)
    {
        labell1.Text = "0,";
        fd = false;
    }
    else
        // если запятой еще не было
        if (labell1.Text.IndexOf(",") == -1)
            labell1.Text += btn_c.Text;
}

private void ButtonOp_Click(object sender, System.EventArgs e)
{
    // нажатая кнопка
    Button btn_c = (Button)sender;
    double ind_n;
    if (btn_c.Name != "ButtonClear")
    {
        // кнопки с операциями
        ind_n = Convert.ToDouble(labell1.Text);
        // выполняем предыдущую операцию
        if (fd == false)
        {
            if (co.Equals("ButtonPlus")) ac += ind_n;
            if (co.Equals("ButtonMinus")) ac -= ind_n;
            if (co.Equals("ButtonResult")) ac = ind_n;
        }
        if (btn_c.Name == "ButtonPlus") co = "ButtonPlus";
        if (btn_c.Name == "ButtonMinus") co = "ButtonMinus";
        if (btn_c.Name == "ButtonResult") co = "ButtonResult";

        labell1.Text = ac.ToString();
    }
    else
    {
        // кнопка "очистить"
        ac = 0;
        labell1.Text = "0";
        co = "ButtonResult";
    }
    fd = true;
}
}
}

```

Задания для самостоятельной работы

Задача 1. Электроэнергия

Вводится предыдущее и текущее показания счетчика электроэнергии. Вводится цена за 1 киловатт. Посчитать, сумму к оплате. Выполнять контроль ввода данных. Кнопка «Вычислить» будет доступна, если во все поля введены данные, иначе кнопка «Вычислить» не доступна (рис. 40).

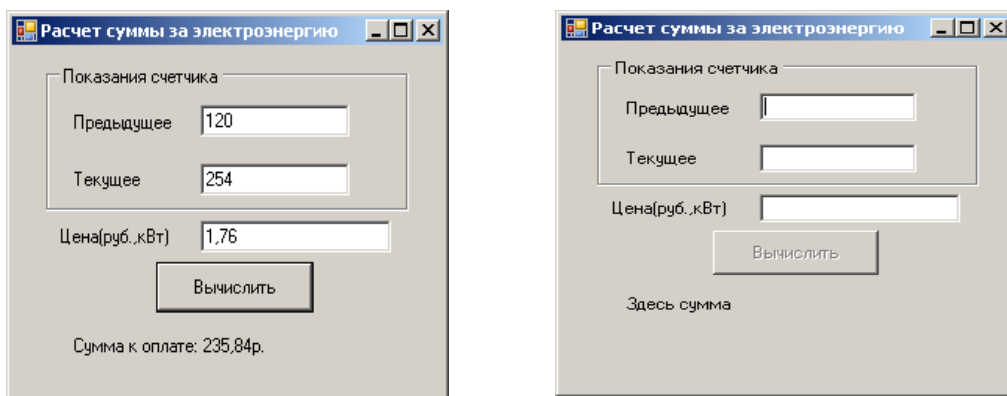


Рис. 40

Задача 2. Автозаправка

Определить сколько бензина разного качества можно купить на заданную сумму. Автозаправка продает бензин следующих марок:

92 по цене 17,12 руб.

95 по цене 19,45 руб.

98 по цене 20,25 руб.

ДТ по цене 17,00 руб.

41. Выполнять контроль ввода данных. Примерный внешний вид окна приведен на рис.

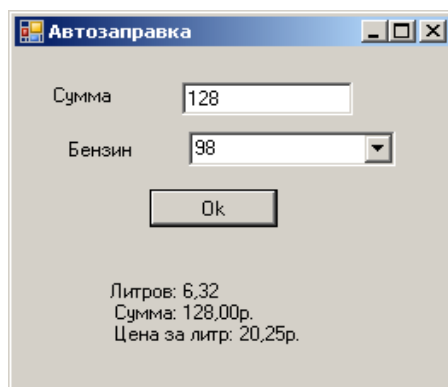


Рис. 41

Задача 3. Простой калькулятор на 4 операции (+, -, /, *)

Создать приложение «Простой калькулятор». Внешний вид окна приложение приведен на рис. 42. Выполнять контроль ввода данных.

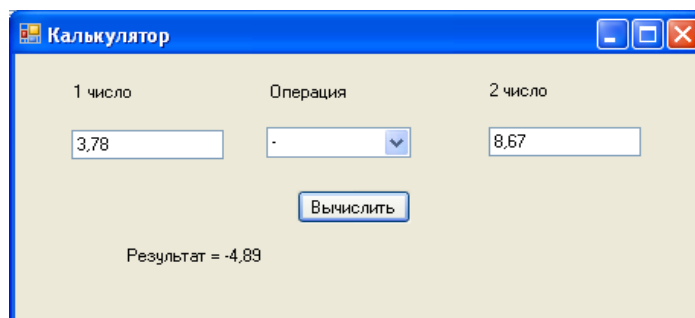


Рис. 42

Задача 4. Кафе

Требуется подсчитать стоимость заказа в кафе. Сэндвич стоит 54,00 руб. Картошка стоит 27,50 руб. Причем, если Вы выбираете картошку, то становится доступным флажок «Соус», иначе флажок «Соус» недоступен. Соус стоит 10,55 руб. Кока-кола стоит 18 руб. причем, если клиент заказал все предлагаемые продукты, то ему предоставляется скидка 10%. По кнопке «ОК» вычисляется сумма заказа. Выполнять контроль ввода данных. Внешний вид формы приведен на рис. 43.

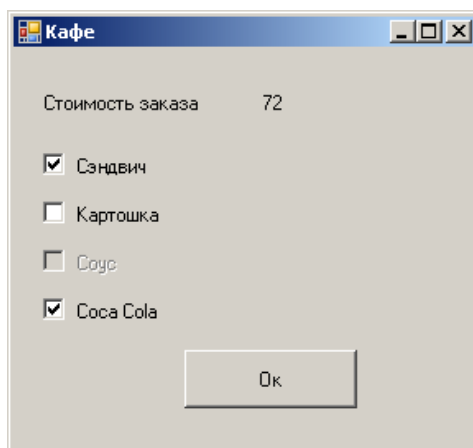


Рис. 43

Задача 5. Радиокнопки

Даны 5 радио кнопок, как показано на рис. 44. С помощью одного обработчика события Click, общего для всех кнопок определить текст выбранной кнопки.

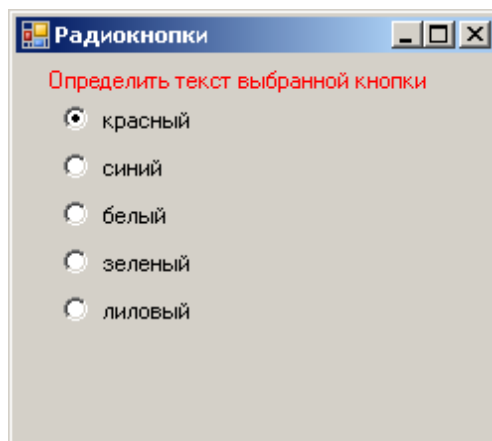


Рис. 44

Задача 7. Стеклопакет

Однокамерный стеклопакет стоит 5000 руб. за 1 м куб. Двухкамерный пакет стоит 6000 руб. за 1м куб. Размер подоконника это ширина*20 см. Стоимость подоконника равна стоимости стеклопакета. По кнопке «ОК» выдать стоимость стеклопакета. Выполнять контроль ввода данных. Внешний вид приложения приведен на рис. 45.

Рис. 45

Задача 8. Регистрация клиентов отеля

Разработать форму (рис. 46) с помощью которой вводится информация о клиентах отеля. После заполнения всех полей высчитывается поле «Сумма». Стоимость одно-местного номера – 990 рублей в день, стоимость двухместного номера – 850 руб. в день, стоимость люкса – 1100 рублей в день, завтрак в номере стоит 250 рублей. Стоимость зависит от продолжительности проживания в днях и рассчитывается по следующей формуле: $\langle \text{продолжительность} \rangle * \langle \text{стоимость} \rangle + \langle \text{продолжительность} \rangle * \langle \text{завтрак} \rangle$.

По кнопке «Добавить» полная информация о новом клиенте выводится в окне MessageBox. По кнопке «Отмена» - очищаются поля формы. По кнопке «Выход» - форма закрывается. Выполнять контроль ввода данных.

Рис. 46

Задача 9. Расчет стоимости тура

Разработать форму следующего вида (рис. 47) и рассчитать стоимость тура:

Рис. 47

Задача 10. Контроль ввода данных

1. В поле формы типа TextBox введен адрес электронной почты. Проверить правильность введенного адреса. Использовать класс Char и его статические методы или возможности работы со строками. Как известно адрес электронной почты может содержать буквы, цифры, символ @, точки. Например:

kentavr@ci.nsu.ru

2. Проверить правильность введения номера телефона в поле формы типа TextBox. Номер телефона может содержать цифры, скобки, черточки. Например:
8(383)347-68-90
8-913-240-38-57

Примерный вид формы приведен на рис. 48:

Рис. 48

Работа с файлами

В C# определяются классы байтовых потоков и классы потоков символов. Однако классы потоков символов в действительности представляют собой оболочку, которая преобразует на более низком уровне поток байтов в поток символов. Причем выполнение этого преобразования происходит автоматически. Таким образом, символьные потоки построены на основе байтовых потоков и являются их логическим дополнением.

Все классы потоков определены в пространстве имен System.IO. Для применения этих классов в начале программы обычно помещают следующий оператор:

```
using System.IO
```

Для консольного ввода-вывода нет необходимости в указании класса `System.IO`. Класс `Console` определен в пространстве имен `System`.

Чтение и запись байтов данных с помощью класса `FileStream`

Для работы с байтовым потоком необходимо создать объект класса `FileStream`. Конструктор класса `FileStream` имеет два параметра: первый задает местоположение файла, второй режим открытия файла:

```
FileStream <объект класса> = new FileStream(<путь к файлу>, <режим>);
```

Режим открытия задается с помощью перечисления `FileMode`, который задает набор констант для определения режимов открытия. В табл. 42 приведены некоторые режимы:

Таблица 42

Режим	Описание
Append	Открытие существующего файла или создание нового файла в режиме добавления.
Create	Создание файла. Если файл уже существует, то он будет переписан. Если файл не существует лучше использовать <code>CreateNew</code> или <code>Truncate</code> .
CreateNew	Создание нового файла, если файл существует, то вырабатывается исключение <code>IOException</code> .
Open	Открытие существующего файла.
OpenOrCreate	Открытие существующего файла или создание нового файла.
Truncate	Существующий файл открывается и усекается до нулевого размера.

Класс **`FileStream`** определяет два метода, которые позволяют считывать байты из файла: **`ReadByte`** и **`Read`**. Для считывания одного байта используется метод `ReadByte`, общая форма которого выглядит так:

```
int ReadByte();
```

Каждый раз при вызове этого метода из файла считывается один байт, который возвращается в виде целочисленного значения. Если при этом встречается признак конца файла, то возвращается значение -1(минус 1). В случае ошибки генерируется исключение `IOException`.

Для считывания блока байтов используется метод `Read()`:

```
int Read(<буфер>, <смещение>, <число байт>);
```

Метод `Read` пытается считать байты, количество которых определяется третьим параметром, из файла в буфер, начиная со смещения, заданного вторым параметром. В случае ошибки генерируется исключение `IOException`. Возможны и другие исключения, например `NotSupportedException`, если операция не поддерживается потоком.

Для записи данных в файл используются два метода **`WriteByte`** и **`Write`**. Метод `WriteByte` имеет следующий синтаксис:

`void WriteByte(byte val).`

Этот метод записывает байт, указанный параметром *val* в файл. В случае ошибки генерируется исключение *IOException*. Если поток вывода не открыт, то генерируется исключение *NotSupportedException*.

Байтовый массив данных записывается с помощью метода *Write*. Синтаксис метода:

`int Write(byte[] buf, int offset, int numbytes)`

Метод *Write* записывает в файл количество байтов, указанное параметром *numbytes*, из массива *buf*, начиная с позиции *offset*. При этом возвращается количество записанных байтов.

Без использования буферизации можно применять метод *flush()*.

В следующем примере выполняется копирование файла. Имена исходного и целевого файлов задаются в полях формы (рис. 49):

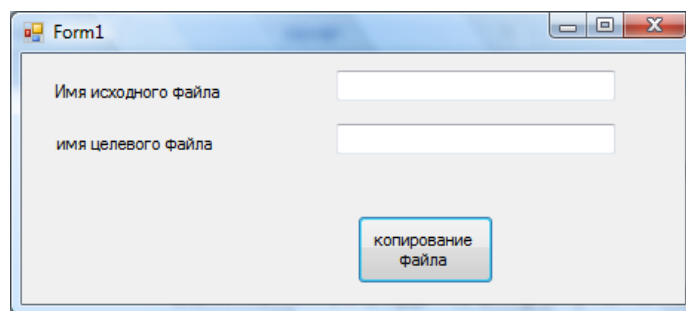


Рис. 49

```
int i;
FileStream fin;
FileStream fout;
string fromFile = textBox1.Text;
string toFile = textBox2.Text;

try
// открытие исходного файла
{
    fin = new FileStream(fromFile, FileMode.Open);
}
catch (FileNotFoundException exc)
{
    MessageBox.Show(exc.Message);
    return;
}
// открытие файла приемника
try
{
    fout = new FileStream(toFile, FileMode.Create);
}
catch (FileNotFoundException exc)
{
    MessageBox.Show("Ошибка создания файла");
    return;
}
// копирование файла
try
{
    do
    {
```

```

        i = fin.ReadByte();
        if (i != -1) fout.WriteByte((byte)i);
    } while (i != -1);
}
catch (IOException exc)
{
    MessageBox.Show(exc.Message + "    файловая ошибка");
}

fin.Close();
fout.Close();
}
}

```

Ввод/вывод в символьные файлы. Использование класса *StreamWriter*

Для хранения текста в формате *Unicode* наилучшим выбором будет использование символьных потоков. Как правило при выполнении файловых операций, основанных на использовании символов, класс *FileStream* включается в класс *StreamReader* или *StreamWriter*. Эти классы обеспечивают автоматическое преобразование байтовых потоков в символьные и наоборот.

На уровне операционной системы файл остается набором байтов. Это остается справедливым и для классов *StreamReader* или *StreamWriter*. В следующем примере выполняется запись двух строк на диск.

```

FileStream fout;
try
{ // открытие байтового потока
fout = new FileStream("d:/c#/text.txt", FileMode.Create );
}
catch (IOException exc)
{
    MessageBox.Show(exc.Message); return;
}
// на основе байтового потока создается символьный поток
StreamWriter fstrout = new StreamWriter(fout);
// в который записывается две строки
string str1="первая строка", str2="вторая строка ";
fstrout.Write(str1);
fstrout.Write(str2);
fout.Close();
fstrout.Close();

```

Использование класса *StreamReader*

Для создания символьного потока ввода байтовый поток включается в символьный поток *StreamReader*. Класс *StreamReader* имеет несколько конструкторов. Наиболее часто используется следующий конструктор:

StreamReader(Stream stream)

Где параметр *stream* определяет имя открытого байтового потока. Если значение параметра *stream* равно 0, генерируется исключение *ArgumentNullException*. Сразу после создания класс *StreamReader* будет автоматически выполнять преобразование байтов в символы. В следующем примере считываем файл и отображаем его на экране:

```

FileStream fin;
string s;
try
{ // создается байтовый поток
  fin = new FileStream("d:/c#/number.txt", FileMode.Open);
}
catch (IOException exc)
{
  MessageBox.Show("Невозможно открыть файл");
  return;
}
// создание символьного потока на основе байтового потока
StreamReader fstr_in = new StreamReader(fin);
while ((s = fstr_in.ReadLine()) != null)
  label3.Text += s;
fstr_in.Close();

```

Если в файле присутствуют русские буквы, то перед созданием потока необходимо задать нужную кодировку. Кодовая страница для русских букв – 1251. Поэтому необходимо добавить следующий код:

```

Encoding e1 = Encoding.GetEncoding(1251);
StreamReader fstr_in = new StreamReader(fin, e1);

```

Произвольный доступ к содержимому файла

В C# возможен не только последовательный, но и произвольный доступ к содержимому файла. Для реализации подобного доступа используется метод *Seek()*, определенный в классе *FileStream*. Этот метод позволяет задавать индикатор позиции текущего элемента данных в файле(который также называется файловым указателем). Синтаксис метода таков:

Long Seek(long newPos, SeekOrigin origin)

Здесь параметр *newPos* указывает новую позицию в байтах для файлового указателя, начиная от местоположения, указанного параметром *origin*. Этот параметр может иметь следующие значения:

Begin – отсчет от начала файла
Current – отсчет от текущего местоположения.
End – отсчет от конца файла.

После вызова метода *Seek* следующие операции чтения-записи производятся в новой позиции файлового указателя. Если во время позиционирования указателя произойдет ошибка, генерируется исключение *IOException*. Если поток не поддерживает операцию позиционирования, генерируется исключение *NotSupportedException*.

Примеры работы с файлами

Задание 1. Создайте Windows Forms- приложение, которое создает файл numbers.txt и записывает в него пять введенных пользователем целых чисел. Посмотрите с помощью текстового редактора созданный файл.

Решение.

```

FileStream fout;
try
{
  // открытие файла в режиме создания
  fout = new FileStream("d:/c#/number.txt", FileMode.Create);
}

```



```

    }
    catch (IOException exc)
    {
        MessageBox.Show(exc.Message);
        return;
    }
    StreamWriter fstrout = new StreamWriter(fout);
    // запись в файл

    string str1 = textBox1.Text;
    fstrout.Write(str1);
    fstrout.Close();

```

Задание 2. Созданное в предыдущем пункте приложение дополните возможностью дописывать в конец файла пять введенных пользователем целых чисел. Вывести содержимое файла в форме.

Решение: для записи в конец символьного файла необходимо открыть его в режиме *Append*. Напоминаем, что метод *Seek* для потока *StreamWriter* не поддерживается.

```

FileStream fout;
    try
    {
        fout = new FileStream("d:/ita/c#/number.txt", FileMode.Append);
    }
    catch (IOException exc)
    {
        MessageBox.Show(exc.Message); return;
    }
    StreamWriter fstrout = new StreamWriter(fout);
    // запись в конец файла

    string str1 = textBox1.Text;
    fstrout.Write(str1);
    fstrout.Close();

```

Для записи на другую строку нужно использовать метод *WriteLine*.

Задание 3. Напишите Windows Forms-приложение, которое вычисляет среднее арифметическое чисел, находящихся в файле *numbers.txt* и выводит это значение в форме.

Решение:

```

// прочитать файл
FileStream fin;
string s="";
try
{
    fin = new FileStream("c:/temp/number.txt", FileMode.Open);
}
catch (IOException exc)
{
    MessageBox.Show(exc.Message + " Невозможно открыть файл");
    return;
}
StreamReader fstr_in = new StreamReader(fin);
while ((s = fstr_in.ReadLine()) != null)
    label5.Text += s;
    fstr_in.Close();

// посчитать ср арифм чисел из строки s
char ch; int sum=0;

```

```

s = label5.Text;
for (int i = 0; i < s.Length; i++)
{
    ch=s[i];
    if (ch == ' ') continue;
    sum += Int16.Parse(ch.ToString());
}

label5.Text = sum.ToString();

```

Задание 4. Создайте Windows Forms-приложение, которое дописывает в файл phone.txt имя, фамилию и номер телефона, например, вашего товарища. Если файла на диске нет, то программа должна его создать.

Решение:

```

// запись телефона друга, каждый с новой строки
FileStream fout;
try
{
    // дописать строку в конец файла
    fout = new FileStream("c:/temp/phone.txt", FileMode.Append);
}
catch (IOException exc)
{
    MessageBox.Show(exc.Message); return;
}
StreamWriter fstrout = new StreamWriter(fout);

// запись в конец файла

string fam = textBox3.Text;
string im = textBox4.Text;
string tel = textBox5.Text;
string str1 = fam + " " + im + " " + tel;
fstrout.WriteLine(str1);
MessageBox.Show("Запись завершена");
fstrout.Close();

```

Задание 4. Создайте Windows-приложение, которое позволяет найти в телефонном справочнике (файл phone.txt) телефон человека по его фамилии. Если в справочнике есть люди с одинаковыми фамилиями, программа должна вывести список всех этих людей.

Решение:

```

// в переменной fam - фамилия человека, которого
// нужно найти в справочнике
string fam = textBox6.Text;
// открыть файл
FileStream fin;
try
{
    fin = new FileStream("c:/temp/phone.txt", FileMode.Open);
}
catch (IOException exc)
{
    MessageBox.Show(exc.Message); return;
}
StreamReader fstr_in = new StreamReader(fin);
string s="";

while ((s = fstr_in.ReadLine()) != null)
{
    if (s.IndexOf(fam)!=-1)
        label11.Text += s;
}

fstr_in.Close();

```

Задание 5. Результаты сдачи экзаменов

В файле input.txt содержатся сведения о сдаче экзаменов учениками 9-х классов некоторой средней школы. В первой строке сообщается количество учеников N, которое не меньше 10, но не превосходит 100, каждая из последующих N строк имеет следующий формат:

<Фамилия> <Имя> <оценки>

где <Фамилия> – строка, состоящая не более чем из 20 символов, <Имя> – строка, состоящая не более чем из 15 символов, <оценки> – через пробел три целых числа, соответствующие оценкам по пятибалльной системе. <Фамилия> и <Имя>, а также <Имя> и <оценки> разделены одним пробелом. Пример входного файла:

12

Иванов	Петр	4	5	3
Сидоров	Иван	3	3	4
Петрова	Катя	5	4	4
Соловьев	Данил	3	5	5
Петухова	Наташа	4	4	4
Курочкин	Денис	5	5	5
Воробьева	Света	4	3	4
Скворцова	Дарья	3	3	3
Иволгин	Данил	5	4	5
Синицина	Люда	3	4	3
Галкин	Петр	5	4	3
Дроздов	Сергей	4	4	5

Требуется написать программу, которая будет выводить на экран фамилии трех худших по среднему баллу учеников. Если среди остальных есть ученики, набравшие тот же средний балл, что и один из трех худших, то следует вывести и их фамилии. Фамилии выводятся в файл output.txt в одну строку, через пробел. Фамилии выводятся в порядке их следования во входном файле.

Решение

Создадим структуру puple, состоящую из трех полей:

```
// структура учащиеся
struct puple
{
    public string fam;
    public string im;
    public byte sr_ball;
};
```

Создадим массив структур, состоящий из 100 элементов:

```
puple[] mas_pup = new puple[100];
// byte sr_ball = new byte[100];

// считать данные из файла input.txt в массив структур
FileStream fin;
string s = "", s1="";
try
{
    fin = new FileStream("c:/Users/c#/input.txt", FileMode.Open);
}
catch (IOException exc)
{
}
```

```

        MessageBox.Show(exc.Message + " Невозможно открыть файл");
        return;
    }
    int i=0, poz1, N;

    // задать правильную кодировку
    Encoding e1 = Encoding.GetEncoding(1251);
    // открыть символьный поток для чтения
    StreamReader fstr_in = new StreamReader(fin, e1);

    // прочитали первую строку файла - кол-во учащихся
    s = fstr_in.ReadLine();
    N = Int16.Parse(s);
    // вывести все строки из файла input.txt в элементе label1
    while ((s = fstr_in.ReadLine()) != null)
    { // считали очередную строку файла
        label2.Text += s + '\n';
        // делаем разбор строки
        poz1 = s.IndexOf(" ");
    // определили фамилию
        mas_pup[i].fam = s.Substring(0, poz1);
        s1 = s.Substring(poz1+1);
        poz1 = s1.IndexOf(" ");
    // определили имя
        mas_pup[i].im = s1.Substring(0, poz1);

        // посчитали суммарный балл
        mas_pup[i].sr_ball += Byte.Parse(s1.Substring(poz1 + 1, 1));
        mas_pup[i].sr_ball += Byte.Parse(s1.Substring(poz1 + 3, 1));
        mas_pup[i].sr_ball += Byte.Parse(s1.Substring(poz1 + 5, 1));
        i++;
    }

    // отсортировать массив структур в порядке возрастания сумм балла
    string fam1 = "", im1 = "";
    int sr1 = 0, min = 0, k=0;
    for (i = 0; i < mas_pup.Length; i++)
    { min = mas_pup[i].sr_ball;
        for (int j = i+1; j < mas_pup.Length; j++)
        { if (mas_pup[j].sr_ball < min)
            {
                k = j; min = mas_pup[j].sr_ball ;
            }
        }
        if (k > 0)
        {
            fam1 = mas_pup[k].fam;
            im1 = mas_pup[k].im;
            sr1 = mas_pup[k].sr_ball;

            mas_pup[k].fam = mas_pup[i].fam;
            mas_pup[k].im = mas_pup[i].im;
            mas_pup[k].sr_ball = mas_pup[i].sr_ball;

            mas_pup[i].fam = fam1;
            mas_pup[i].im = im1;
            mas_pup[i].sr_ball = (byte)sr1;
            k = 0;
        }
    }

    // вывести всех учащихся в порядке возрастания суммарного балла
    // в элементе label1
    for (i=0; i<mas_pup.Length ; i++)

```

```

label1.Text+= mas_pup [i].fam + " "+mas_pup[i].im + " "+
                                                    mas_pup[i].sr_ball.ToString()+"\n";
fstr_in.Close();

```

Задания для самостоятельной работы

1. Создайте Windows Forms-приложение, которое позволяет добавлять нового игрока в файл Igrok1.txt. В файле хранится имя игрока и количество набранных им очков в виде:

```

Volchek 12
Molodoy 15
Avaterisk 20
SuperStar 10

```

В случае если игрок с таким именем уже существует, необходимо увеличить число набранных им очков на количество очков, введенное пользователем.

2. Содержимое файла dohod.txt имеет следующий вид:

```

Доходы:
100
11.12.2009
200
15.05.2010
Расходы:
150
23.09.2010

```

Структура файла dohod.txt жестко задана. На следующей строке после слова «Доход» сумма дохода в рублях, затем дата дохода и т.д. После слов «Расход» на следующей строке приведена сумма расхода, затем на следующей строке дата расхода.

Требуется вывести содержимое файла в форму и посчитать общий баланс: «доход» минус «расход». Примерный вид формы приведен на рис. 50.

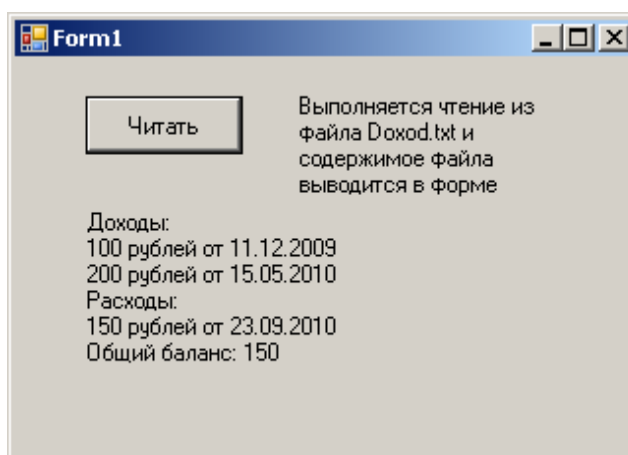


Рис. 50

Делегаты

Делегат это прямой аналог указателя на функцию, но делегата отличает типовая защищенность, безопасность и полное соответствие концепциям объектно-ориентированного программирования.

Делегаты это объекты, производные от общего базового класса *System.Delegate*. Этот тип являет собой некую сущность, которую можно вызывать. В случае с экземпляром объекта - эта сущность состоит из экземпляра и его метода, в случае статического метода - из класса и статического метода этого класса. Именно благодаря этому не нарушатся концепции объектно-ориентированного программирования, а именно не используются методы объекта в отрыве от самого объекта.

Сначала несколько слов о том, что такое делегаты. В языке программирования могут быть переменные, содержащие значения целого или вещественного типов. С такими типами все хорошо знакомы. Так вот, в первом приближении делегаты - это тоже некоторый тип данных. Что же за тип может храниться в делегатах? В делегатах могут храниться функции (вернее указатели на функции). Т. е. переменной типа делегат можно сначала присвоить одну функцию, поработать с ней, а затем присвоить другую функцию. Но, во-первых, делегаты бывают разных типов. Тип делегата определяется типом и количеством параметров и типом возвращаемого значения. Это значит, что если, например, делегат типа `void` и у него только один параметр типа `int`, то в такой делегат мы можем записать только функцию типа `void` и с одним единственным параметром типа `int`. Во-вторых, делегаты, в отличие от указателей на функции языка C++, представляют собой безопасные типы данных. Рассмотрим простой пример делегата:

```
//Объявление делегата.
delegate float MyFunc(float x);
//Класс для тестирования делегата.
class Test
{
    //Первая функция.
    static float f1(float x)
    {
        return x;
    }
    //Вторая функция.
    static float f2(float x)
    {
        return x * x;
    }
    //Третья функция.
    static float f3(float x)
    {
        return (float)Math.Sqrt(x);
    }
    //Метод Main.
    static void Main()
    {
        //Создаем экземпляр делегата.
        MyFunc f = new MyFunc(f1);
        int a; //Ответ пользователя.

        Console.WriteLine("Выберите функцию:\n1.f1=x.\n2.f2=x*x.\n3.f3=sqrt(x).");

        //Преобразуем ответ пользователя к типу Int32
        a = Int32.Parse(Console.ReadLine());
        switch (a)
        {
            case 1:
                //Записываем в делегат 1-ю функцию.
```

```

        f = new MyFunc(f1);
        break;
    case 2:
        //Записываем в делегат 2-ю функцию.
        f = new MyFunc(f2);
        break;
    case 3:
        //Записываем в делегат 3-ю функцию.
        f = new MyFunc(f3);
        break;
    }
    //Печатаем результат.
    int x = 4;
    Console.WriteLine("f({0})={1}", x, f(x));
    Console.ReadLine();
}
}
}

```

В этой программе мы сначала объявляется делегат:

```

//Объявление делегата.
delegate float MyFunc(float x);

```

Синтаксис объявления делегата такой: сначала пишем ключевое слово *delegate*, затем - тип возвращаемого значения (*float* в нашем примере), потом - произвольное имя делегата (у нас это *MyFunc*), после которого в круглых скобках перечисляем параметры (у нас только один параметр типа *float*).

Далее мы в нашем тестовом классе объявляем несколько методов - *f1*, *f2* и *f3*. Все эти методы имеют тип *float* и один параметр типа *float* (как и у делегата *MyFunc*). Функции делают свой подсчет по-разному - первая просто возвращает параметр, вторая - квадрат параметра, третья - корень из параметра.

Далее мы в методе *Main* создаем экземпляр нашего делегата и в зависимости от ответа пользователя записываем в него *f1*, *f2* или *f3*. Потом в строке

```

...
Console.WriteLine("f({0})={1}", x, f(x));
...

```

выводим значение делегата при некотором *x* (равном 4 в нашем примере). Вернее сказать, мы выводим не значение делегата, а значение функции, которую мы записали в делегат.

В зависимости от ответа пользователя в делегат запишется тот или иной вариант функции, и программа выведет само значение *x*, в нашем примере равное 4, *x* умножить на *x* (равное 16) или корень из *x* (равный 2). Результат выполнения программы приведен на рис. 51.

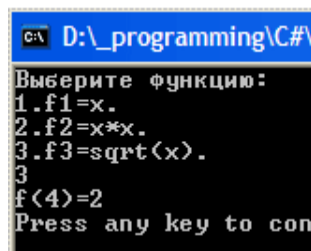


Рис. 51

Создание графических объектов

Перед тем как начать рисовать линии, фигуры, выводить текст и манипулировать другими графическими объектами, необходимо создать графическую поверхность. Графическая поверхность представляет собой поверхность на форме или элементе управления, и именно она, как холст, используется для рисования и создания графических изображений. Для создания графической поверхности необходимо выполнить следующие шаги:

- Создать графическую поверхность;
- Использовать ее для рисования.

Создание графической поверхности

Графическую поверхность можно создать одним из следующих способов:

- Получить ссылку на графическую поверхность как параметр события *Paint* в форме или элементе управления.
- Вызовом метода *CreateGraphics* формы или элемента управления для получения ссылки на графическую поверхность.
- Путем создания графической поверхности из любого объекта, который унаследован из *Image*. Этот подход полезен, когда Вы хотите изменить уже существующее изображение.

Рассмотрим более подробно все эти способы.

1 способ. Параметр *PaintEventArgs* в событии *Paint*

Обработчик события *Paint* формы в качестве параметра передает объект *e* типа *PaintEventArgs*. Приведем пример обработчика:

```
private void Form1_Paint(object sender, PaintEventArgs e)
{
    // Создание графической поверхности
    Graphics g = e.Graphics;
    // Код для рисования вставляется сюда
}
```

Объект *g* – это графическая поверхность, объект, методы которой позволяют рисовать различные фигуры, линии, поверхности.

2 способ. Метод *CreateGraphics*

Используется метод *CreateGraphics* элемента управления или формы для получения ссылки на графический объект, который позволяет рисовать в форме или по элементу управления.

```
g = this.CreateGraphics();
```

Где *this* – это ссылка на форму.

3 способ. Создание из объекта *Image*

Можно создать графический объект из любого объекта, наследника класса *Image*. Для создания графической поверхности из объекта типа *Image* вызывается метод *Graphics.FromImage*. Например:


```
// создается объект типа Bitmap
Bitmap myBitmap = new Bitmap(@"C:\temp\myPic.bmp");
Graphics g = Graphics.FromImage(myBitmap);
```

Рисование и манипулирование поверхностями и изображениями

После создания графической поверхности можно рисовать линии, фигуры, поверхности. Для этого используются следующие классы, инкапсулированные внутри графической поверхности (объекта):

- Класс **Pen** - используется для рисования линий, поверхностей или других геометрических представлений.
- Класс **Brush** – используется для заполнения фигур, изображений цветом.
- Класс **Font** – обеспечивает текст нужным шрифтом, при этом задается тип шрифта, размер шрифта и т.д.
- Структура **Color** -представляет различные цвета.

Перед рисованием фигур и линий необходимо задать карандаш и кисть. Карандаш задает цвет и толщину линии, кисть задает цвет и тип закрашки.

Карандаш – класс Pen

Создание объекта карандаша:

```
Pen myPenRed = new Pen();
```

Объект имеет следующие основные свойства:

Color – задает цвет карандаша. Например:

```
Color.Red // задается красный цвет.
```

Width – задает толщину карандаша. Например:

```
Pen myPenRed = new Pen();
myPenRed.Color = Color.Red;
myPenRed.Width = 3;
```

Конструктор *Pen* имеет два параметра, первый – цвет, второй – толщина:

```
Pen myPenRed = new Pen(Color.Red, 3);
```

Нарисовать линию. Метод DrawLine

Метод *DrawLine* рисует линию, соединяющую две точки. Метод имеет несколько вариантов использования, говорят перегружен. Различные варианты метода приведены в табл. 43.

Таблица 43

Название метода	Описание
Graphics.DrawLine (Pen, Point, Point)	Рисует линию, соединяющую две точки, заданные с помощью структуры <i>Point</i> .
Graphics.DrawLine (Pen, PointF, PointF)	Рисует линию, соединяющую две точки, заданные с помощью структуры <i>PointF</i> .

Graphics.DrawLine (Pen, Int32, Int32, Int32, Int32)	Рисует линию, соединяющую две точки, заданные парой координат типа <i>Int32</i>
Graphics.DrawLine (Pen, Single, Single, Single, Single)	Рисует линию, соединяющую две точки, заданные парой координат типа <i>Single</i>

Рассмотрим более подробно третий вариант метода. Его синтаксис:

DrawLine(карандаш, x1,y1,x2,y2)

Где *x1,y1* – координаты левого верхнего угла, а *x2,y2* – координаты правого нижнего угла.

Например:

```
Pen myPenGreen = new Pen(Color.Green, 3);
// линия
g.DrawLine(myPenRed, 10, 10, 350, 350);
```

Нарисовать прямоугольник. Метод DrawRectangle

Метод рисует прямоугольник, заданный парой координат вершины, шириной и высотой. Метод перегружен. Различные варианты метода приведены в табл. 44.

Таблица 44

Метод	Описание
Graphics.DrawRectangle (Pen, Rectangle)	Рисует прямоугольник, заданный структурой <i>Rectangle</i> .
Graphics.DrawRectangle (Pen, Int32, Int32, Int32, Int32)	Рисует прямоугольник, заданный координатами левого верхнего угла, шириной и высотой, используется тип <i>Int32</i> .
Graphics.DrawRectangle (Pen, Single, Single, Single, Single)	Рисует прямоугольник, заданный координатами левого верхнего угла, шириной и высотой, используется тип <i>Single</i> .

Например:

```
Graphics g = e.Graphics;
// карандаш
Pen myPenRed = new Pen(Color.Red, 3);
g.DrawRectangle(myPenRed, 15.0F, 15.0F, 250.0F, 250.0F);
```

В результате выполнения этого кода будет нарисован прямоугольник красного цвета, вершина которого находится в точке (15.0F, 15.0F), а ширина и высота равны 250.0F.

Задание для самостоятельной работы

1. Нарисовать прямоугольник синего цвета с диагоналями зеленого цвета.
2. Нарисовать домик в виде прямоугольника зеленого цвета с крышей красного цвета.

Нарисовать эллипс. Метод DrawEllipse

Метод рисует эллипс, вписанный в прямоугольник. Метод перегружен. Различные варианты метода приведены в табл. 45.

Таблица 45

Метод	Описание
Graphics.DrawEllipse (Pen, Rectangle)	Рисует эллипс, вписанный в прямоугольник, границы которого описаны структурой <i>Rectangle</i>
Graphics.DrawEllipse (Pen, RectangleF)	Рисует эллипс, вписанный в прямоугольник, границы которого описаны структурой <i>RectangleF</i> .
Graphics.DrawEllipse (Pen, Int32, Int32, Int32, Int32)	Рисует эллипс, вписанный в прямоугольник.
Graphics.DrawEllipse (Pen, Single, Single, Single, Single)	Рисует эллипс, вписанный в прямоугольник.

Параметры

Pen - карандаш, определяет цвет, ширину и стиль границы эллипса

Rectangle – это структура, которая определяет размер и расположение прямоугольника, в который вписывается эллипс.

Метод значение не возвращает.

Например:

```
Rectangle rect = new Rectangle(0, 0, 200, 100);  
Pen blackPen = new Pen(Color.Black);  
e.Graphics.DrawEllipse(blackPen, rect);
```

Кисть – класс Brush

Brush - это абстрактный класс, от которого происходят различные классы, задающие разные типы кистей (табл. 46).

Таблица 46

Класс	Тип кисти
SolidBrush	задает кисть сплошной заливки
TextureBrush	задает текстурную кисть, закрашивает область растровым изображением
LinearGradientBrush	задает градиентную кисть, обеспечивает плавное смешение двух цветов для получения градиентного перехода.

Пример 1:

```
SolidBrush myBrush1 = new SolidBrush(Color.White);
```

Пример 2:

Создается пользовательская кисть полупрозрачная, серого цвета, которая используется для заполнения прямоугольника

```
Color customColor = Color.FromArgb(50, Color.Gray);  
SolidBrush shadowBrush = new SolidBrush(customColor);
```

Пример 3: Градиентная кисть, обеспечивает плавное смешение красного и желтого цвета:

```
// подключение пространства имен
using System.Drawing.Drawing2D;
...
Rectangle rect = new Rectangle(0, 0, 300, 300);
LinearGradientBrush myBr = new LinearGradientBrush(ClientRectangle,
    Color.Red, Color.Yellow, LinearGradientMode.Vertical);
// заполнить кистью прямоугольник
g.FillRectangle(myBr, rect);
```

Метод FillRectangle. Прямоугольник, заполненный цветом

Метод FillRectangle рисует прямоугольник, который заполняется цветом кисти. Метод перегружен. Различные варианты метода приведены в табл. 47.

Таблица 47

Метод	Описание
Graphics.FillRectangle (Brush, Rectangle)	Заполняет прямоугольник, заданный структурой <i>Rectangle</i> цветом кисти.
Graphics.FillRectangle (Brush, RectangleF)	Заполняет прямоугольник, заданный структурой <i>RectangleF</i> цветом кисти.
Graphics.FillRectangle (Brush, Int32, Int32, Int32, Int32)	Заполняет прямоугольник, заданный парой координат верхнего левого угла, шириной и высотой цветом кисти
Graphics.FillRectangle (Brush, Single, Single, Single, Single)	Заполняет прямоугольник, заданный парой координат верхнего левого угла, шириной и высотой цветом кисти

Например:

```
// 2 способ создания графич объекта
Graphics g;
g = this.CreateGraphics();
// рисуем российский флаг
int x0 = 10, y0 = 10;
int w = 100, h = 40;
Pen PenRed = new Pen(Color.Black, 2);

SolidBrush myBrush1 = new SolidBrush(Color.White);
g.FillRectangle(myBrush1, x0, y0, w, h);

SolidBrush myBrush2 = new SolidBrush(Color.Blue);
g.FillRectangle(myBrush2, x0, y0 + h, w, h);

SolidBrush myBrush3 = new SolidBrush(Color.Red);
g.FillRectangle(myBrush3, x0, y0 + 2 * h, w, h);
```

Замкнутая фигура. Методы DrawPolygon, FillPolygon и FillEllipse

Метод *FillPolygon* позволяет заполнять замкнутую фигуру цветом кисти. Синтаксис:

FillPolygon(кисть, массив)

Где *массив* описывает точки замкнутой фигуры, которая будет заполнена цветом кисти. Например:

```
g.DrawPolygon(myPen,
    new PointF[] {new PointF(1, 1), new PointF(20, 10), new PointF(100, 2),
    new PointF(200, 50)});
```

```
g.FillPolygon(myBrush2, new PointF[] {new PointF(220, 20), new
PointF(100, 100), new PointF(280, 100)});
```

Метод *FillEllipse* заполняет эллипс цветом кисти. Перегруженные варианты метода приведены в табл. 48.

Таблица 48

Метод	Описание
Graphics.FillEllipse (Brush, Rectangle)	Заполняет эллипс, заданный структурой <i>Rectangle</i> цветом кисти.
Graphics.FillEllipse (Brush, RectangleF)	Заполняет эллипс, заданный структурой <i>RectangleF</i> цветом кисти.
Graphics.FillEllipse (Brush, Int32, Int32, Int32, Int32)	Заполняет прямоугольник, заданный парой координат верхнего левого угла, шириной и высотой цветом кисти
Graphics.FillEllipse (Brush, Single, Single, Single, Single, Single)	Заполняет прямоугольник, заданный парой координат верхнего левого угла, шириной и высотой цветом кисти

Например, нарисуем японский флаг: красный эллипс внутри белого прямоугольника:

```
Graphics g = this.CreateGraphics();
int x0 = 150, y0 = 100, w = 300, h = 300;

Pen PenBlk = new Pen(Color.Black);
SolidBrush WhiteBr = new SolidBrush(Color.White);
SolidBrush RedBr = new SolidBrush(Color.Red);

g.DrawRectangle(PenBlk, x0, y0, w, h);
g.FillRectangle(WhiteBr, x0, y0, w, h);

int x01 = x0 + w / 2; int y01 = y0 + h / 2;
int ww = 70;
g.FillEllipse(RedBr, x01 - ww, y01 - ww, 2 * ww, 2 * ww);
```

Рассмотрим еще один пример. Нарисуем елочку, состоящую из зеленых треугольников разной величины с границей желтого цвета.

```
Graphics g = e.Graphics;
Pen PenYell = new Pen(Color.Yellow, 3);
// задать массив точек для рисования замкнутой фигуры
Point[] ArrPoint1 = new Point[] { new Point(200, 10), new Point(300, 60),
new Point(100, 60), new Point(200, 10) };

// Создать кисть для наполнения цветом
SolidBrush BrGreen = new SolidBrush(Color.Green);
```

```

        // контур фигуры
        g.DrawPolygon(PenYell, ArrPoint1);
        // заливка цветом замкнутой фигуры
        g.FillPolygon(BrGreen, ArrPoint1);

        Point[] ArrPoint2 = new Point[] { new Point(200, 60), new Point(330, 110),
        new Point(70, 110), new Point(200, 60) };
        g.DrawPolygon(PenYell, ArrPoint2);
        g.FillPolygon(BrGreen, ArrPoint2);

        Point[] ArrPoint3 = new Point[] { new Point(200, 110), new Point(360, 160),
        new Point(40, 160), new Point(200, 110) };
        g.DrawPolygon(PenYell, ArrPoint3);
        g.FillPolygon(BrGreen, ArrPoint3);

```

Результат выполнения программы приведен на рис. 52.

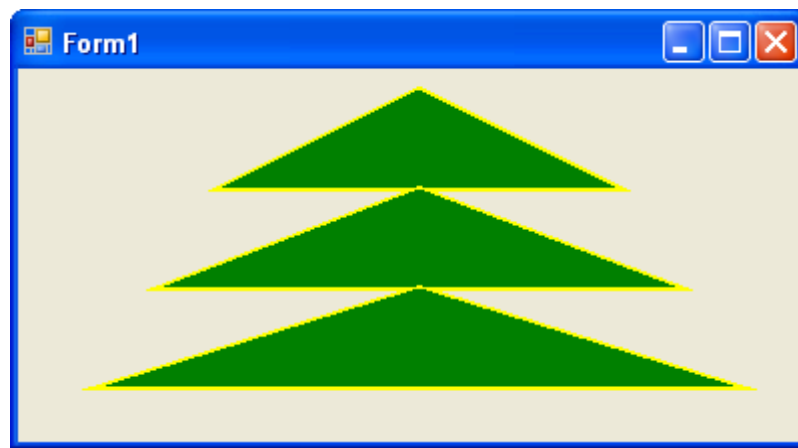


Рис. 52

Рассмотрим еще один пример. Требуется нарисовать концентрические окружности случайного цвета, исходящие из центра экрана. Программный код, выполняющий данную задачу, приведен ниже с комментариями.

```

        Graphics p = e.Graphics;
        int x0, y0, w, h;
        int r, g, b;
        Color col;
        // центр экрана
        x0 = ClientSize.Width / 2;
        y0 = ClientSize.Height / 2;

        // высота и ширина прямоугольника, в который
        // вписывается круг
        w=10; h=10;
        // случайный объект
        Random rnd = new Random(90);
        // объект карандаш
        Pen pn = new Pen(Color.Black,2);

        for (int i = 1; i < 15; i++)
        {
            r = rnd.Next() % 255;
            g = rnd.Next() % 255;
            b = rnd.Next() % 255;
            // метод FromArgb возвращает объект типа Color.
            // в качестве параметров методу указываются значения цветов
            // красного, голубого, зеленого
            col=Color.FromArgb(r,b,g);

```

```

        pn.Color=col;
        // рисуем эллипс заданного цвета
        e.Graphics.DrawEllipse(pn,x0,y0,w,h);
        x0 -= 5; y0 -= 5;
        w += 10; h += 10;
    }
}

```

Пояснения: Свойство формы *ClientSize* позволяет получить размер клиентской части окна, т.е. окна без границ и заголовка. Клиентская часть окна это часть где располагаются элементы управления. Для получения размера всей формы (с заголовком и границами) можно использовать свойство *Size* или свойства *Width* и *Height*.

Результат выполнения этого программного кода приведен на рис. 53.

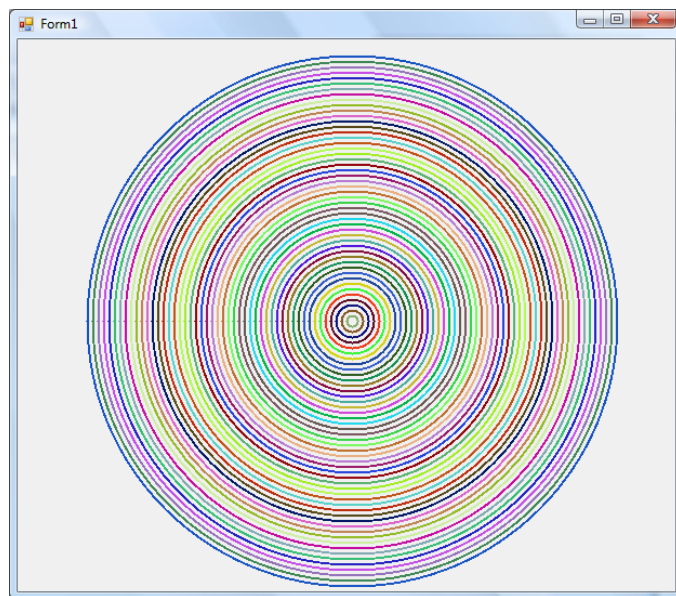


Рис. 53

Задания для самостоятельной работы

1. Нарисовать светофор как изображено на рис. 54.

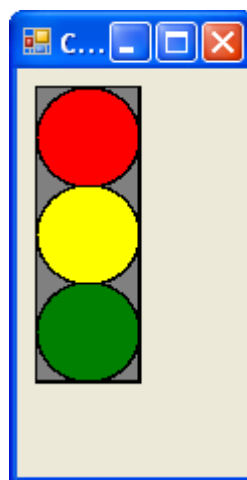


Рис. 54

2. Нарисовать олимпийские кольца.

3. Нарисовать прямоугольники случайного цвета, случайного рамера, расположенные в случайных местах экрана (рис. 55).

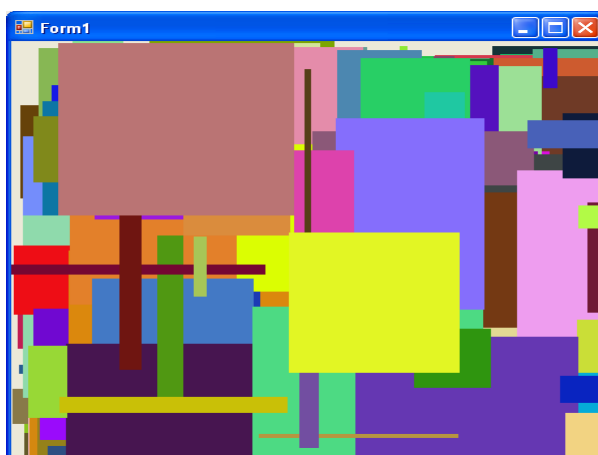


Рис. 55

4. Нарисовать шахматную доску (рис. 56).



Рис. 56

Рисование дуг и секторов на экране

Метод *DrawArc* позволяет нарисовать дугу (хорду) на экране. Метод *FillPie* – позволяет залить цветом сектор, от центра эллипса до хорды. Рассмотрим пример, изображенный на рис. 57. Программный код, реализующий данный рисунок приведен ниже.

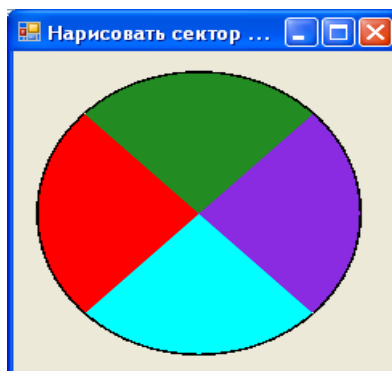


Рис. 57


```

        // Создать карандаш
        Pen blackPen = new Pen(Color.Black, 3);
        // Создать координаты прямоугольника, в который вписывается эллипс
        float x = 15.0F;
        float y = 15.0F;
        float width = 200.0F;
        float height = 200.0F;
        // Создать начальный угол и угол поворота для дуги эллипса
        // угол отсчитывается в градусах от оси абсцисс против часовой стрелки.
        // угол поворота - это угол приращение.
        float startAngle = 45.0F;
        float sweepAngle = 90.0F;

        // Нарисовать бирюзовый сектор
        e.Graphics.DrawArc(blackPen, x, y, width, height, startAngle, sweepAngle);
        SolidBrush br = new SolidBrush(Color.Aqua);
        // залить ее цветом
        e.Graphics.FillPie(br, x, y, width, height, startAngle, sweepAngle);

        // Нарисовать красный сектор
        startAngle = 135.0F;
        sweepAngle = 90.0F;
        // Нарисовать дугу(хорду) на экране
        e.Graphics.DrawArc(blackPen, x, y, width, height, startAngle, sweepAngle);
        // залить ее цветом
        SolidBrush br1 = new SolidBrush(Color.Red);
        e.Graphics.FillPie(br1, x, y, width, height, startAngle, sweepAngle);

        // Нарисовать зеленый сектор
        startAngle = 225.0F;
        sweepAngle = 90.0F;
        // Нарисовать дугу(хорду) на экране
        e.Graphics.DrawArc(blackPen, x, y, width, height, startAngle, sweepAngle);
        // залить цветом
        SolidBrush br2 = new SolidBrush(Color.ForestGreen);
        e.Graphics.FillPie(br2, x, y, width, height, startAngle, sweepAngle);

        // Нарисовать фиолетовый сектор
        startAngle = 315.0F;
        sweepAngle = 90.0F;
        // Нарисовать дугу(хорду) на экране
        e.Graphics.DrawArc(blackPen, x, y, width, height, startAngle, sweepAngle);
        SolidBrush br3 = new SolidBrush(Color.BlueViolet);
        // залить цветом
        e.Graphics.FillPie(br3, x, y, width, height, startAngle, sweepAngle);
    }

```

Метод Clear

Метод Clear – очищает всю поверхность окна и закрашивает его цветом фона. Пример использования приведен ниже:

```

private void button1_Click(object sender, System.EventArgs e)
{
    Graphics g;
    g = this.CreateGraphics();
    g.Clear(Color.Teal);
}

```

Кривая Безье

Метод *DrawBezier* рисует сплайн, определяемый 4-мя точками. Перегруженные варианты метода приведены в табл. 49.

Таблица 49

Метод	Описание
Graphics.DrawBezier (Pen, Point, Point, Point, Point)	Рисует кривую Безье, задаваемую четырьмя точками, описываемыми структурой Point.
Graphics.DrawBezier (Pen, PointF, PointF, PointF, PointF)	Рисует сплайн Белье, определяемый точками структур PointF
Graphics.DrawBezier (Pen, Single, Single, Single, Single, Single, Single, Single)	Рисует сплайн Белье, определяемый 4-мя точками, заданными своими координатами

Кривая Безье это сглаженная кривая, которая строится по точкам (рис. 58).

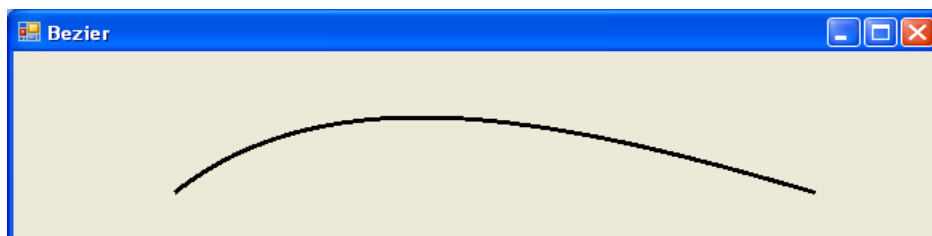


Рис. 58

```
// Создать карандаш
Pen blackPen = new Pen(Color.Black, 3);
// Создать точки для кривой
Point start = new Point(100, 100);
Point control1 = new Point(200, 10);
Point control2 = new Point(350, 50);
Point end = new Point(500, 100);
// Нарисовать кривую на экране
e.Graphics.DrawBezier(blackPen, start, control1, control2, end);
```

Использование мыши

Событиям *MouseDown*, *MouseUp* и *MouseMove* в качестве аргумента передается объект *e*, который обеспечивает эти события данными, текущими координатами (x,y) курсора мыши.

Следующий пример демонстрирует использование событий мыши *MouseDown* и *MouseUp* для управления размером прямоугольника, а также использование метода *Invalidate*. В этом примере можно рисовать прямоугольники, используя мышь. При нажатии мыши задается первая вершина прямоугольника, при отпускании задается вторая вершина прямоугольника, и прямоугольник появляется в окне. Программный код выглядит следующим образом:

```
private Rectangle RcDraw;
private float PenWidth = 5;
```

```

private void Form1_MouseDown(object sender,
                               System.Windows.Forms.MouseEventArgs e)
{
    // Задание начальных координат прямоугольника
    RcDraw.X = e.X;
    RcDraw.Y = e.Y;
}

private void Form1_MouseUp(object sender,
                            System.Windows.Forms.MouseEventArgs e)
{
    // Определение высоты и ширины прямоугольника
    // с помощью координат мыши
    if(e.X < RcDraw.X)
    {
        RcDraw.Width = RcDraw.X - e.X;
        RcDraw.X = e.X;
    }
    else
    {
        RcDraw.Width = e.X - RcDraw.X;
    }

    if(e.Y < RcDraw.Y)
    {
        RcDraw.Height = RcDraw.Y - e.Y;
        RcDraw.Y = e.Y;
    }
    else
    {
        RcDraw.Height = e.Y - RcDraw.Y;
    }

    // иницирует перерисовку области, занятой прямоугольником
    this.Invalidate(RcDraw);
}

private void Form1_Paint(object sender,
                          System.Windows.Forms.PaintEventArgs e)
{
    // Рисует прямоугольник...
    e.Graphics.DrawRectangle(new Pen(Color.Blue, PenWidth), RcDraw);
}

```

Рассмотрим еще пример. На форме программно создается элемент управления *PictureBox*, фон элемента управления делается белого цвета. В этом примере событие *Paint* элемента *PictureBox* связывается с событием *Paint* формы.

```

private PictureBox pictureBox1 = new PictureBox();

private void Form1_Load(object sender, System.EventArgs e)
{
    // Внедрить PictureBox на форму и сделать его фон белого цвета
    pictureBox1.Dock = DockStyle.Fill;
    pictureBox1.BackColor = Color.White;

    // событие Paint элемента PictureBox связывается с событием Paint формы.
    pictureBox1.Paint += new
        System.Windows.Forms.PaintEventHandler(this.pictureBox1_Paint);

    // Добавить элемент PictureBox на форму
    this.Controls.Add(pictureBox1);
}

```

```

private void pictureBox1_Paint(object sender,
                                System.Windows.Forms.PaintEventArgs e)
{
    // Создать локальную версию графического поверхности для PictureBox
    Graphics g = e.Graphics;
    // Вывести строку на PictureBox.
    g.DrawString("Эта диагональная линия нарисована на элементе управления",
        new Font("Arial", 10), System.Drawing.Brushes.Blue, new Point(30, 30));

    // Нарисовать линию на PictureBox.
    g.DrawLine(System.Drawing.Pens.Red, pictureBox1.Left, pictureBox1.Top,
        pictureBox1.Right, pictureBox1.Bottom);
}

```

Вывод текста на экран. Метод DrawString

Вывод текста на экран выполняется с помощью метода *DrawString*. Метод выводит текст на экран заданным шрифтом (*Font*), заданным цветом (*Brush*), в заданном месте. Описание метода приведено в табл. 50.

Таблица 50

Описание метода	Пояснение
Graphics.DrawString (String, Font, Brush, PointF)	Рисует текстовую строку(<i>String</i>), заданной кистью (<i>Brush</i>) и заданным шрифтом(<i>Font</i>), в заданной позиции (<i>PointF</i>).

Пример 1:

```

// Создать ссылку на графич объект:
Graphics g = this.CreateGraphics();

Font currFont = new Font("Times New Roman", 24);
g.DrawString("Тестирование шрифта... ", currFont,
    new SolidBrush(Color.BlueViolet), 0, 0);

```

Пример 2:

```

// Создать ссылку на графич объект:
Graphics g = this.CreateGraphics();

// Создать кисть, например градиентную:
System.Drawing.Drawing2D.LinearGradientBrush myBrush = new
    System.Drawing.Drawing2D.LinearGradientBrush(ClientRectangle,
        Color.Red, Color.Yellow,
        System.Drawing.Drawing2D.LinearGradientMode.Horizontal);

// Создать шрифт
Font myFont = new Font("Times New Roman", 48);

// Вызвать метод Graphics.DrawString объекта Graphics для создания
// текста: текст внутри прямоугольника
g.DrawString("Look at this text!", myFont, myBrush,
    new RectangleF(10, 10, 100, 200));

// текст начинается в точке (10,10)
g.DrawString("Look at this text!", myFont, myBrush, 10, 10);

```

Работа с цветом

Существует набор системных цветов, которые берутся из структуры *Color*. Пример:

```
Color myColor;  
myColor = Color.Red;  
myColor = Color.Aquamarine;  
myColor = Color.LightGoldenrodYellow;  
myColor = Color.PapayaWhip;  
myColor = Color.Tomato;
```

Можно создавать свои цвета, задавая интенсивность каждого цвета с помощью метода *Color.FromArgb*. Первый аргумент этого метода задает интенсивность красного, второй – интенсивность зеленого, и последний – интенсивность голубого цвета. Пример:

```
Color myColor;  
myColor = Color.FromArgb(23, 56, 78);
```

Прозрачность

В одном из перегруженных вариантов метода *FromArgb* первый параметр задает прозрачность объекта. Прозрачность задается от 0 до 255. Например:

```
Color myColor;  
myColor = Color.FromArgb(127, 23, 56, 78);
```

В этом примере создается голубовато-серый цвет с примерно 50% прозрачностью.

Можно создать также цвет на основе предыдущего цвета, например:

```
Color myColor;  
myColor = Color.FromArgb(128, Color.Tomato);
```

Работа с изображениями

Класс *Image* имеет большое количество свойств и методов для работы с изображениями. Следующие свойства задают:

Width – ширину изображения

Height – высоту изображения

Size – размер изображения.

Объявленные переменные *Image* присваиваются объектам класса *Bitmap*. Кроме того, можно создавать объекты класса *Bitmap* напрямую. Например,

1. Объявим переменные типа *Image*:

```
private Image Bit1;  
private Image Bit2;  
private Image Bit3;
```

2. Создадим объекты класса *Bitmap*. Конструктор *Bitmap* возвращает объект типа *Image*.

```
Bit1 = new Bitmap("f1.bmp");  
Bit2 = new Bitmap("f2.bmp");
```

3. Выведем изображения на экран с помощью метода *DrawImage*

```
e.DrawImage(Bit1, 10, 10, 100, 100);
```

Полностью код выглядит так:

```

Graphics p = e.Graphics;
Image Bit1;
Bit1 = new Bitmap("C:/C#/Image1/f1.bmp");
p.DrawImage(Bit1, 10, 10, 100, 100);

```

Перемещение изображения с помощью стрелок

Следующий пример демонстрирует перемещение изображения в форме с помощью стрелок.

Создаем переменные, видимые для всех методов класса:

```

// изображение
private Image Bit1;
// начальное положение изображения
int x0=10,y0=10,w=100,h=100;

```

На событие *Paint* формы пишем обработчик, который с помощью метода *DrawImage* помещает изображение в прямоугольник с координатами $(x0,y0,w,h)$.

```

private void Form1_Paint(object sender, PaintEventArgs e)
{
    Graphics p = e.Graphics;
    Bit1= new Bitmap("D:/C#/pilrc.bmp");
    p.DrawImage(Bit1,x0,y0,w,h);
}

```

На событие *KeyDown* формы пишем обработчик:

```

private void Form1_KeyDown(object sender, KeyEventArgs e)
{ // стрелка вниз
    if ((e.KeyCode == Keys.Down) && (y0 + h <= ClientSize.Height))
    {
        y0 += 10;
        Invalidate();
    }
    // стрелка вверх
    if ((e.KeyCode == Keys.Up) && (y0 - 10 >= 0))
    {
        y0 -= 10;
        Invalidate();
    }
    // стрелка вправо
    if ((e.KeyCode == Keys.Right) && (x0 + w <= ClientSize.Width))
    {
        x0 += 10;
        Invalidate();
    }
    // стрелка влево
    if ((e.KeyCode == Keys.Left) && (x0 - 10 >= 0))
    {
        x0 -= 10;
        Invalidate();
    }
}

```

Обработчику через параметр «e» передается код нажатой клавиши. Анализируя нажатия клавиш смещаем изображение, изменяя его начальные координаты. Метод *validate* используется для перерисовки окна. Свойство *ClientSize* задает размеры клиентской части окна:– высоту (*Height*) и ширину (*Width*). Таким образом, отслеживаются попытки выхода за границы окна.

Список литературы

1. Википедия
2. Культин Н. С# в примерах и задачах, Изд-во «БХВ-Петербург», Санкт-Петербург, 2007
3. Intuit.ru
4. Шилдт Г., С# учебный курс, Изд-во «БХВ-Петербург», Санкт-Петербург, 2002
5. Троелсен Э., С# и платформа .Net. Библиотека программиста. Издательство Питер, 2006